

计算机网络面试题，63道计算机网络八股文（2.2万字80张手绘图），面渣逆袭必看



👤 三分恶 📅 2022年7月11日 📄 面渣逆袭 💡 面渣逆袭 📖 约 30749 字 ⏱ 大约 103 分钟

2.2 万字 80 张手绘图，详解 63 道计算机网络面试高频题（让天下没有难背的八股），面渣背会这些计算机网络八股文，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，戳[转载链接 \(https://mp.weixin.qq.com/s/FvxyiMyq0422yifcyoG8vg\)](https://mp.weixin.qq.com/s/FvxyiMyq0422yifcyoG8vg)，作者：三分恶，戳[原文链接 \(https://mp.weixin.qq.com/s/yAlErlC09GnjaVvwUo3Acg\)](https://mp.weixin.qq.com/s/yAlErlC09GnjaVvwUo3Acg)。

大家好，我是二哥呀，今天继续来给大家分享三弟的面渣逆袭！

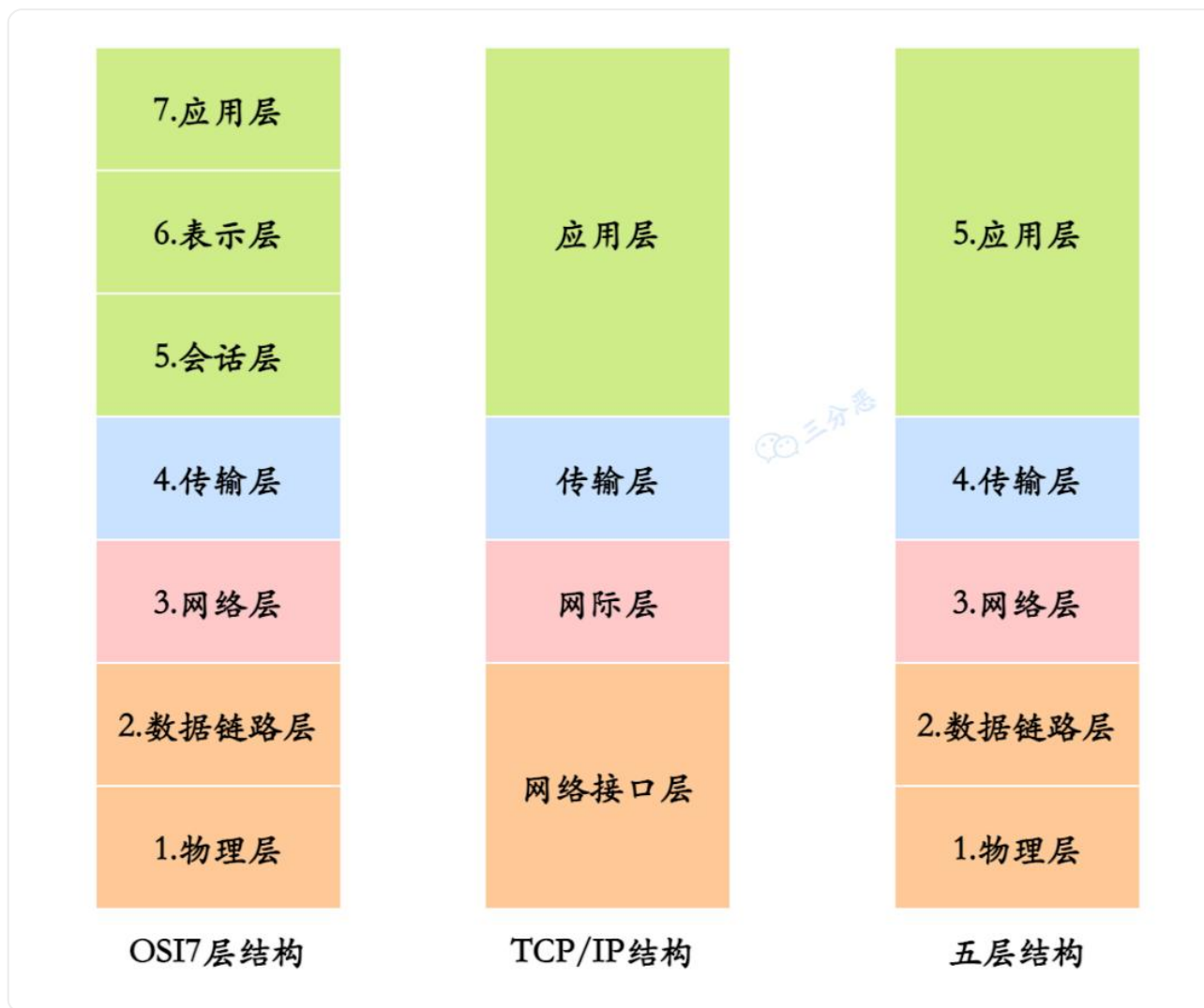
这次带来的是计算机网络六十二问，三万字，七十图详解，大概是全网最全的计算机网络面试题。

建议大家收藏了慢慢看，秋招、春招、金九银十、金三银四冲！

基础

1. 说下计算机网络体系结构

计算机网络体系结构通过将复杂的网络通信分解成不同的层次，来标准化交互的过程。常见的模型包括 OSI 七层模型、TCP/IP 四层模型和五层体系结构。



三分恶面渣逆袭：三种网络体系结构

OSI 是理论上的网络通信模型，TCP/IP 是实际应用层面上的网络通信模型，五层结构是为了方便理解和记忆。

说说 OSI 七层模型？

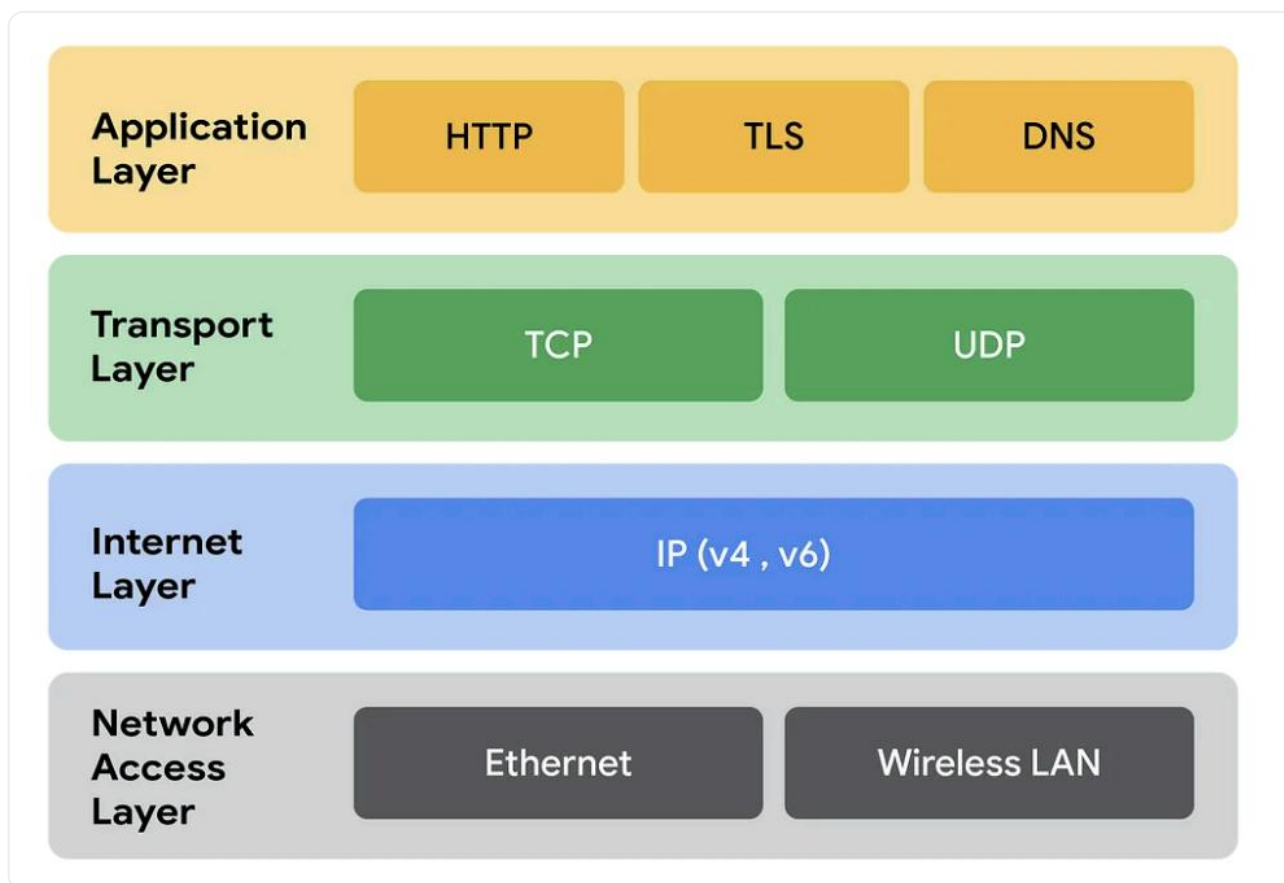
OSI（Open System Interconnection）七层参考模型是一个网络架构模型，由国际标准化组织（ISO）提出，用于描述和标准化各种计算机网络的功能和过程。这七层从低到高分别是：

- **应用层：**最靠近用户的层，负责处理特定的应用程序细节。这一层提供了网络服务与用户应用软件之间的接口。例如，Web 浏览器、FTP 客户端和服务端、电子邮件客户端等。
- **表示层：**确保从一个系统发送的信息可以被另一个系统的应用层读取。它负责数据的转换、压缩和加密。例如，确保数据从一种编码格式转换为另一种，如 ASCII 到 EBCDIC。
- **会话层：**管理用户的会话，控制网络上两节点间的对话和数据交换的管理。它负责建立、维护和终止会话。例如，建立一个会话令牌，以便在网络上的两个节点之间传递。

- **传输层**：提供端到端的通信服务，保证数据的完整性和正确顺序。这一层包括 TCP 和 UDP 等。
- **网络层**：负责在多个网络之间进行数据传输，确保数据能够在复杂的网络结构中找到从源到目的地的最佳路径。这层使用的是 IP (Internet Protocol) 协议。
- **数据链路层**：在物理连接中提供可靠的传输，负责建立和维护两个相邻节点间的链路。包括帧同步、MAC（媒体访问控制）。
- **物理层**：负责在物理媒介上实现原始的数据传输，比如电缆、光纤和无线信号传输。涉及的内容包括电压、接口、针脚、电缆的规格和传输速率等。

说说 TCP/IP 四层模型？

TCP/IP 四层模型是互联网通信的核心，定义了一系列协议和标准，确保设备间可以可靠地进行数据传输。



medium: Victor Aaron Winnercoz

①、**应用层 (Application Layer)**：直接面向用户和应用程序，提供各种网络服务。它包含了用于特定应用的协议和服务，如 HTTP (HyperText Transfer Protocol)、FTP (File Transfer Protocol)、SMTP (Simple Mail Transfer Protocol) 等。

示例：当在浏览器中输入一个 URL 并访问一个网页时，浏览器使用 HTTP 协议从 Web 服务器请求页面内容。

②、**传输层 (Transport Layer)**：提供端到端的通信服务，确保数据可靠传输。它负责分段数据、流量控制、错误检测和纠正。常见的传输层协议有 TCP 和 UDP。

示例：当发送一封电子邮件时，TCP 协议确保邮件从你的客户端可靠地传输到邮件服务器。

③、**网际层**：或者叫网络层 (Internet Layer)，负责在不同网络之间路由数据包，提供逻辑地址 (IP 地址) 和网络寻址功能。用于处理数据包的分组、转发和路由选择，确保数据可以从源端传输到目标端。

常见协议：IPv4、IPv6、ICMP (Internet Control Message Protocol)。

示例：当访问一个网站时，网络层协议 (如 IPv4) 将你的请求从你的计算机通过多个路由器传输到目标服务器。

④、**网络接口层 (Network Access Layer)**：或者叫链路层 (Link Layer)，负责将数字信号在物理通道 (网线) 中准确传输，定义了如何在单一网络链路上传输数据，如何处理数据帧的发送和接收，包括物理地址 (MAC 地址) 的解析。

常见协议：以太网 (Ethernet)、Wi-Fi。

示例：在一个局域网 (LAN) 中，计算机通过以太网连接交换机，链路层协议负责数据帧在网络设备间的传输。

说说五层体系结构？

是对 OSI 和 TCP/IP 的折衷，它保留了 TCP/IP 的实用性，同时提供了比四层模型更细致的分层，便于教学和理解网络的各个方面。

- 应用层：作为网络服务和最终用户之间的接口。它提供了一系列供应用程序使用的协议，如 HTTP (网页)、FTP (文件传输)、SMTP (邮件传输) 等。使用户的应用程序可以访问网络服务。
- 传输层：提供进程到进程的通信管理，这一层确保数据按顺序、无错误地传输。主要协议包括 TCP 和 UDP。
- 网络层：负责数据包从源到目的地的传输和路由选择，包括跨越多个网络 (即互联网)。它使用逻辑地址 (如 IP 地址) 来唯一标识设备。路由器是网络层设备。
- 数据链路层：确保从一个节点到另一个节点的可靠、有效的数据传输。交换机、网桥是数据链路层设备。
- 物理层：电缆、光纤、无线电频谱、网络适配器等。

TCP三次握手四次挥手工作在哪一层？

三次握手和四次挥手都是工作在传输层。传输层（Transport Layer）是 OSI 模型的第四层，负责提供端到端的通信服务，包括数据传输的建立、维护和终止。

TCP 作为一种面向连接的协议，通过三次握手建立连接，通过四次挥手终止连接，确保数据传输的可靠性和完整性。

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的
华为 OD 面经同学 1 一面面试原题：说一些 OSI 七层参考模型

2. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的
京东面经同学 2 后端面试原题：TCP三次握手四次挥手工作在哪一层？

2.说一下每一层对应的网络协议有哪些？

一张表格总结常见网络协议：

OSI七层网络模型	TCP/IP四层模型	对应的网络协议
应用层	应用层	HTTP、DNS、FTP、NFS、WAIS、SMIP、Telnet、SNMP
表示层		TIFF、GIF、JPEG、PICT
会话层		RPC、SQL、NFS、NetBIOS、names、AppleTalk
传输层	传输层	TCP、UDP
网络层	网络层	IP、ICMP、ARP、RAPP、RIP、IPX
数据链路层	网络接口层	FDDI、Frame Relay、HDLC、PPP
物理层		EIA/TIA-232、EIA/TIA-499

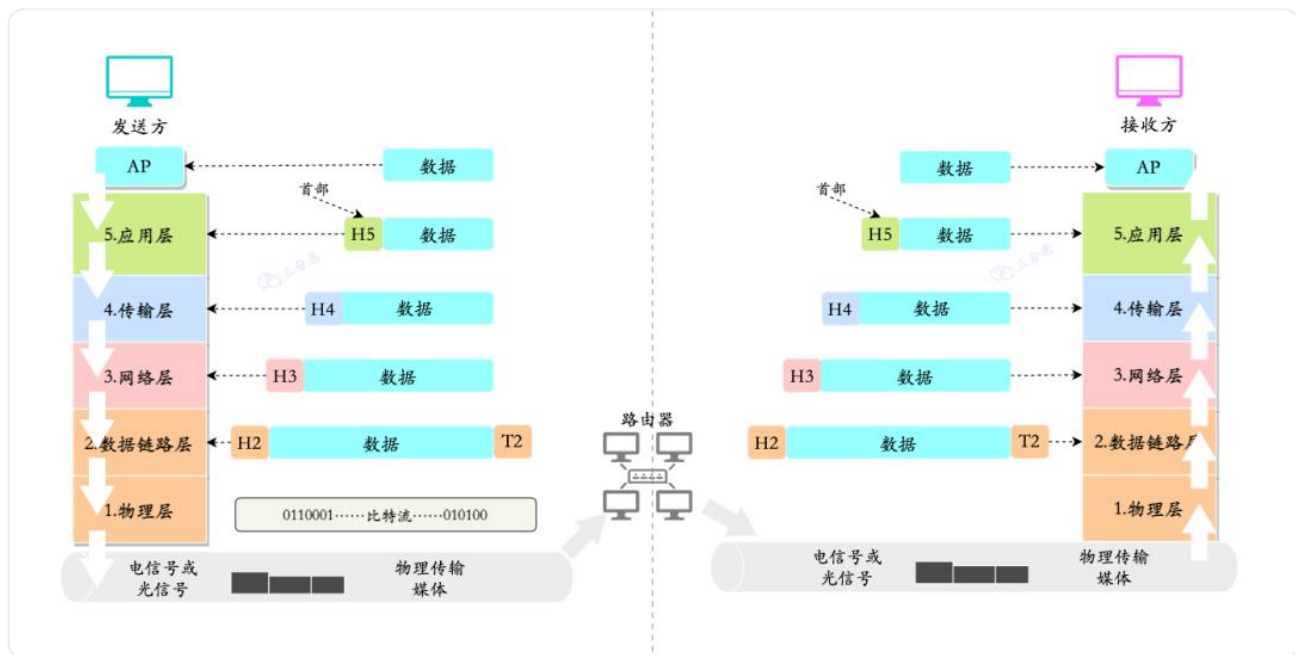
各层网络对应的网络协议

3.那么数据在各层之间是怎么传输的呢？

对于发送方而言，从上层到下层层层包装，对于接收方而言，从下层到上层，层层解开包装。

- 发送方的应用进程向接收方的应用进程传送数据

- AP 先将数据交给本主机的应用层，应用层加上本层的控制信息 H5 就变成了下一层的数据单元
- 传输层收到这个数据单元后，加上本层的控制信息 H4，再交给网络层，成为网络层的数据单元
- 到了数据链路层，控制信息被分成两部分，分别加到本层数据单元的首部（H2）和尾部（T2）
- 最后的物理层，进行比特流的传输



数据在各层之间的传输

这个过程类似写信，写一封信，每到一层，就加一个信封，写一些地址的信息。到了目的地之后，又一层层解封，传向下一个目的地。

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer)

[.https://github.com/itwanger/toBeBetterJavaer](https://github.com/itwanger/toBeBetterJavaer)》第一版 PDF 终于来了！包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等，共计 32 万余字，500+张手绘图，可以说是通俗易懂、风趣幽默.....详情戳：[太赞了，GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/)
[.https://javabetter.cn/overview/](https://javabetter.cn/overview/)。

微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二，回复 **222** 即可免费领取。



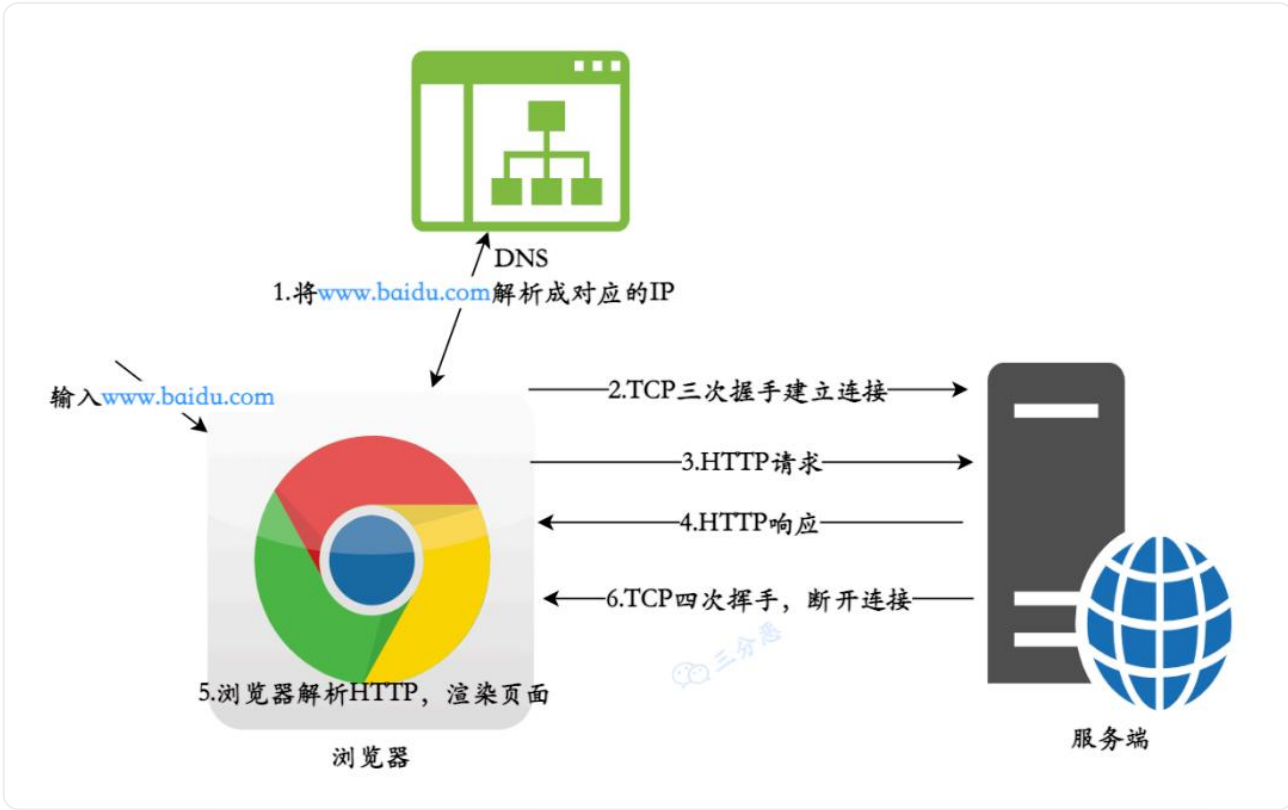
网络综合

4.从浏览器地址栏输入 url 到显示主页的过程?

从在浏览器地址栏输入 URL 到显示主页的过程包括多个步骤, 涵盖了 DNS 解析、TCP 连接、发送 HTTP 请求、服务器处理请求并返回 HTTP 响应、浏览器处理响应并渲染页面等多个环节。

1. **DNS 解析**: 浏览器发起一个 DNS 请求到 DNS 服务器, 将域名解析为服务器的 IP 地址。
2. **TCP 连接**: 浏览器通过解析得到的 IP 地址与服务器建立 TCP 连接 (通常是通过 443 端口进行 SSL 加密的 HTTPS 连接)。这一步涉及到 TCP 的三次握手过程, 确保双方都准备好进行数据传输。
3. **发送 HTTP 请求**: 浏览器构建 HTTP 请求消息, 包括请求行 (如 GET / HTTP/1.1)、请求头 (包含用户代理、接受的内容类型等信息) 和请求体 (如果有); 将请求发送到服务器。
4. **服务器处理请求**: 服务器接收到 HTTP 请求后, 根据请求的资源路径, 经过后端处理 (可能包括数据库查询等), 生成 HTTP 响应消息; 响应消息包括状态行 (如 HTTP/1.1 200 OK)、响应头 (内容类型、缓存控制等信息) 和响应体 (请求的资源内容)。
5. **浏览器接收 HTTP 响应**: 浏览器接收到服务器返回的 HTTP 响应数据, 开始解析响应体中的 HTML 内容; 然后构建 DOM 树、解析 CSS 和 JavaScript 文件等, 最终渲染页面。
6. **断开连接**: TCP 四次挥手, 连接结束

我们以输入 www.baidu.com (<http://www.baidu.com>) 为例：



三分恶面渣逆袭：www.baidu.com (<http://www.baidu.com>) URL 到显示主页

各个过程都使用了哪些协议？

过程	使用的协议
1.浏览器查找域名的IP地址 (DNS查找过程：浏览器缓存、本地DNS路由缓存、DNS解析服务)	DNS:获取域名对应的IP
2.浏览器和服务端TCP三次握手，建立连接	TCP: 与服务端建立连接和断开连接
3.浏览器向服务端发起一个HTTP请求	IP: 使用TCP协议时，网络层需要使用IP协议。
4.服务端处理请求，返回HTTP响应	OPSF: IP数据包在路由器之间，路由选择使用OPSF协议
5.浏览器解析并渲染页面	ARP: 路由器再与服务器通信时，需要将IP地址转换为MAC地址，需要使用ARP协议
6.断开连接	HTTP: TCP连接建立完成之后，使用HTTP协议传递HTTP报文

三分恶面渣逆袭：www.baidu.com (<http://www.baidu.com>)_URL 到显示主页过程使用的协议

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>)_收录的字节跳动商业化一面的原题：url 请求的全过程（要求详细）
2. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>)_收录的字节跳动面经同学 9 飞书后端技术一面面试原题：输入 URL 会发生什么

5.说说 DNS 的解析过程？

DNS 的全称是 **Domain Name System**，也就是域名解析系统，它可以将域名映射到对应的 IP 地址上，比如说我们访问 www.javabetter.cn (<http://www.javabetter.cn>)，实际上访问的是我在阿里云上一台丐版服务器，它的 IP 地址是 xxx.xxx.xxx.xxx (<http://xxx.xxx.xxx.xxx>)。

当然了，也可以通过 IP 地址直接访问服务器，但不方便记忆，所以就有了域名系统。一个好的域名可以卖好多好多钱，像 javabetter.cn (<http://javabetter.cn>) 这个域名，一年需要 39 块钱。

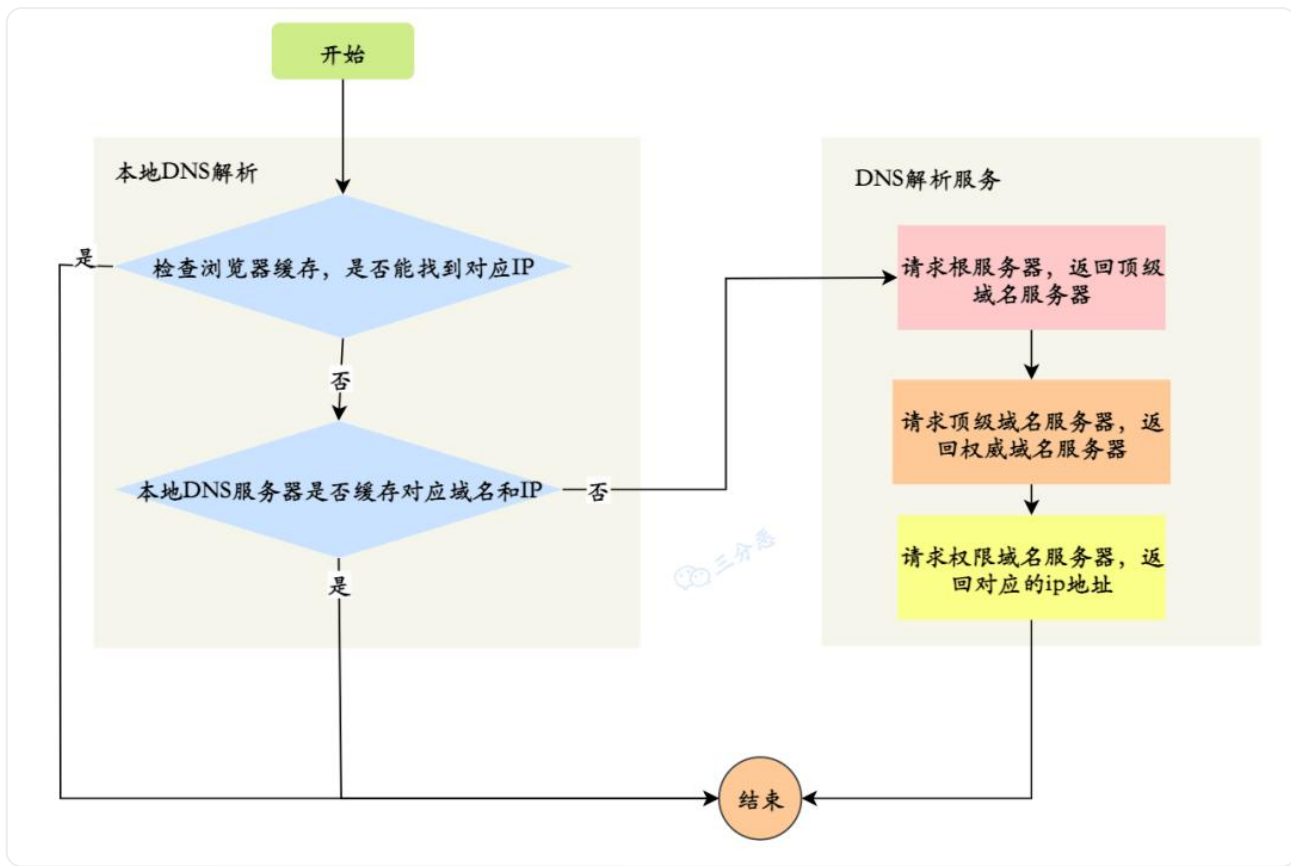
域名到 IP 之间的映射，就需要 DNS 来完成。

详细英文注册信息如下

```
Domain Name: javabetter.cn
ROID: 20230301s10001s51447050-cn
Domain Status: ok
Registrant: REDACTED FOR PRIVACY
Registrant Contact Email: REDACTED FOR PRIVACY
Sponsoring Registrar: 阿里云计算有限公司 (万网)
Name Server: dns25.hichina.com
Name Server: dns26.hichina.com
Registration Time: 2023-03-01 10:38:51
Expiration Time: 2033-03-01 10:38:51
DNSSEC: unsigned
```

二哥的 Java 进阶之路：javabetter.cn (<http://javabetter.cn>)

我来说说 DNS 的解析过程吧：



三分析面渣逆袭：DNS 解析流程

假设我们在浏览器地址栏里键入了 [paicoding.com \(https://paicoding.com\)](https://paicoding.com)：

浏览器会首先检查自己的缓存中是否有这个域名对应的 IP 地址，如果有，直接返回；如果没有，进入下一步。



检查本地 DNS 缓存是否有该域名的记录。如果没有，向**根域名服务器**发送请求，根域名服务器将请求指向更具体的服务，如 com 顶级域名服务器。

顶级域名服务器再将请求指向权威域名服务器, 通常由域名注册机构直接管理, `paicoding.com` 是在阿里云上注册的, 所以阿里云会提供对应的 DNS 解析服务, 将域名和阿里云服务器绑定起来。

最终, 浏览器使用获得的 IP 地址发起一个 HTTP 请求到目标服务器, 然后该服务器返回所请求的网页内容。

1. [Java 面试指南 \(付费\) \(https://javabetter.cn/zhishixingqiu/mianshi.html\)](https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的华为面经同学 6 Java 通用软件开发一面面试原题: 说说 DNS 的解析过程

6.说说 WebSocket 与 Socket 的区别?

- Socket 其实就是等于 **IP 地址 + 端口 + 协议**。

具体来说, Socket 是一套标准, 它完成了对 TCP/IP 的高度封装, 屏蔽网络细节, 以方便开发者更好地进行网络编程。

- WebSocket 是一个持久化的协议, 它是伴随 H5 而出的协议, 用来解决 **http 不支持持久化连接**的问题。
- Socket 一个是**网编编程的标准接口**, 而 WebSocket 则是应用层通信协议。

7.说一下你了解的端口及对应的服务？

端口	服务
21	FTP(文件传输协议)
22	SSH
23	Telnet(远程登录服务)
53	DNS域名解析服务
80	HTTP超文本传输协议
443	HTTPS
1080	Sockets
3306	MySQL默认端口号

常见端口和服务

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了! 包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等, 共计 32 万余字, 500+张手绘图, 可以说是通俗易懂、风趣幽默.....详情戳: [太赞了, GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/) (<https://javabetter.cn/overview/>).

微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二, 回复 **222** 即可免费领取。



HTTP

8.说说 HTTP 常用的状态码及其含义?

HTTP 响应状态码是由服务器返回给客户端, 用于表示对请求的响应结果。

这些状态码分为五个不同的类别, 每个类别用一个数字开头, 共有三位数:

- 1XX: 临时的响应, 客户端应继续请求。
- 2XX: 请求已成功被服务器接收。
- 3XX: 用来重定向。
- 4XX: 请求可能出错。
- 5XX: 服务器在尝试处理请求时发生了错误。

状态码	含义
101	切换请求协议
200	请求成功
301	请求资源永久移动，返回新URI
302	请求资源临时移动，继续使用原有URI
400	客户端请求的语法错误，服务端无法理解
401	当前请求需要认证
403	服务端理解请求，但拒绝执行
500	服务器内部错误

三分恶面渣逆袭：常见 HTTP 状态码

说一下 301 和 302 的区别？

- 301：永久性移动，请求的资源已被永久移动到新位置。服务器返回此响应时，会返回新的资源地址。
- 302：临时性移动，服务器从另外的地址响应资源，但是客户端还应该使用这个地址。

用一个比喻，301 就是嫁人的新垣结衣，302 就是有男朋友的长泽雅美。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>)_收录的字节跳动面经同学 13 Java 后端二面面试原题: http 的响应号有哪些

9.HTTP 有哪些请求方式?

HTTP 协议定义了多种请求方式, 用以指示请求的目的。常见的请求方式有 GET、POST、DELETE、PUT。

请求方式	描述
GET	对服务器获取资源的简单请求
POST	向服务器提交数据请求
PUT	修改指定资源
DELETE	删除URL标记的指定资源
CONNECT	用于代理服务器
TRACE	主要用于回环测试
OPTIONS	返回所有可用的方法
HEAD	获取URL标记资源的首部

三分恶面渣逆袭：HTTP 请求方式

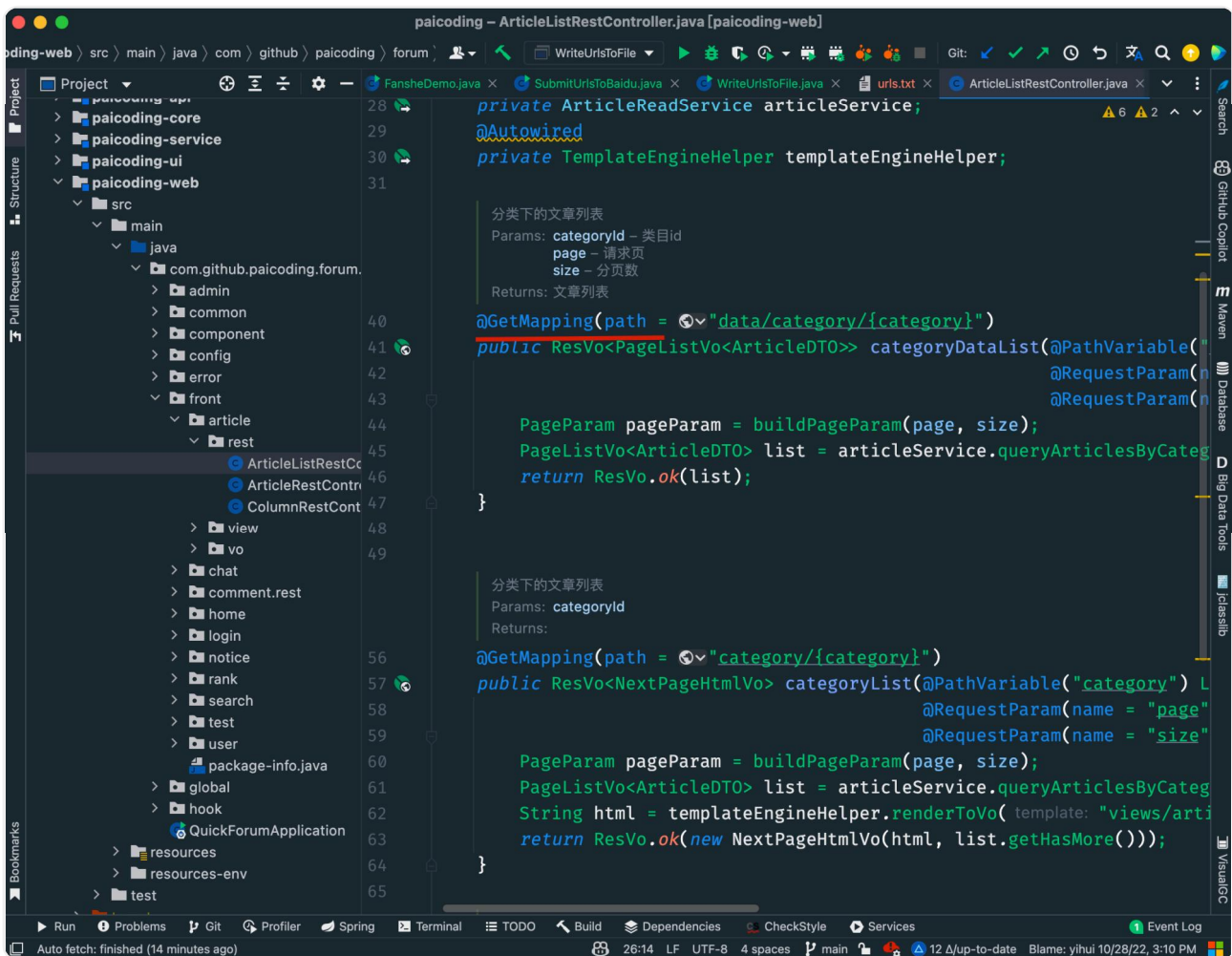
- GET：请求检索指定的资源。应该只用于获取数据，并且是幂等的，即多次执行相同的 GET 请求应该返回相同的结果，并且不会改变资源的状态。
- POST：向指定资源提交数据，请求服务器进行处理（如提交表单或上传文件）。数据被包含在请求体中。可能会创建新的资源或修改现有资源。
- DELETE：删除指定的资源。
- PUT：用于替换指定的资源。如果指定的资源不存在，创建一个新资源。

HTTP 的 GET 方法可以实现写操作吗?

可以是可以, 但是不推荐。

使用 GET 执行写操作可能导致严重的安全问题, 如跨站请求伪造 (CSRF)。

实际开发中, 也应该杜绝使用 GET 方法执行写操作。在[技术派实战项目](https://javabetter.cn/zhishixingqiu/paicoding.html) (<https://javabetter.cn/zhishixingqiu/paicoding.html>) 中, 我们会在接口上明确规定应该使用哪种请求方式。



技术派实战项目源码

客户端一旦使用错误 ✘, 将会收到一个 405 Method Not Allowed 的响应。

什么是幂等? 幂等方法了解哪些?

幂等 (Idempotence) 是一个数学概念, 用于描述某些操作的特性, 即无论操作执行多少次, 结果都是相同的。换句话说, 幂等操作可以重复执行而不会改变系统状态。

如果一个操作是幂等的, 那么对同一资源执行该操作一次和执行多次的效果相同。

在正确实现的条件下, GET、HEAD、PUT 和 DELETE 等方法都是幂等的, 而 POST 方法不是。

例如, GET /pageX HTTP/1.1 幂等的。连续调用多次, 客户端接收到的结果都是一样的:

```
GET /pageX HTTP/1.1
GET /pageX HTTP/1.1
GET /pageX HTTP/1.1
GET /pageX HTTP/1.1
```

text

DELETE /idX/delete HTTP/1.1 是幂等的, 即便是不同请求之间接收到的状态码不一样:

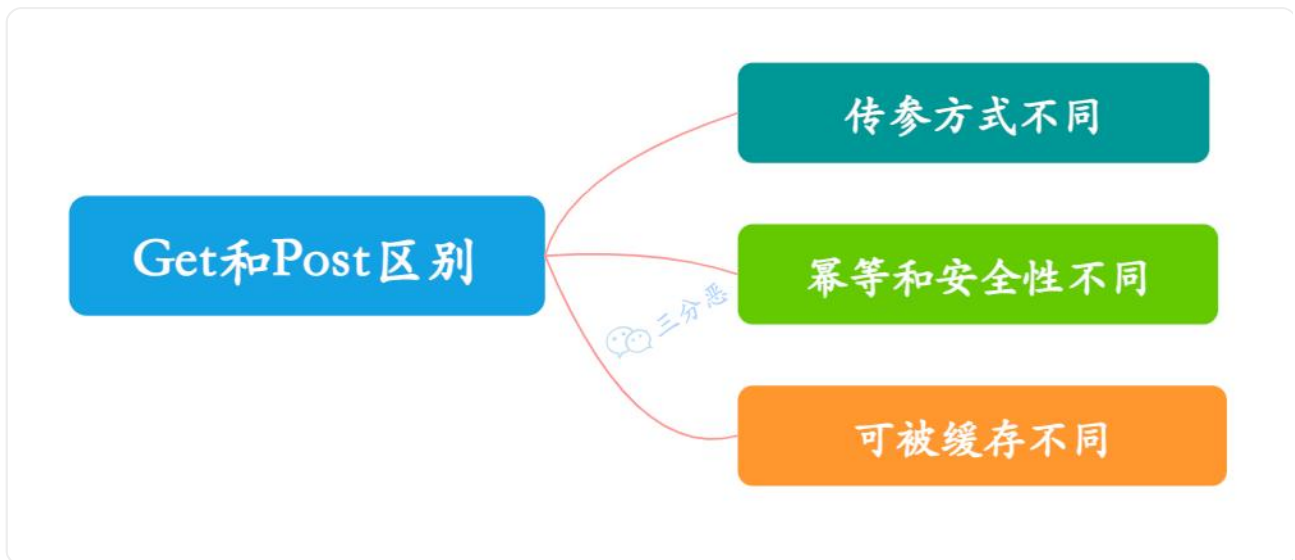
```
DELETE /idX/delete HTTP/1.1 -> Returns 200 if idX exists
DELETE /idX/delete HTTP/1.1 -> Returns 404 as it just got deleted
DELETE /idX/delete HTTP/1.1 -> Returns 404
```

text

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的字节跳动面经同学 13 Java 后端二面面试原题: http 有哪些方法, http 的 get 方法可以实现写操作吗, https 传递 url 安全吗, 为什么数据在浏览器中, 中间人攻击是什么
2. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的字节跳动面经同学 1 技术二面面试原题: 什么是幂等? 幂等方法了解哪些?

10.说一下 GET 和 POST 的区别?

可以从以下几个方面来说明 GET 和 POST 的区别:



Get 和 Post 区别

1. 从 HTTP 报文层面来看，GET 请求将信息放在 URL，POST 将请求信息放在请求体中。这一点使得 GET 请求携带的数据量有限，因为 URL 本身是有长度限制的，而 POST 请求的数据存放在报文体中，因此对大小没有限制。而且从形式上看，GET 请求把数据放 URL 上不太安全，而 POST 请求把数据放在请求体里想比较而言安全一些。
2. 从数据库层面来看，GET 符合幂等性和安全性，而 POST 请求不符合。这个其实和 GET/POST 请求的作用有关。按照 HTTP 的约定，GET 请求用于查看信息，不会改变服务器上的信息；而 POST 请求用来改变服务器上的信息。正因为 GET 请求只查看信息，不改变信息，对数据库的一次或多次操作获得的结果是一致的，认为它符合幂等性。安全性是指对数据库操作没有改变数据库中的数据。
3. 从其他层面来看，GET 请求能够被缓存，GET 请求能够保存在浏览器的浏览记录里，GET 请求的 URL 能够保存为浏览器书签。这些都是 POST 请求所不具备的。缓存是 GET 请求被广泛应用的根本，它能够被缓存也是因为它的幂等性和安全性，除了返回结果没有其他多余的动作，因此绝大部分的 GET 请求都被 CDN 缓存起来了，大大减少了 Web 服务器的负担。

11.GET 的长度限制是多少？

HTTP 中的 GET 方法是通过 URL 传递数据的，但是 URL 本身其实并没有对数据的长度进行限制，真正限制 GET 长度的是浏览器。

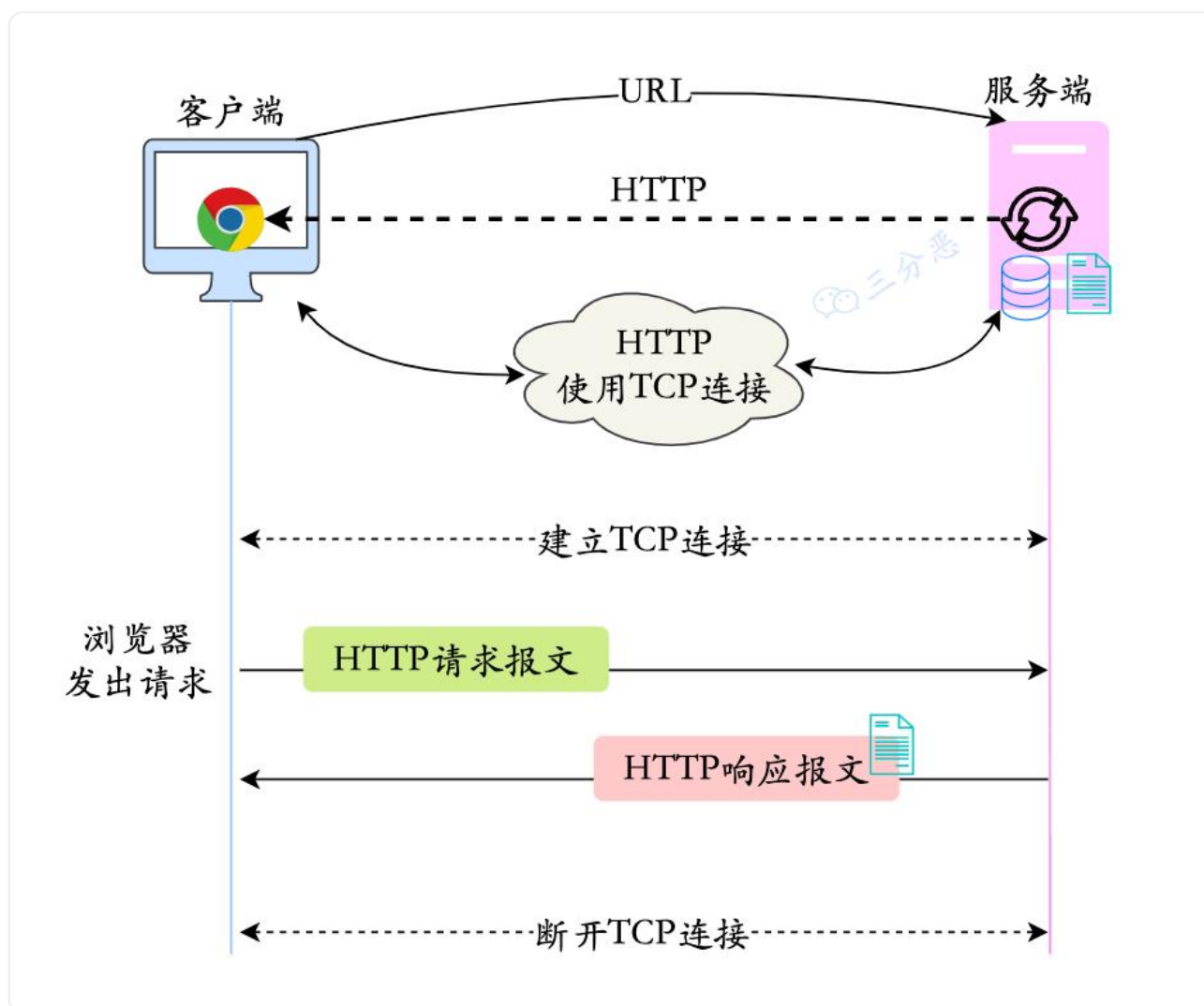
例如 IE 浏览器对 URL 的最大限制是 2000 多个字符，大概 2kb 左右，像 Chrome、Firefox 等浏览器支持的 URL 字符数更多，其中 FireFox 中 URL 的最大长度限制是 65536 个字符，Chrome 则是 8182 个字符。

这个长度限制也不是针对数据部分，而是针对整个 URL。

12.HTTP 请求的过程与原理？

HTTP 全称是超文本传输协议（HyperText Transfer Protocol），是一个基于请求与响应模式的应用层协议，基于 TCP/IP 协议传输数据。

HTTP 遵循标准的客户端-服务器模型，客户端打开连接以发出请求，然后等待它收到服务器端响应。



三分恶面渣逆袭：HTTP 请求的过程和原理

- 在浏览器输入 URL 后，浏览器首先会通过 DNS 解析获取到服务器的 IP 地址，然后与服务器建立 TCP 连接。
- TCP 连接建立后，浏览器会向服务器发送 HTTP 请求。
- 服务器收到请求后，会根据请求的信息处理请求。
- 处理完请求后，服务器会返回一个 HTTP 响应给浏览器。

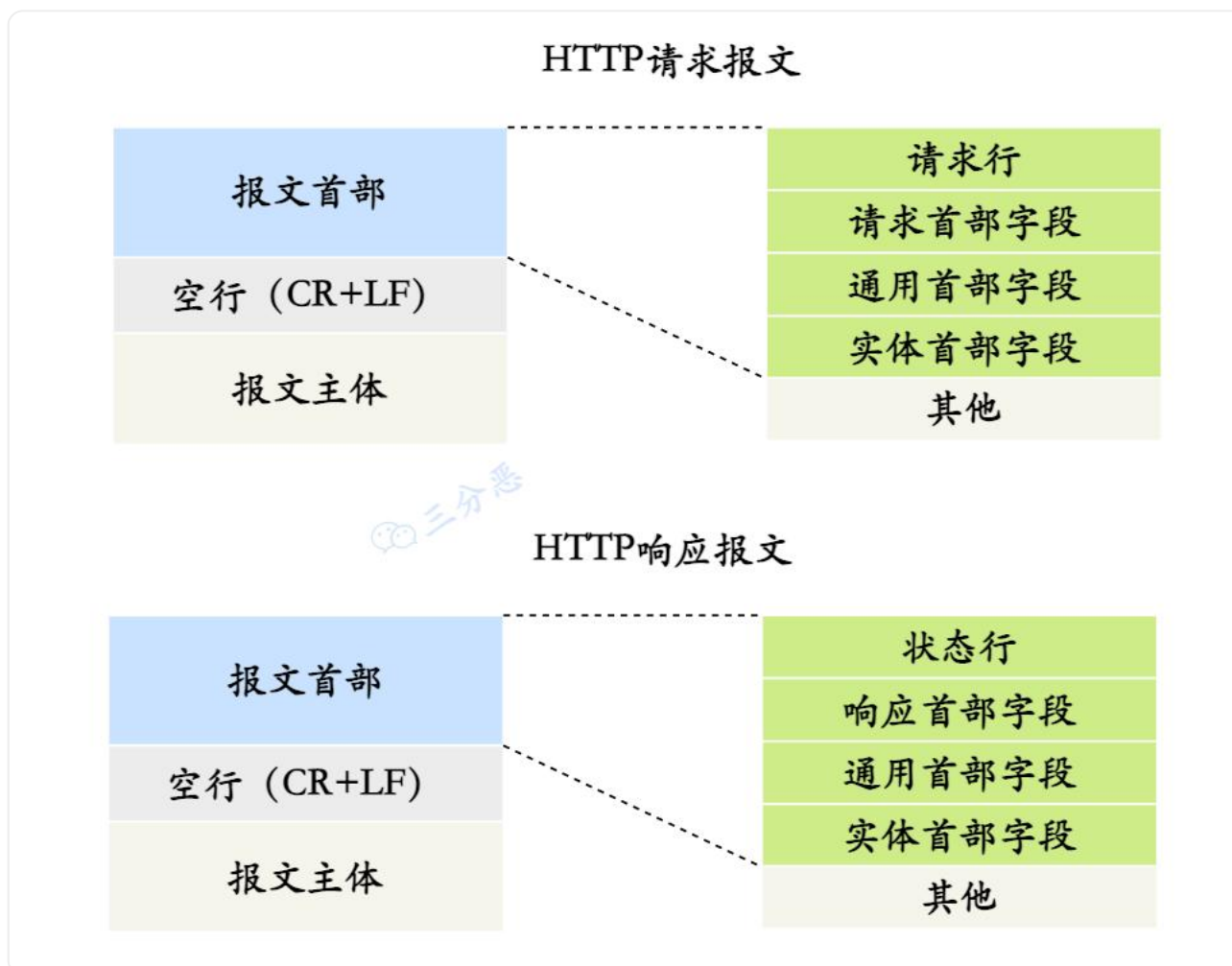
- 浏览器收到响应后，会根据响应的信息渲染页面。然后，浏览器和服务器断开 TCP 连接。

客户端发送一个请求到服务器，服务器处理请求并返回一个响应。这个过程是同步的，也就是说，客户端在发送请求后必须等待服务器的响应。在等待响应的过程中，客户端不会发送其他请求。

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的华为 OD 面经同学 1 一面面试原题：什么是 HTTP？
2. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的招商银行面经同学 6 招银网络科技面试原题：HTTP 传送请求的一次流程

13.说一下 HTTP 的报文结构？

HTTP 的报文结构可以分为两类：请求报文和响应报文。两者在结构上相似，都包含了**起始行、头部**和**消息正文**。



三分恶面渣逆袭：HTTP 报文

说下 HTTP 请求报文结构?

```
GET /index.html HTTP/1.1
Host: www.javabetter.cn
Accept: text/html
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/58.0.3029.110 Safari/537.3
```

http

①、请求行

包括方法 (如 GET、POST) 、请求的 URL 和 HTTP 协议的版本。例如: GET /index.html HTTP/1.1 。

②、请求头部

包含请求的附加信息, 如客户端想要接收的内容类型、浏览器类型等。

请求头部由键值对组成, 键和值之间用冒号分隔, 每一行一个键值对。例如:

- Host: www.javabetter.cn , 表示请求的主机名 (域名)
- Accept: text/html , 表示客户端可以接收的媒体类型
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.3 , 表示客户端的浏览器类型

③、空行

请求头部和消息正文之间有一个空行, 表示请求头部结束。

④、消息正文 (可选)

请求的具体内容, 如 POST 请求中的表单数据; GET 请求中没有消息正文。

说下 HTTP 响应报文结构?

```
HTTP/1.0 200 OK
Content-Type: text/plain
Content-Length: 137582
Expires: Thu, 05 Dec 1997 16:00:00 GMT
Last-Modified: Wed, 5 August 1996 15:55:28 GMT
```

http


```
Server: Apache 0.84
<html>
  <body>沉默王二很天真</body>
</html>
```

①、状态行

包括 HTTP 协议的版本、状态码 (如 200、404) 和状态消息 (如 OK、NotFound)。例如: HTTP/1.0 200 OK。

②、响应头部

包含响应的附加信息, 如服务器类型、内容类型、内容长度等。也是键值对, 例如:

- Content-Type: text/plain, 表示响应的内容类型
- Content-Length: 137582, 表示响应的内容长度
- Expires: Thu, 05 Dec 1997 16:00:00 GMT, 表示资源的过期时间
- Last-Modified: Wed, 5 August 1996 15:55:28 GMT, 表示资源的最后修改时间
- Server: Apache 0.84, 表示服务器类型

③、空行

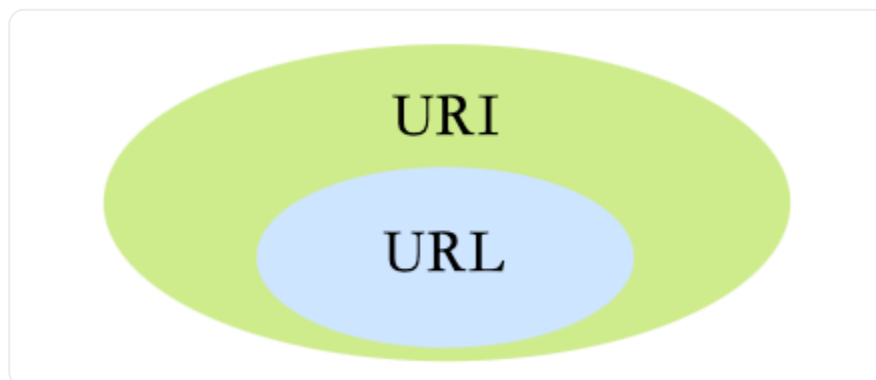
表示响应头部结束。

④、消息正文 (可选)

响应的具体内容, 如 HTML 页面。不是所有的响应都有消息正文, 如 204 No Content 状态码的响应。

1. [Java 面试指南 \(付费\) \(https://javabetter.cn/zhishixingqiu/mianshi.html\)](https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的 比亚迪面经同学 3 Java 技术一面面试原题: 说一下 HTTP 的结构和 HTTPS 的原理
2. [Java 面试指南 \(付费\) \(https://javabetter.cn/zhishixingqiu/mianshi.html\)](https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的 小公司面经合集好未来测开面经同学 3 测开一面面试原题: HTTP 请求消息和响应消息的格式

14.URI 和 URL 有什么区别?



URI 和 URL

- URI, 统一资源标识符(Uniform Resource Identifier, URI), 标识的是 Web 上每一种可用的资源, 如 HTML 文档、图像、视频片段、程序等都是由一个 URI 进行标识的。
- URL, 统一资源定位符 (Uniform Resource Location), 它是 URI 的一种子集, 主要作用是提供资源的路径。

它们的主要区别在于, URL 除了提供了资源的标识, 还提供了资源访问的方式。这么比喻, URI 像是身份证, 可以唯一标识一个人, 而 URL 更像一个住址, 可以通过 URL 找到这个人——人类住址协议://地球/中国/北京市/海淀区/xx 职业技术学院/14 号宿舍楼/525 号寝/张三男。

15.说下 HTTP1.0, 1.1, 2.0 的区别?

如果记不住下面这么多, 可以回答: **HTTP1.0** 默认短连接, HTTP 1.1 默认长连接, HTTP 2.0 采用**多路复用**。

说下 HTTP1.0

- **无状态协议**: HTTP 1.0 是无状态的, 每个请求之间相互独立, 服务器不保存任何请求的状态信息。
- **非持久连接**: 默认情况下, 每个 HTTP 请求/响应对之后, 连接会被关闭, 属于短连接。这意味着对于同一个网站的每个资源请求, 如 HTML 页面上的图片和脚本, 都需要建立一个新的 TCP 连接。可以设置 Connection: keep-alive 强制开启长连接。

说下 HTTP1.1

- **持久连接**：HTTP 1.1 引入了持久连接（也称为 HTTP keep-alive），默认情况下不会立即关闭连接，可以在一个连接上发送多个请求和响应。极大减轻了 TCP 连接的开销。
- **流水线处理**：HTTP 1.1 支持客户端在前一个请求的响应到达之前发送下一个请求，以提高传输效率。

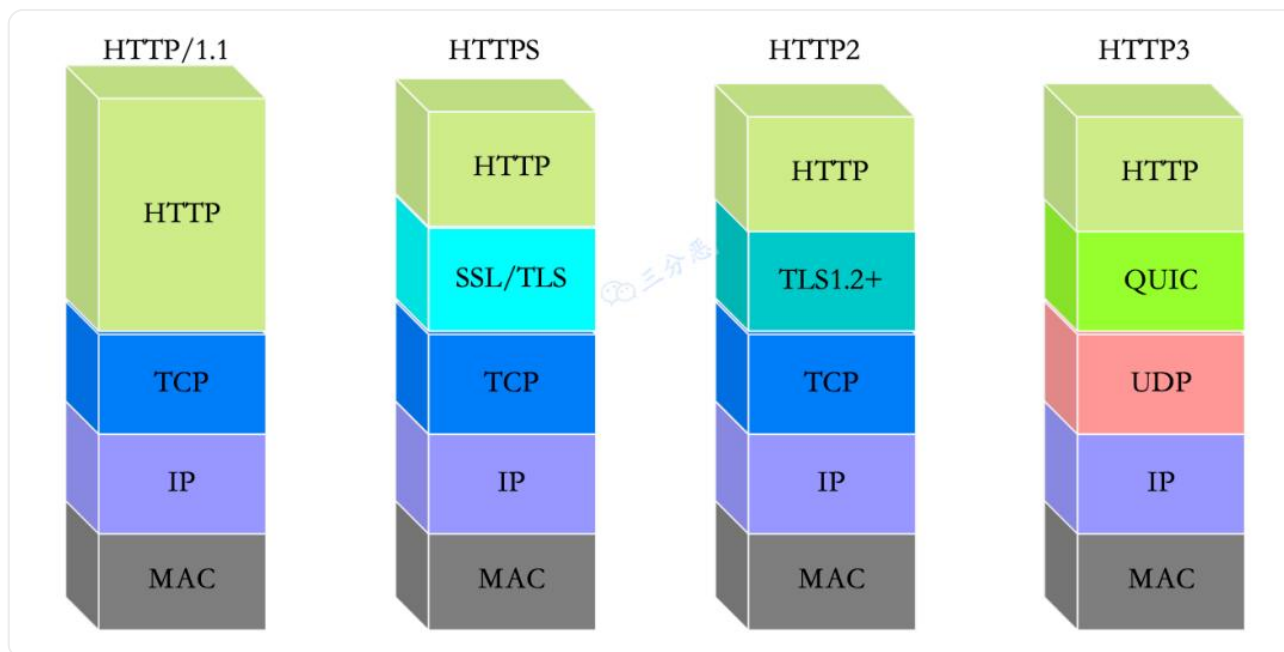
说下 HTTP2.0

- **二进制协议**：HTTP 2.0 使用二进制而不是文本格式来传输数据，解析更加高效。
- **多路复用**：一个 TCP 连接上可以同时进行多个 HTTP 请求/响应，解决了 HTTP 1.x 的队头阻塞问题。
- **头部压缩**：HTTP 协议不带状态，所以每次请求都必须附上所有信息。HTTP 2.0 引入了头部压缩机制，可以使用 gzip 或 compress 压缩后再发送，减少了冗余头部信息的带宽消耗。
- **服务端推送**：服务器可以主动向客户端推送资源，而不需要客户端明确请求。

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的腾讯面经同学 23 QQ 后台技术一面面试原题：HTTP 1 和 2 的区别

16.HTTP/3 了解吗？

HTTP/2.0 基于 TCP 协议，而 HTTP/3.0 则基于 QUIC 协议，Quick UDP Connections，直译为快速 UDP 网络连接。



三分恶面渣逆袭：HTTP 协议变迁

基于 TCP 的 HTTP/2.0，尽管从逻辑上来说，不同的流之间相互独立，不会相互影响，但在实际传输的过程中，数据还是要一帧一帧的发送和接收，一旦某一个流的数据有丢包，仍然会阻塞在它之后传输的流数据。

而基于 UDP 的 QUIC 协议可以更彻底地解决这样的问题，让不同的流之间真正的实现相互独立传输，互不干扰。

同时，QUIC 协议在传输的过程中就完成了 TLS 加密握手，更直接了。

目前使用最广泛的是哪个HTTP版本？

应该是 HTTP/2，在 2022 年 1 月达到峰值，占有网站的 46.9%。

统计网站：[w3techs](https://w3techs.com/technologies/history_overview/site_element/all)
(https://w3techs.com/technologies/history_overview/site_element/all)

Historical trends in the usage statistics of site elements for websites															
This report shows the historical trends in the usage of the top site elements since May 2023.															
	2023 1 May	2023 1 Jun	2023 1 Jul	2023 1 Aug	2023 1 Sep	2023 1 Oct	2023 1 Nov	2023 1 Dec	2024 1 Jan	2024 1 Feb	2024 1 Mar	2024 1 Apr	2024 1 May	2024 22 May	
None	1.0%	1.0%	0.9%	1.0%	1.0%	0.9%	0.9%	0.8%	0.8%	0.9%	0.9%	1.1%	1.0%	0.9%	
CSS	97.1%	97.1%	97.1%	97.2%	97.2%	97.3%	97.0%	97.0%	97.0%	97.1%	97.0%	96.9%	96.8%	96.4%	
Compression	88.3%	88.3%	88.4%	88.6%	88.5%	88.7%	88.6%	88.7%	88.6%	88.7%	88.8%	88.7%	88.8%	88.6%	
Default protocol https	82.6%	82.8%	83.2%	83.6%	83.9%	84.2%	84.4%	84.7%	84.9%	85.1%	85.3%	85.4%	85.6%	85.7%	
Cookies	41.8%	42.2%	42.2%	42.2%	42.2%	42.4%	42.3%	42.8%	43.0%	42.4%	42.3%	42.1%	41.9%	42.0%	
Default subdomain www	40.8%	40.6%	40.6%	40.4%	40.4%	40.3%	40.4%	40.5%	40.3%	40.1%	40.0%	39.8%	39.7%	39.8%	
HTTP/2	39.3%	38.1%	36.5%	36.2%	35.5%	35.6%	35.5%	35.7%	35.5%	35.6%	35.7%	35.5%	35.4%	35.3%	
HTTP/3	25.7%	25.9%	26.1%	26.4%	26.6%	26.8%	27.0%	27.3%	27.8%	28.8%	29.0%	29.2%	29.5%	29.7%	
HTTP Strict Transport Security	25.7%	25.8%	26.0%	26.2%	26.4%	26.6%	26.6%	26.8%	27.3%	27.4%	27.4%	27.2%	27.3%	27.4%	
IPv6	21.2%	21.3%	21.6%	21.8%	21.9%	22.1%	22.2%	22.5%	22.6%	22.7%	22.9%	24.6%	24.8%	25.0%	
Etag	18.2%	20.6%	20.9%	21.0%	21.1%	21.2%	21.4%	21.9%	21.9%	23.0%	22.9%	23.3%	23.4%	23.4%	
QUIC	7.8%	7.3%	7.4%	7.5%	7.4%	7.5%	7.6%	7.7%	7.7%	7.8%	7.8%	7.8%	7.9%	8.0%	
Frameset	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	0.2%	

w3techs：使用趋势

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的
华为面经同学 8 技术二面面试原题：HTTP 2.0 和 3.0 的区别
2. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的
字节跳动面经同学 1 技术二面面试原题：目前使用最广泛的是哪个HTTP版本？

17.HTTP 如何实现长连接？在什么时候会超时？

什么是 HTTP 的长连接？

1. HTTP 分为长连接和短连接, **本质上说的是 TCP 的长短连接**。TCP 连接是一个双向的通道, 它是可以保持一段时间不关闭的, 因此 TCP 连接才具有真正的长连接和短连接这一说法。
2. TCP 长连接可以复用一個 TCP 连接, 来发起多次的 HTTP 请求, 这样就可以减少资源消耗, 比如一次请求 HTML, 如果是短连接的话, 可能还需要请求后续的 JS/CSS。

如何设置长连接?

通过在头部 (请求和响应头) 设置 **Connection** 字段指定为 `keep-alive`, HTTP/1.0 协议支持, 但是是默认关闭的, 从 HTTP/1.1 以后, 连接默认都是长连接。

在什么时候会超时呢?

- HTTP 一般会有 `httpd` 守护进程, 里面可以设置 **keep-alive timeout**, 当 tcp 连接闲置超过这个时间就会关闭, 也可以在 HTTP 的 header 里面设置超时时间
- TCP 的 **keep-alive** 包含三个参数, 支持在系统内核的 `net.ipv4` 里面设置; 当 TCP 连接之后, 闲置了 **tcp_keepalive_time**, 则会发生探测包, 如果没有收到对方的 ACK, 那么会每隔 `tcp_keepalive_intvl` 再发一次, 直到发送了 **tcp_keepalive_probes**, 就会丢弃该连接。

```
1. tcp_keepalive_intvl = 15
2. tcp_keepalive_probes = 5
3. tcp_keepalive_time = 1800
```

text

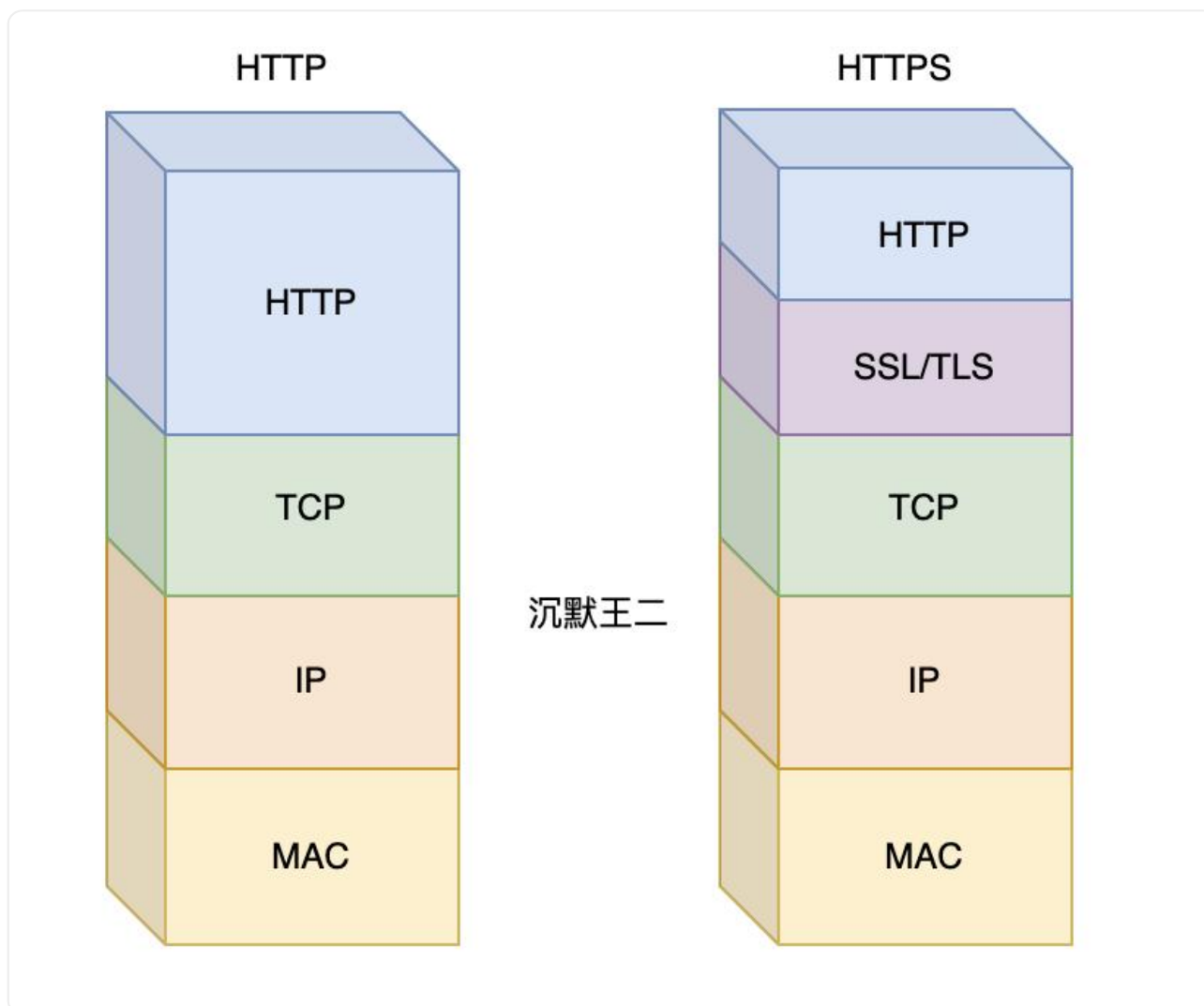
18.说说 HTTP 与 HTTPS 有哪些区别?

1. HTTPS 是 HTTP 的增强版, 在 HTTP 的基础上加入了 SSL/TLS 协议, 确保数据在传输过程中是加密的。SSL/TLS 需要向 CA (证书权威机构) 申请数字证书, 用于验证服务器的身份。
2. HTTP 的默认端口号是 80, URL 以 `http://` 开头; HTTPS 的默认端口号是 443, URL 以 `https://` 开头。

1. [Java 面试指南 \(付费\) \(https://javabetter.cn/zhishixingqiu/mianshi.html\)](https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的字节跳动面经同学 13 Java 后端二面面试原题: http 和 https 的区别, https 是怎么建立连接, https 是对称还是非对称加密
2. [Java 面试指南 \(付费\) \(https://javabetter.cn/zhishixingqiu/mianshi.html\)](https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的小公司面经合集好未来测开面经同学 3 测开一面面试原题: 说说 HTTP 和 HTTPS 的区别

19.为什么要用 HTTPS? 解决了哪些问题?

使用 HTTPS 主要是为了解决 HTTP 传输过程中的一些安全问题，因为 HTTP 是明文传输，所以 HTTPS 在 HTTP 的基础上加入了 SSL/TLS 协议。

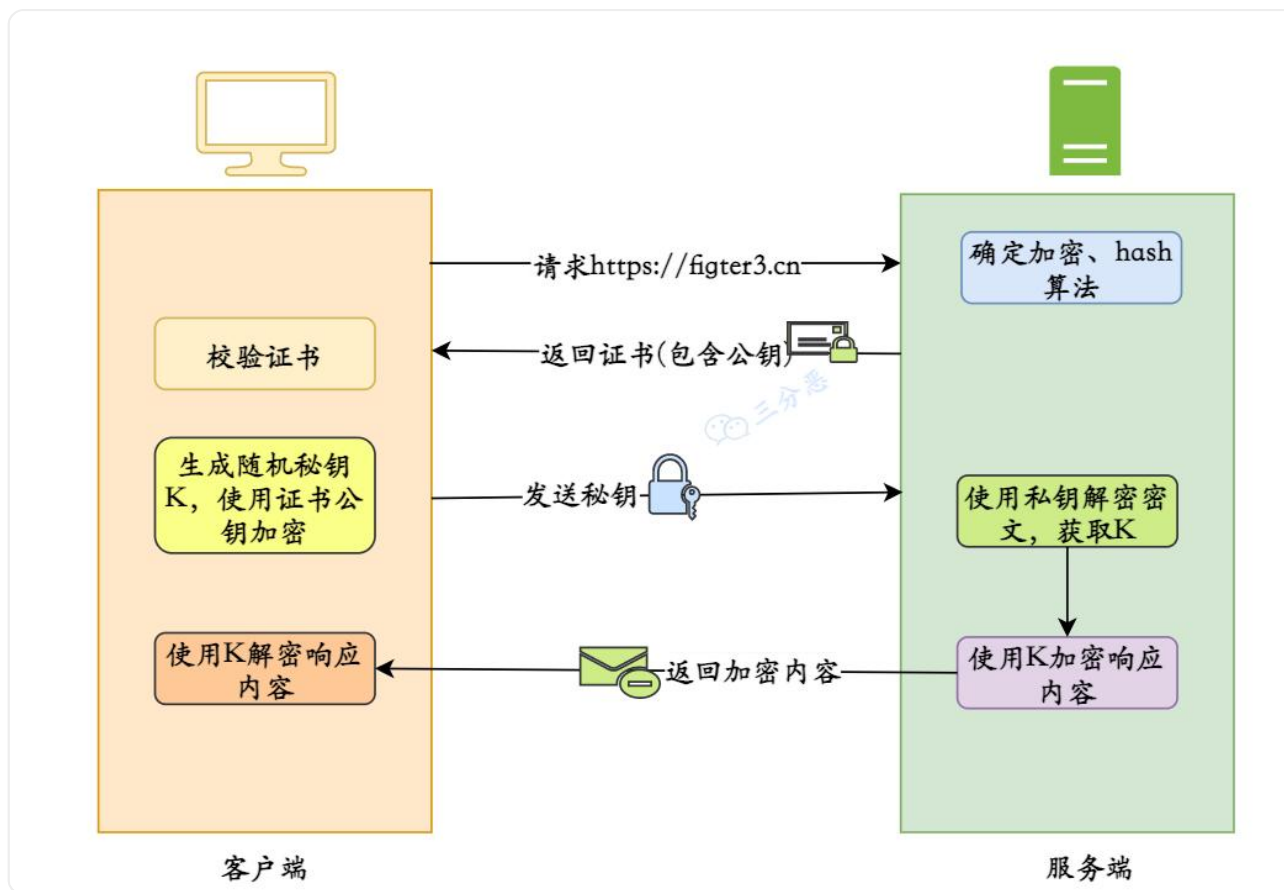


二哥的 Java 进阶之路：http和 https 的区别

SSL（安全套接字）/TLS（传输层安全）协议可以用来加密通信内容，保证通信过程中的数据不被窃取和篡改。整个加密过程主要涉及两种类型的加密方法：

- 非对称加密：服务器向客户端发送公钥，然后客户端用公钥加密自己的随机密钥，也就是会话密钥，发送给服务器，服务器用私钥解密，得到会话密钥。
- 然后双方用会话密钥加密通信内容。

客户端会通过数字证书来验证服务器的身份，数字证书由 CA（证书权威机构）签发，包含了服务器的公钥、证书的颁发机构、证书的有效期等信息。



三分恶面渣逆袭：HTTPS 主要流程

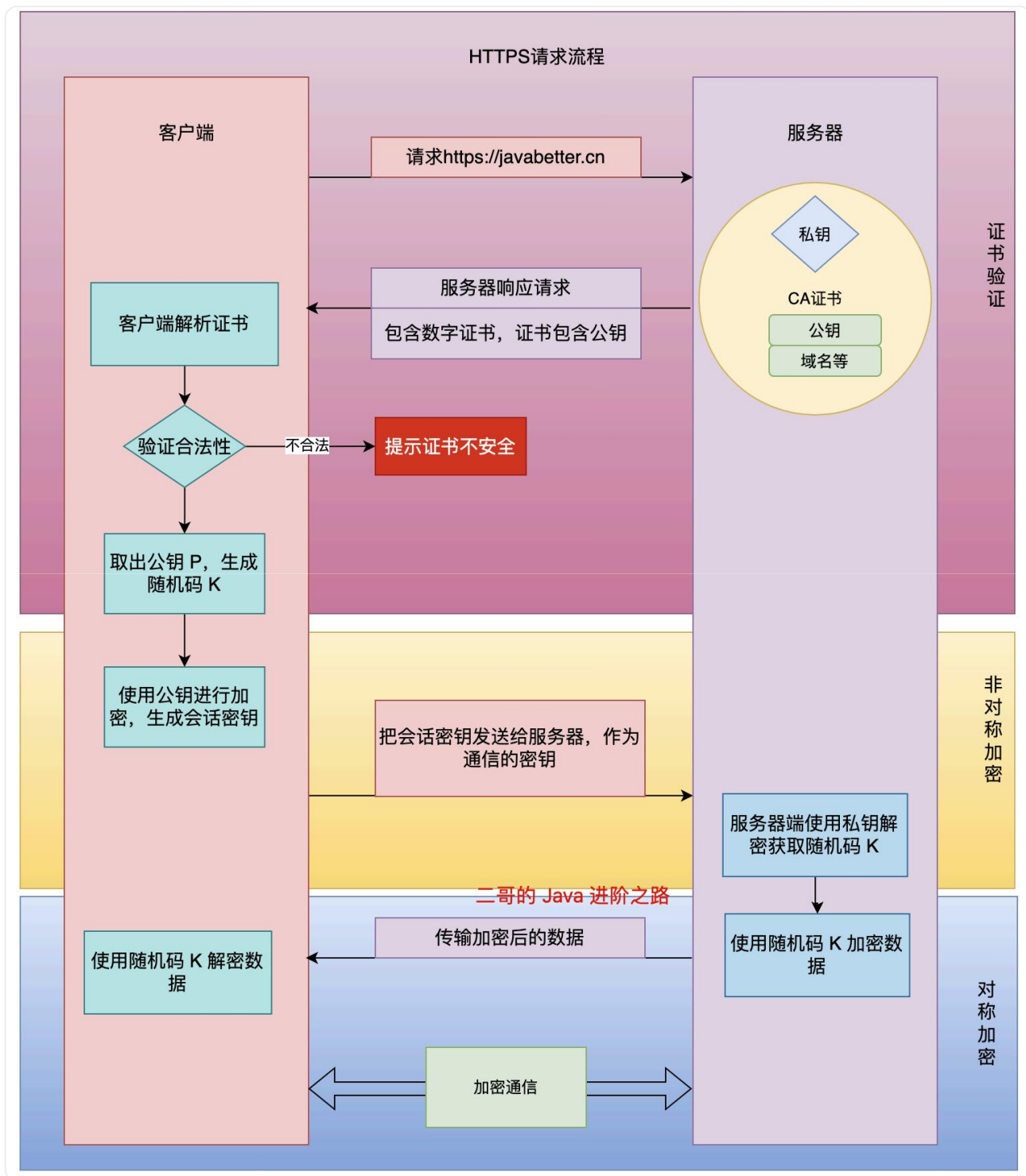
HTTPS 主要解决了以下几个问题：

- **窃听风险**：第三方可以截获传输的数据包，获取敏感信息。
- **篡改风险**：第三方可以在传输过程中篡改数据包，修改数据。
- **冒充风险**：第三方可以冒充服务器，与客户端通信。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的比亚迪面经同学 3 Java 技术一面面试原题：说一下 HTTP 的结构和 HTTPS 的原理

20.HTTPS 工作流程是怎样的？

HTTPS 的连接建立在 SSL/TLS 握手之上，主要分为以下几个步骤：



- ①、客户端向服务器发起请求
- ②、服务器接收到请求后，会返回自己的数字证书，包含了公钥、颁发机构等信息。
- ③、客户端收到服务器的数字证书后，会验证证书的合法性，如果合法，就会生成一个随机码，然后用服务器的公钥加密这个随机码，发送给服务器。
- ④、服务器收到会话密钥后，用私钥解密，得到会话密钥。

⑤、客户端和服务端通过会话密码对通信内容进行加密，然后传输。

如果通信内容被截取，但由于没有会话密钥，所以无法解密。当通信结束后，连接会被关闭，会话密钥也会被销毁，下次通信会重新生成一个会话密钥。

HTTPS 在不同阶段会使用不同的加密方式：

- 非对称加密：在握手阶段使用，特别是在密钥交换过程中。非对称加密使用公钥和私钥，其中公钥可以公开，私钥保密。客户端使用公钥加密信息，服务器使用私钥解密。
- 对称加密：在完成握手后，所有的数据传输都使用对称加密。对称加密使用相同的密钥进行加密和解密，这种加密方式比非对称加密更快。

HTTPS 会加密 URL 吗？

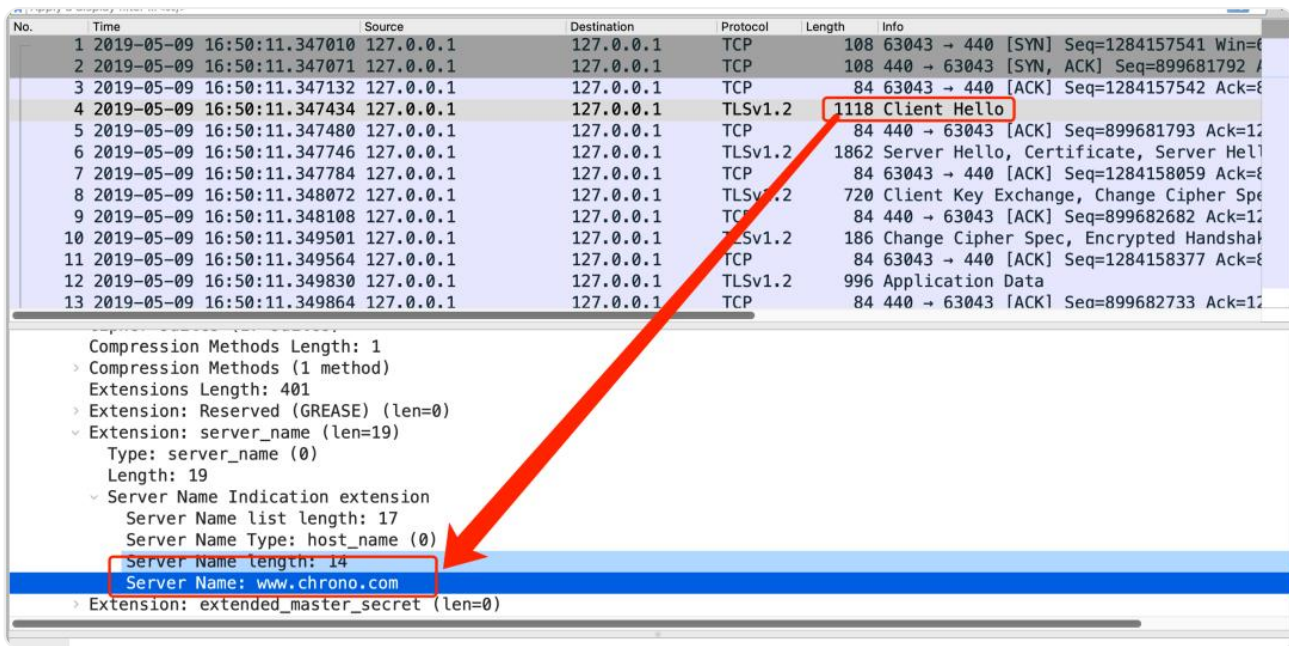
HTTPS 通过 SSL/TLS 协议确保了客户端与服务端之间交换的数据被加密，这包括 HTTP 头部和正文。

而 URL 是 HTTP 头部的一部分，因此这部分信息也是加密的。

HTTP请求报文	请求头	方法	空格	URL	空格	HTTP版本	回车换行 CRLF	请求行
		通用头名	:	空格	通用头值		回车换行 CRLF	实体头
		请求头名	:	空格	请求头值		回车换行 CRLF	
		实体头名	:	空格	实体头值		回车换行 CRLF	
		回车换行 CRLF						头部结束标志
	请求体	HTTP数据体						

人人编程网：HTTP 协议请求报文

但因为涉及到 SSL 握手的过程，所以域名信息会被暴露出来，需要注意。



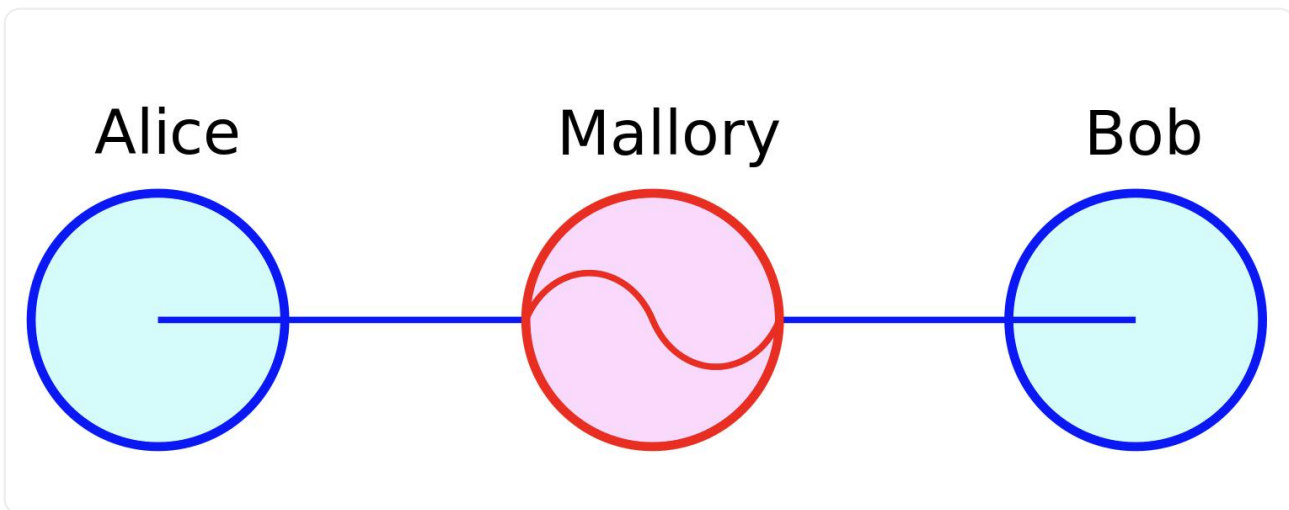
小林: server name

另外，完整的 URL 可能在 Web 服务器的日志中记录，这些日志可能是明文的。还有，URL 在浏览器历史记录中也是可见的。

因此，敏感信息永远不应该通过 URL 传递，即使是在使用 HTTPS 的情况下。

什么是中间人攻击？

中间人攻击（Man-in-the-Middle, MITM）是一种常见的网络安全威胁，攻击者可以在通信的两端插入自己，以窃取通信双方的信息。



维基百科

在很多电影中，都会存在这样的场景：主角通过某种方式，将自己伪装成中间人，然后窃取通信双方的信息，阿汤哥的碟中谍中就有很多类似的手笔。

中间人攻击是一个缺乏相互认证的攻击, 因此大多数加密协议都会专门加入一些特殊的认证方法, 以防止中间人攻击。像 SSL 协议, 就是通过验证服务器的数字证书, 是否由 CA (权威的受信任的数字证书认证机构) 签发, 来防止中间人攻击的。

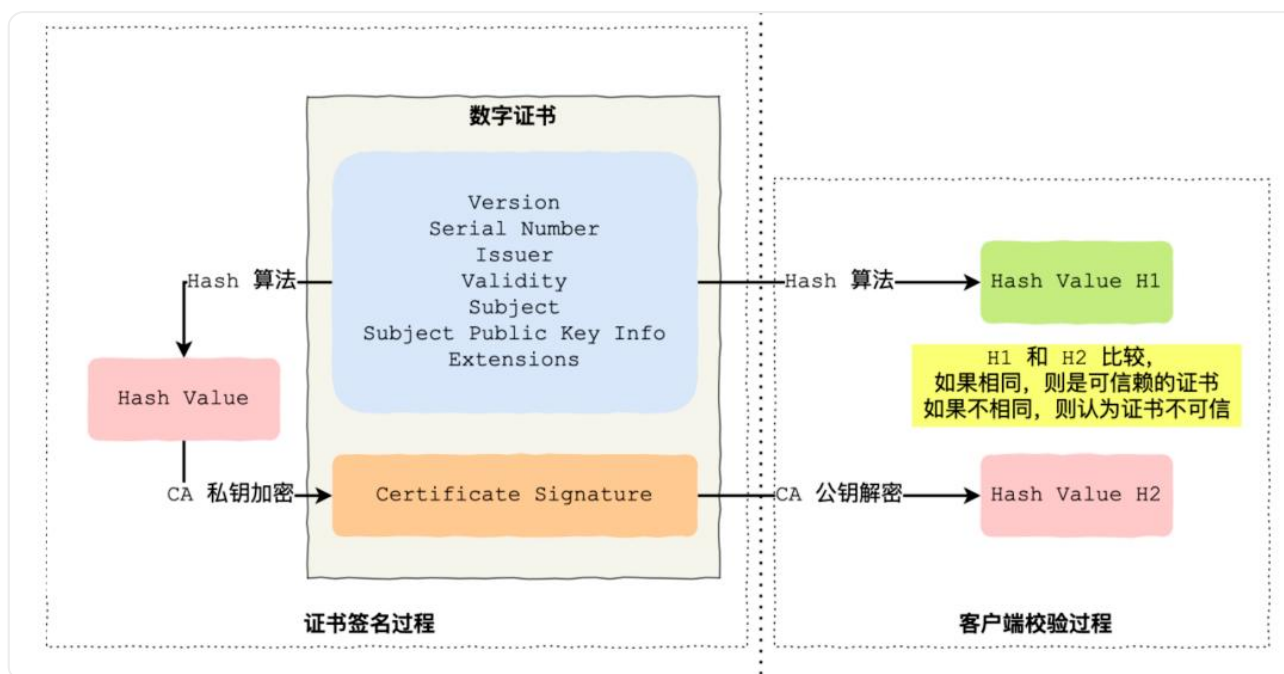
1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的字节跳动面经同学 13 Java 后端二面面试原题: http 和 https 的区别, https 是怎么建立连接, https 是对称还是非对称加密
2. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的字节跳动面经同学 13 Java 后端二面面试原题: http 有哪些方法, http 的 get 方法可以实现写操作吗, https 传递 url 安全吗, 为什么数据在浏览器中, 中间人攻击是什么

21. 客户端怎么去校验证证书的合法性?

推荐阅读: [HTTPS 握手过程中, 客户端如何验证证书的合法性](https://github.com/Advanced-Frontend/Daily-Interview-Question/issues/74) (https://github.com/Advanced-Frontend/Daily-Interview-Question/issues/74)

首先, 所有的证书都是由 CA 机构签发的, CA 机构是一个受信任的第三方机构, 它会对证书的申请者进行身份验证, 然后签发证书。

CA 就像是网络世界的公安局, 具有极高的可信度。



三分恶面渣逆袭: 证书签名和客户端校验-来源参考

CA 签发证书的过程是非常严格的:

- 首先，CA 会把持有者的公钥、用途、颁发者、有效时间等信息打成一个包，然后对这些信息进行 Hash 计算，得到一个 Hash 值；
- 然后 CA 会使用自己的私钥将该 Hash 值加密，生成 Certificate Signature；
- 最后将 Certificate Signature 添加在文件证书上，形成数字证书。

证书查看者：javabetter.cn

基本信息(G)

详细信息(D)

颁发对象

公用名 (CN)

组织 (O)

组织单位 (OU)

javabetter.cn

<未包含在证书中>

<未包含在证书中>

颁发者

公用名 (CN)

组织 (O)

组织单位 (OU)

TrustAsia RSA DV TLS CA G3

TrustAsia Technologies, Inc.

<未包含在证书中>

二哥的 Java 进阶之路：证书信息

客户端（通常是浏览器，通常会集成 CA 的公钥信息）在校验证证书的合法性时，主要通过以下步骤来校验证证书的合法性。

- 浏览器会读取证书的所有者、有效期、颁发者等信息，先校验网站域名是否一致，然后校验证证书的有效期是否过期；
- 浏览器开始查找内置的 CA，与服务器返回证书中的颁发者进行对比，确认是否为合法机构；
- 如果是，从内部植入的 CA 公钥解密 Certificate 的 Signature 内容，得到一个 Hash 值 H2；
- 使用同样的 Hash 算法获取证书的 Hash 值 H1，比较 H1 和 H2，如果值相同，则为可信赖的证书，否则告警。

假如在 HTTPS 的通信过程中，中间人篡改了证书，但由于他没有 CA 机构的私钥，所以无法生成正确的 Signature，因此就无法通过校验。

1. [Java 面试指南（付费）\(https://javabetter.cn/zhishixingqiu/mianshi.html\)](https://javabetter.cn/zhishixingqiu/mianshi.html)收录的得物面经同学 1 面试原题：HTTPS，中间人伪造证书怎么办，伪造证书机构

22. 如何理解 HTTP 协议是无状态的？

这个 无状态 的 状态 值的是什么？是客户端的状态，所以字面意思，就是 HTTP 协议中服务端不会保存客户端的任何信息。

比如当浏览器第一次发送请求给服务器时，服务器响应了；如果同个浏览器发起第二次请求给服务器时，它还是会响应，但是呢，服务器不知道你就是刚才的那个浏览器。

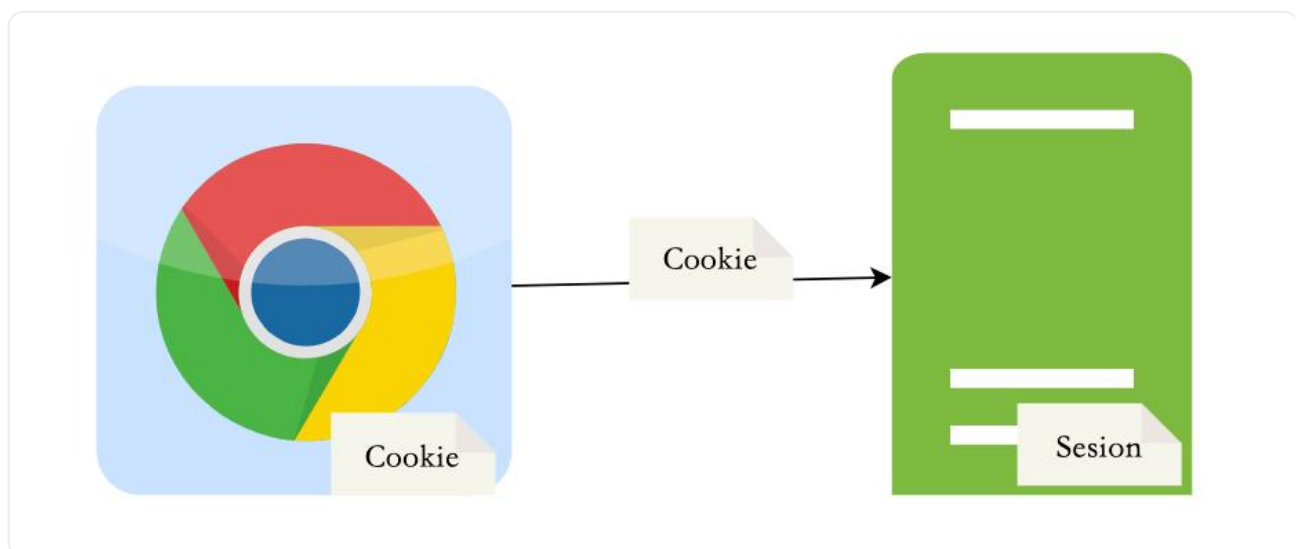
那有什么办法记录状态呢？

主要有两个办法，Session 和 Cookie。

23. 说说 Session 和 Cookie 有什么联系和区别？

先来看看什么是 Session 和 Cookie：

- Cookie 是保存在客户端的一小块文本串的数据。客户端向服务器发起请求时，服务端会向客户端发送一个 Cookie，客户端就把 Cookie 保存起来。在客户端下次向同一服务器再发起请求时，Cookie 被携带发送到服务器。服务端可以根据这个 Cookie 判断用户的身份和状态。
- Session 指的就是服务器和客户端一次会话的过程。它是另一种记录客户状态的机制。不同的是 cookie 保存在客户端浏览器中，而 session 保存在服务器上。客户端浏览器访问服务器的时候，服务器把客户端信息以某种形式记录在服务器上，这就是 session。客户端浏览器再次访问时只需要从该 session 中查找用户的状态。



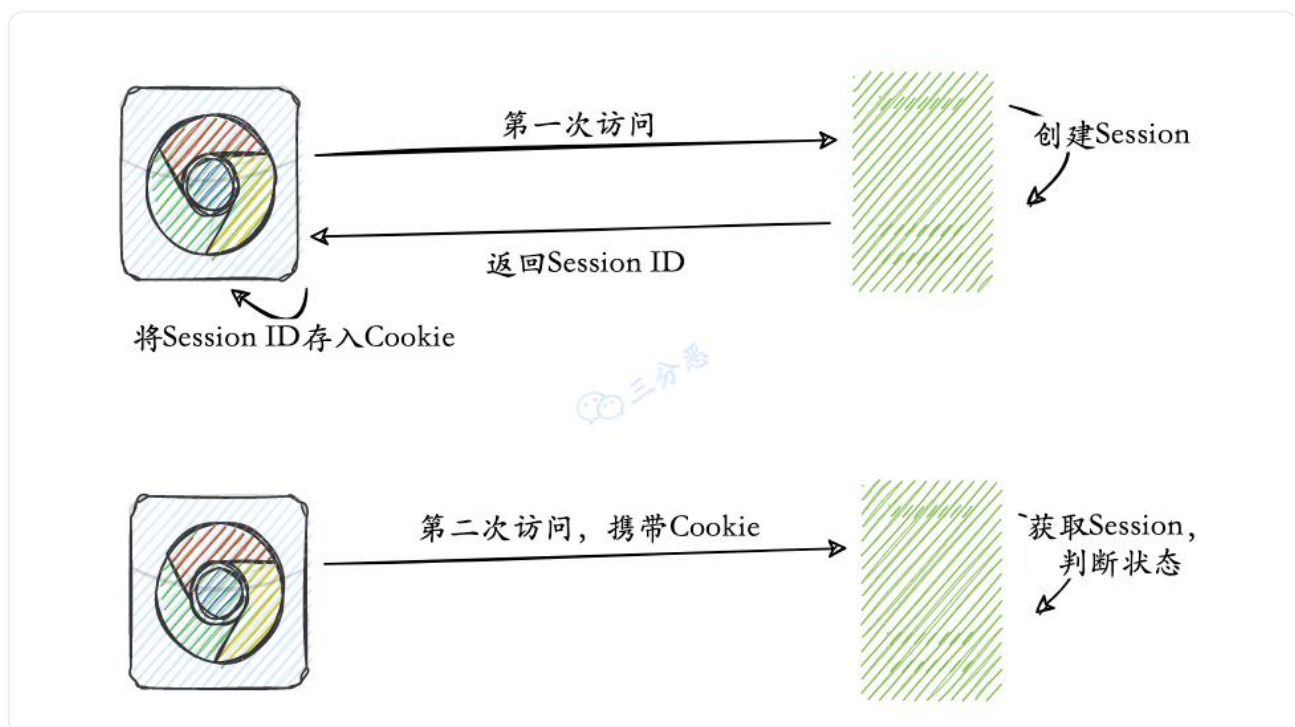
Cookie 和 Session

Session 和 Cookie 到底有什么不同呢?

- 存储位置不一样, Cookie 保存在客户端, Session 保存在服务器端。
- 存储数据类型不一样, Cookie 只能保存 ASCII, Session 可以存任意数据类型, 一般情况下我们可以在 Session 中保持一些常用变量信息, 比如说 UserId 等。
- 有效期不同, Cookie 可设置为长时间保持, 比如我们经常使用的默认登录功能, Session 一般有效时间较短, 客户端关闭或者 Session 超时都会失效。
- 隐私策略不同, Cookie 存储在客户端, 比较容易遭到不法获取, 早期有人将用户的登录名和密码存储在 Cookie 中导致信息被窃取; Session 存储在服务端, 安全性相对 Cookie 要好一些。
- 存储大小不同, 单个 Cookie 保存的数据不能超过 4K, Session 可存储数据远高于 Cookie。

Session 和 Cookie 有什么关联呢?

可以使用 Cookie 记录 Session 的标识。



Session 和 Cookie 的关联

- 用户第一次请求服务器时, 服务器根据用户提交的信息, 创建对应的 Session, 请求返回时将此 Session 的唯一标识信息 SessionID 返回给浏览器, 浏览器接收到服务器返回的 SessionID 信息后, 会将此信息存入 Cookie 中, 同时 Cookie 记录此 SessionID 是属于哪个域名。
- 当用户第二次访问服务器时, 请求会自动判断此域名下是否存在 Cookie 信息, 如果存在, 则自动将 Cookie 信息也发送给服务端, 服务端会从 Cookie 中获取 SessionID, 再

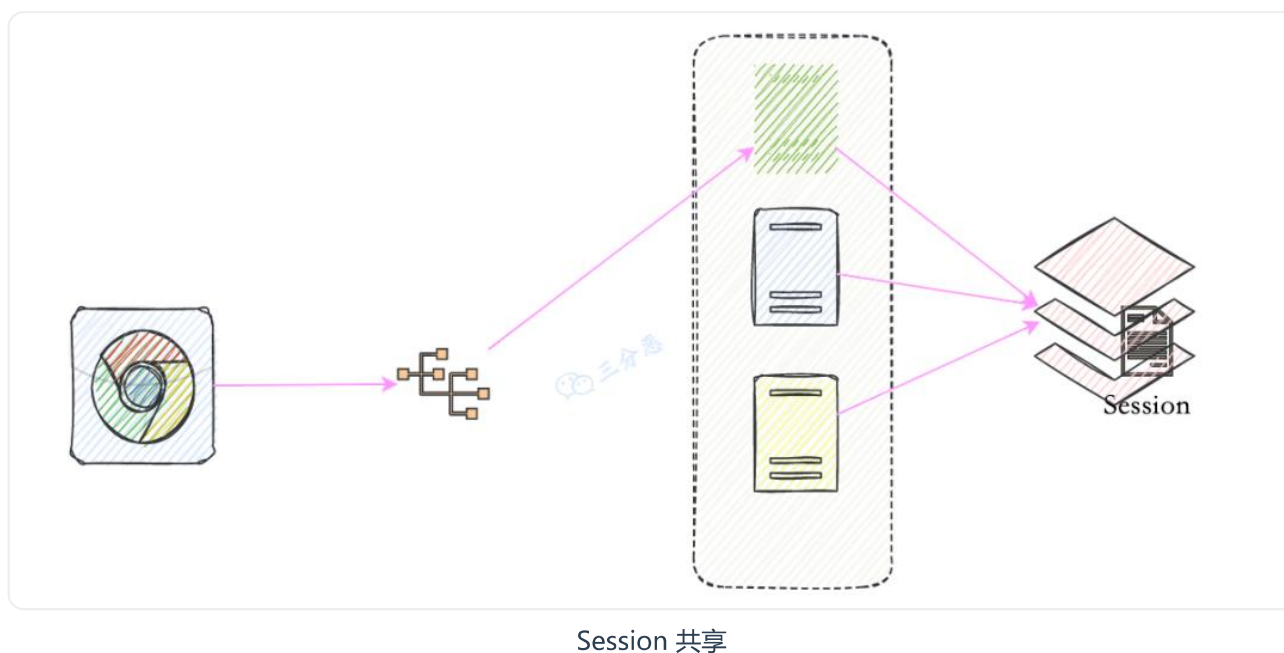
根据 SessionID 查找对应的 Session 信息, 如果没有找到, 说明用户没有登录或者登录失效, 如果找到 Session 证明用户已经登录可执行后面操作。

分布式环境下 Session 怎么处理呢?

分布式环境下, 客户端请求经过负载均衡, 可能会分配到不同的服务器上, 假如一个用户的请求两次没有落到同一台服务器上, 那么在新的服务器上就没有记录用户状态的 Session。

这时候怎么办呢?

可以使用 Redis 等分布式缓存来存储 Session, 在多台服务器之间共享。



客户端无法使用 Cookie 怎么办?

有可能客户端无法使用 Cookie, 比如浏览器禁用 Cookie, 或者客户端是安卓、IOS 等等。

这时候怎么办? SessionID 怎么存? 怎么传给服务端呢?

首先是 SessionID 的存储, 可以使用客户端的本地存储, 比如浏览器的 sessionStorage。

接下来怎么传呢?

- 拼接到 URL 里: 直接把 SessionID 作为 URL 的请求参数
- 放到请求头里: 把 SessionID 放到请求的 Header 里, 比较常用。

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了! 包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、

NIO、并发编程、JVM 等等, 共计 32 万余字, 500+张手绘图, 可以说是通俗易懂、风趣幽默.....详情戳: [太赞了, GitHub 上标星 10000+ 的 Java 教程 \(https://javabetter.cn/overview/\)](https://javabetter.cn/overview/)

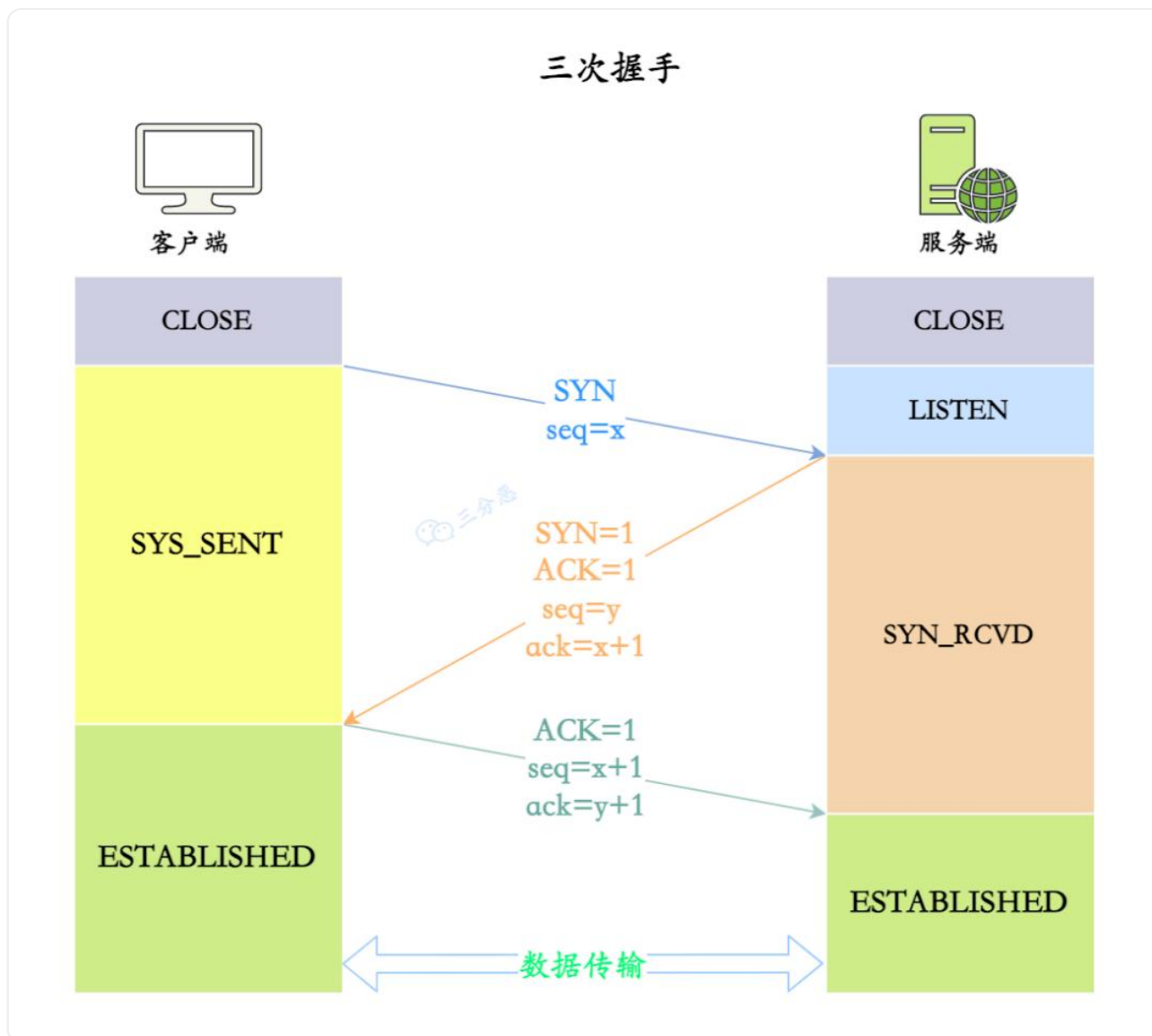
微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二, 回复 **222** 即可免费领取。



TCP

24.详细说一下 TCP 的三次握手机制

TCP (传输控制协议) 的三次握手机制是一种用于在两个 TCP 主机之间建立一个可靠的连接的过程。这个机制确保了两端的通信是同步的, 并且在数据传输开始前, 双方都准备好了进行通信。



三分恶面渣逆袭：TCP 三次握手示意图

那我再说一下三次握手👉的过程：

①、第一次握手：SYN（最开始都是 CLOSE，之后服务器进入 LISTEN）

- **发起连接：**客户端发送一个 TCP 报文段到服务器。这个报文段的头部中，SYN 位被设置为 1，表明这是一个连接请求。同时，客户端会随机选择一个序列号（Sequence Number），假设为 x ，发送给服务器。
- **目的：**客户端通知服务器它希望建立连接，并告知服务器自己的初始序列号。
- **状态：**客户端进入 SYN_SENT 状态。

②、第二次握手：SYN + ACK

- **确认并应答：**服务器收到客户端的连接请求后，如果同意建立连接，它会发送一个应答 TCP 报文段给客户端。在这个报文段中，SYN 位和 ACK 位都被设置为 1。服务器也会选择自己的一个随机序列号，假设为 y ，并将客户端的序列号加 1（即 $x+1$ ）作为确认号（Acknowledgment Number），发送给客户端。

- **目的**: 服务器告诉客户端, 它的连接请求被接受了, 并通知客户端自己的初始序列号。
- **状态**: 服务器进入 SYN_RCVD 状态。

③、第三次握手: ACK

- **最终确认**: 客户端收到服务器的应答后, 还需要向服务器发送一个确认。这个 TCP 报文段的 ACK 位被设置为 1, 确认号被设置为服务器序列号加 1 (即 $y+1$), 而自己的序列号是 $x+1$ 。
- **目的**: 客户端确认收到了服务器的同步应答, 完成三次握手, 建立连接。
- **状态**: 客户端进入 ESTABLISHED 状态, 当服务器接收到这个包时, 也进入 ESTABLISHED 状态

用大白话讲 TCP 三次握手就是:

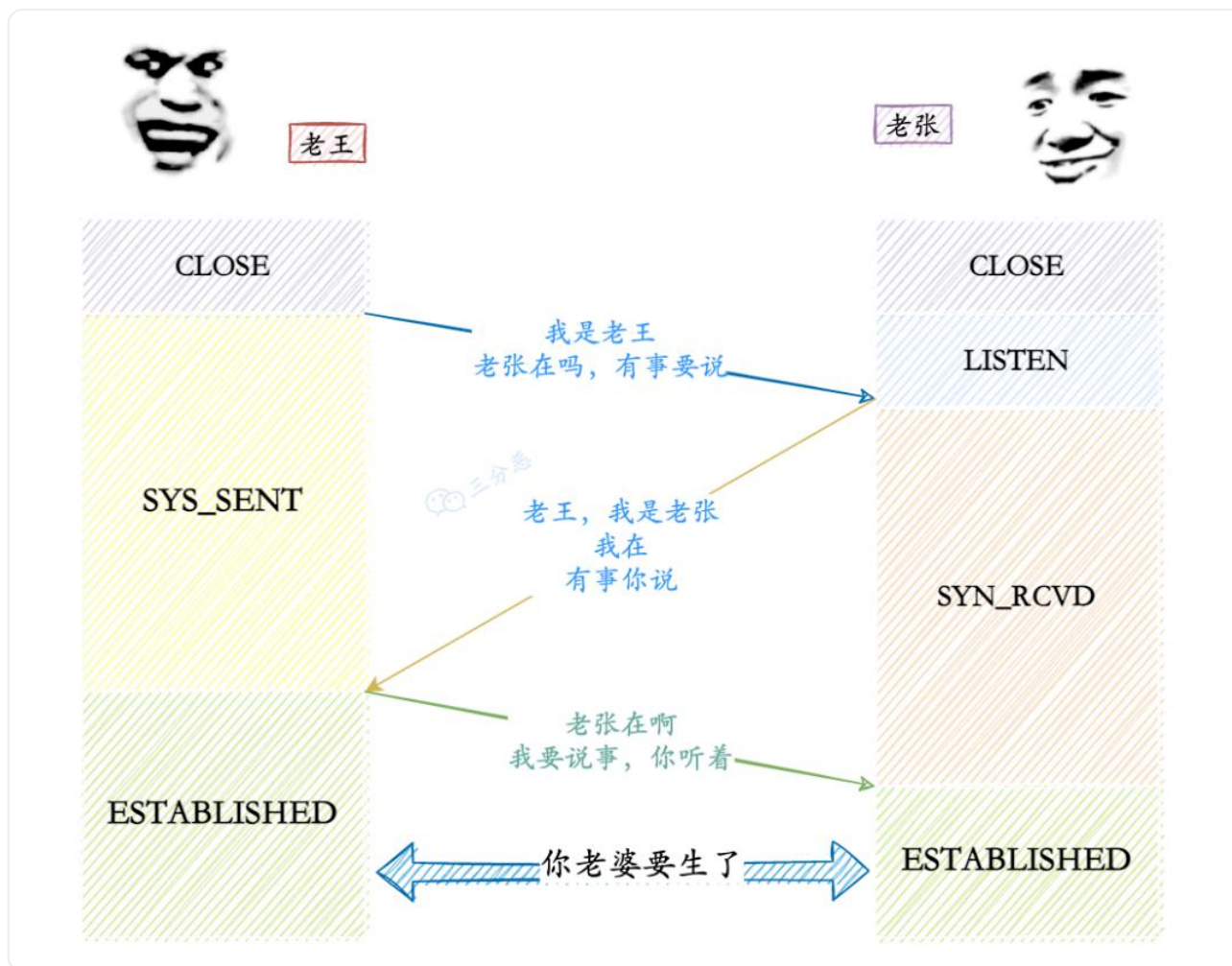
三十年前的农村, 电话还没有普及, 所以, 通信基本靠吼。

老张和老王是邻居, 这天老张下地了, 结果家里有事, 热心的邻居老王赶紧跑到村口, 开始叫唤老王。

- 老王: 老张唉! 我是老王, 你能听得到吗?
- 老张一听, 是老王的声音: 老王老王, 我是老张, 我能听得到, 你能听得到吗?
- 老王一听, 嗯, 没错, 是老张: 老张, 我听到了, 我有事要跟你说。

"你老婆要生了, 赶紧回去吧!"

老张风风火火地赶回家, 老婆顺利地生了个带把的大胖小子。握手的故事充满了幸福和美满。



三分恶面渣逆袭：大白话三次握手

可以再举一个例子说明 TCP 三次握手吗？

当然可以，你（客户端）在一个拥挤的聚会上遇到了你想交谈的美女（服务器）。因为周围很吵，你们需要确认对方都准备好交流，并清楚地听到对方说的每一句话。

①、第一次握手：打招呼

- 你走向那个美女，大声说：“嘿，我是小二，我们可以聊聊吗？”（你发送了一个连接请求，告诉服务器你想深入交流，并提供了你的微信号 x ，也就是你们交谈的起点）

②、第二次握手：对方回应

- 美女一看你挺帅挺有气质，回答说：“嗨，我是小青，可以聊聊。”（服务器接受你的请求，同样愿意深入交流，告诉你它的微信号 y ，并确认了你的微信号 $x+1$ ，表示它准备好了）

③、第三次握手：确认准备就绪

- 你听到美女的回答后, 对她说: “太好了, 我们以后就微信上聊吧。” (你确认了美女的回答, 也告诉她你准备好开始了, 通过发送确认号 $y+1$)

④、聊天开始

这时候, 你们两个就确认彼此都准备好深入交流了, 可以开始你们的对话了。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的字节跳动面经同学 9 飞书后端技术一面面试原题: TCP 为什么要三次握手

25.TCP 握手为什么是三次, 为什么不能是两次? 不能是四次?

使用三次握手可以建立一个可靠的连接。这一过程的目的是确保双方都知道对方已准备好进行通信, 并同步双方的序列号, 从而保持数据包的顺序和完整性。

为什么 TCP 握手不能是两次?

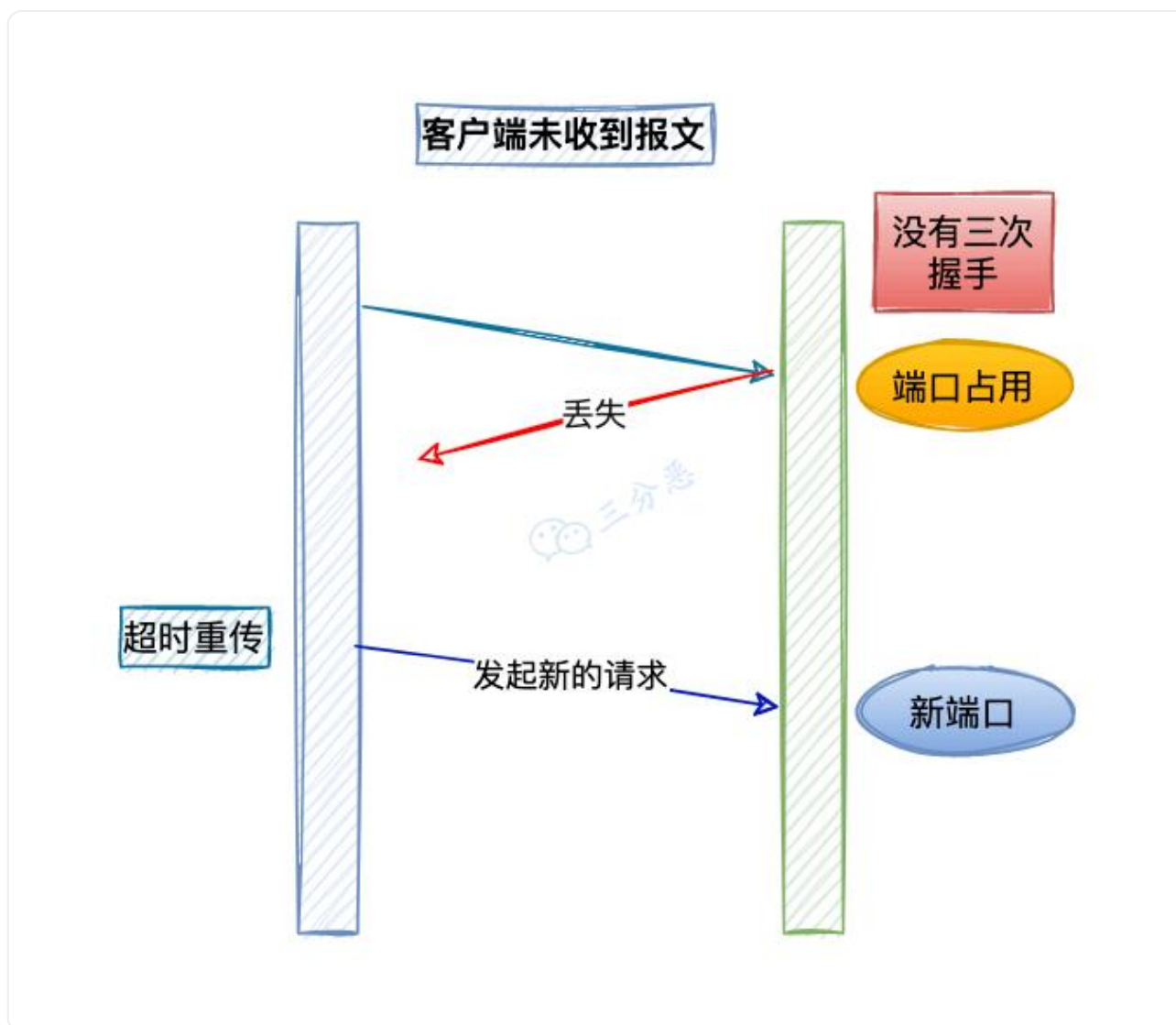
- 为了防止服务器一直等, 等到黄花菜都凉了。
- 为了防止客户端已经失效的连接请求突然又传送到了服务器。

要知道, 网络传输是有延时的 (要通过网络光纤、WIFI、卫星信号传输等)。

假如说客户端发起了 $SYN=1$ 的第一次握手。服务器也及时回复了 $SYN=2$ 和 $ACK=1$ 的第二次握手, 但是这个 $ACK=1$ 的确认报文段因为某些原因在传输过程中丢失了。

如果没有第三次握手告诉服务器, 客户端收到了服务器的回应, 那服务器是不知道客户端有没有接收到的。

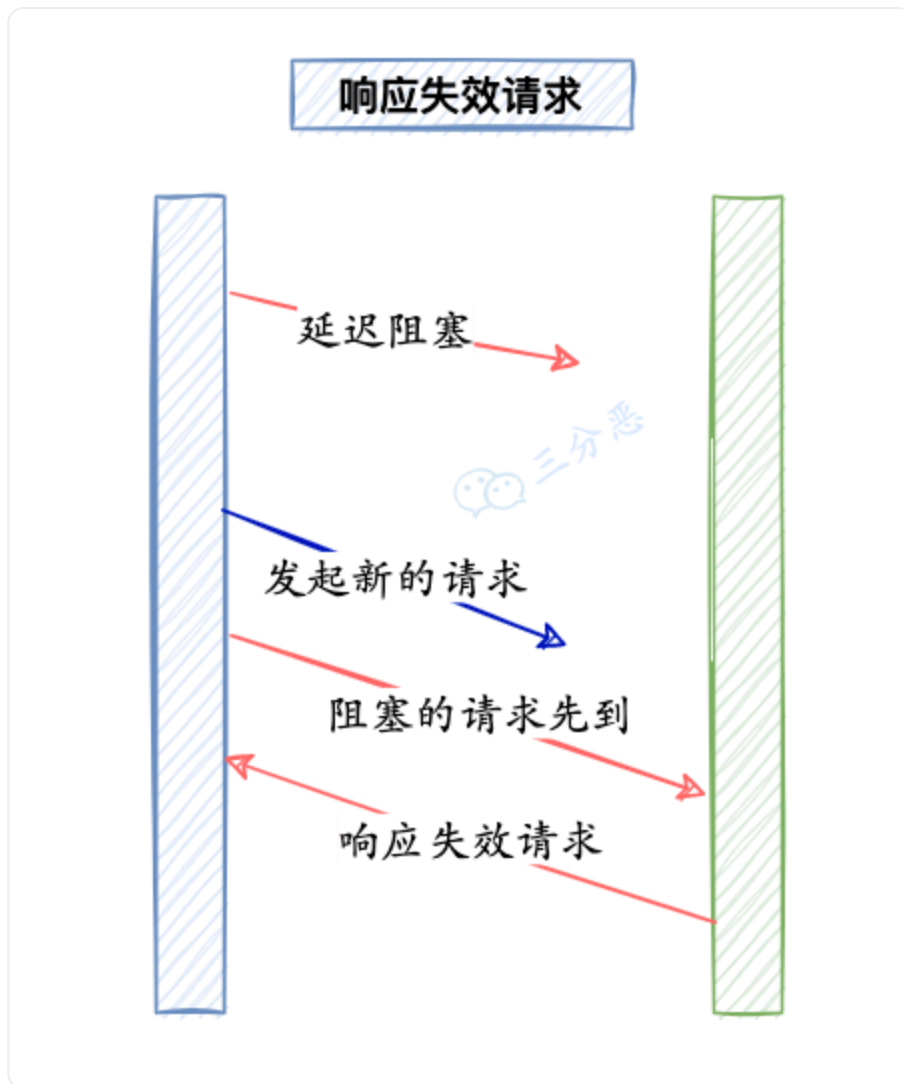
于是服务器就一直干巴巴地开着端口在等着客户端发消息呢, 但其实客户端并没有收到服务器的回应, 心灰意冷地跑了。



三分恶面渣逆袭：无三次握手导致端口占用

这就好像你找美女要联系方式了，人家回你了，你却没听见，还以为人家看不上你，赌气地跑了；剩下的美女却一直在等你。。。

还有一种情况是，一个旧的、延迟的连接请求（SYN=1）被服务器接受，导致服务器错误地开启一个不再需要的连接。



三分恶面渣逆袭：响应失效请求

举个例子：假设你（客户端）给你的朋友（服务器）发送了一个邮件（连接请求）。因为某些原因，这封邮件迟迟没有到达朋友那里，可能是因为邮局的延误。于是你决定再发一封新的邮件。朋友收到了第二封邮件，你们成功地建立了连接并开始通信。

但是，过了很久，那封延误的旧邮件突然也到了你朋友那里。如果没有一种机制来识别和处理这种延误的邮件，你的朋友可能会以为这是一个新的连接请求，并尝试响应它，但其实你已经重新发了请求，原来的不需要了。这就导致了不必要的混乱和资源浪费。

所以我们需要“三次握手”来确认这个过程：

- 第一次握手：客户端发送 SYN 包（连接请求）给服务器，如果这个包延迟了，客户端不会一直等待，它可能会重试并发送一个新的连接请求。
- 第二次握手：服务器收到 SYN 包后，发送一个 SYN-ACK 包（确认接收到连接请求）回客户端。
- 第三次握手：客户端收到 SYN-ACK 包后，再发送一个 ACK 包给服务器，确认收到了服务器的响应。

为什么不是四次?

三次握手已经足够创建可靠的连接了, 没有必要再多一次握手。

什么是泛洪攻击?

泛洪攻击 (SYN Flood Attack) 是一种常见的 DoS (拒绝服务) 攻击, 攻击者会发送大量的伪造的 TCP 连接请求, 导致服务器资源耗尽, 无法处理正常的连接请求。

半连接服务拒绝, 也称为 SYN 洪泛攻击或 SYN Flood。

所谓的半连接就是指在 TCP 的三次握手过程中, 当服务器接收到来自客户端的第一个 SYN 包后, 它会回复一个 SYN-ACK 包, 此时连接处于“半开”状态, 因为连接的建立还需要客户端发送最后一个 ACK 包。

在收到最后的 ACK 包之前, 服务器会为这个尚未完成的连接分配一定的资源, 并在它的队列中保留这个连接的位置。

如果让你重新设计, 怎么设计?

如果重新设计 TCP 的连接建立过程, 可以考虑引入 SYN cookies, 这种技术通过在 SYN-ACK 响应中编码连接信息, 从而在不占用大量资源的情况下验证客户端。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的字节跳动面经同学 9 飞书后端技术一面面试原题: TCP 为什么要三次握手
2. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的美团同学 2 优选物流调度技术 2 面面试原题: 为什么三次握手, 有什么缺点, 洪泛攻击, 半连接服务拒绝, 让你重新设计, 怎么设计

26.三次握手中每一次没收到报文会发生什么情况?

- 第一次握手服务端未收到 SYN 报文

服务端不会进行任何的动作, 而客户端由于一段时间内没有收到服务端发来的确认报文, 等待一段时间后会重新发送 SYN 报文, 如果仍然没有回应, 会重复这个过程, 直到发送次数超过最大重传次数限制, 就会返回连接建立失败。

- 第二次握手客户端未收到服务端响应的 ACK 报文

客户端会继续重传，直到次数限制；而服务端此时会阻塞在 `accept()` 处，等待客户端发送 ACK 报文

- 第三次握手服务端为收到客户端发送过来的 ACK 报文

服务端同样会采用类似客户端的超时重传机制，如果重试次数超过限制，则 `accept()` 调用返回 -1，服务端建立连接失败；而此时客户端认为自己已经建立连接成功，因此开始向服务端发送数据，但是服务端的 `accept()` 系统调用已经返回，此时不在监听状态，因此服务端接收到客户端发送来的数据时会发送 RST 报文给客户端，消除客户端单方面建立连接的状态。

27.第二次握手传回了 ACK，为什么还要传回 SYN？

ACK 是为了告诉客户端传来的数据已经接收无误。

而传回 SYN 是为了告诉客户端，服务端响应的确实是客户端发送的报文。

28.第 3 次握手可以携带数据吗？

第 3 次握手是可以携带数据的。

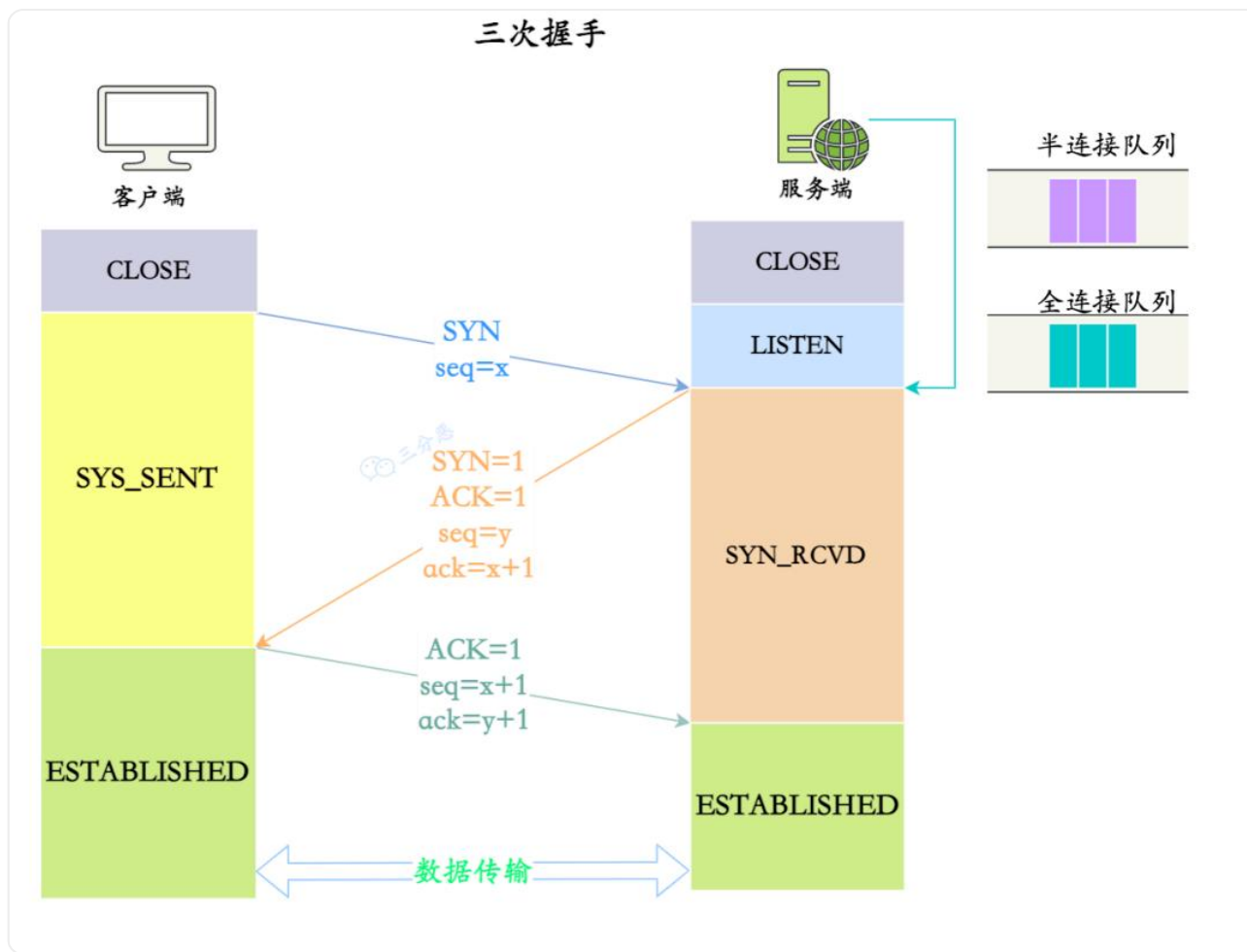
此时客户端已经处于 ESTABLISHED 状态。对于客户端来说，它已经建立连接成功，并且确认服务端的接收和发送能力是正常的。

第一次握手不能携带数据是出于安全的考虑，因为如果允许携带数据，攻击者每次在 SYN 报文中携带大量数据，就会导致服务端消耗更多的时间和空间去处理这些报文，会造成 CPU 和内存的消耗。

29.说说半连接队列和 SYN Flood 攻击的关系？

什么是半连接队列？

TCP 进入三次握手前，服务端会从 **CLOSED** 状态变为 **LISTEN** 状态，同时在内部创建了两个队列：半连接队列（SYN 队列）和全连接队列（ACCEPT 队列）。



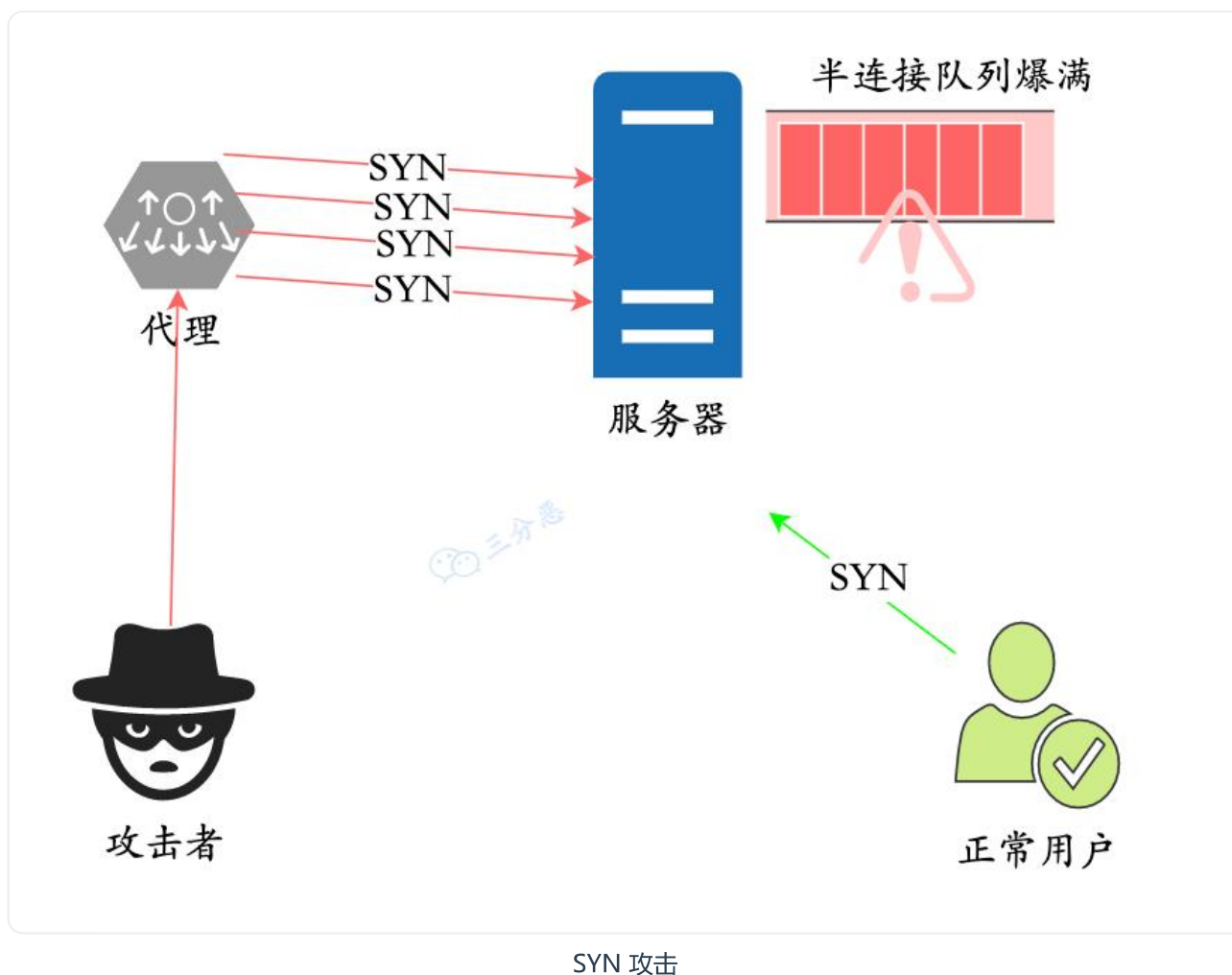
三次握手中创建的队列

顾名思义，半连接队列存放的是三次握手未完成的连接，全连接队列存放的是完成三次握手的连接。

- TCP 三次握手时，客户端发送 SYN 到服务端，服务端收到之后，便回复 **ACK 和 SYN**，状态由 **LISTEN 变为 SYN_RCVD**，此时这个连接就被推入了 **SYN 队列**，即半连接队列。
- 当客户端回复 ACK，服务端接收后，三次握手就完成了。这时连接会等待被具体的应用取走，在被取走之前，它被推入 **ACCEPT 队列**，即全连接队列。

什么是 SYN Flood ?

SYN Flood 是一种典型的 DDos 攻击，它在短时间内，伪造**不存在的 IP 地址**，向服务器发送大量 SYN 报文。当服务器回复 SYN+ACK 报文后，不会收到 ACK 回应报文，那么 SYN 队列里的连接旧不会出对队，久而久之就会占满服务端的 **SYN 接收队列**（半连接队列），使得服务器不能为正常用户服务。



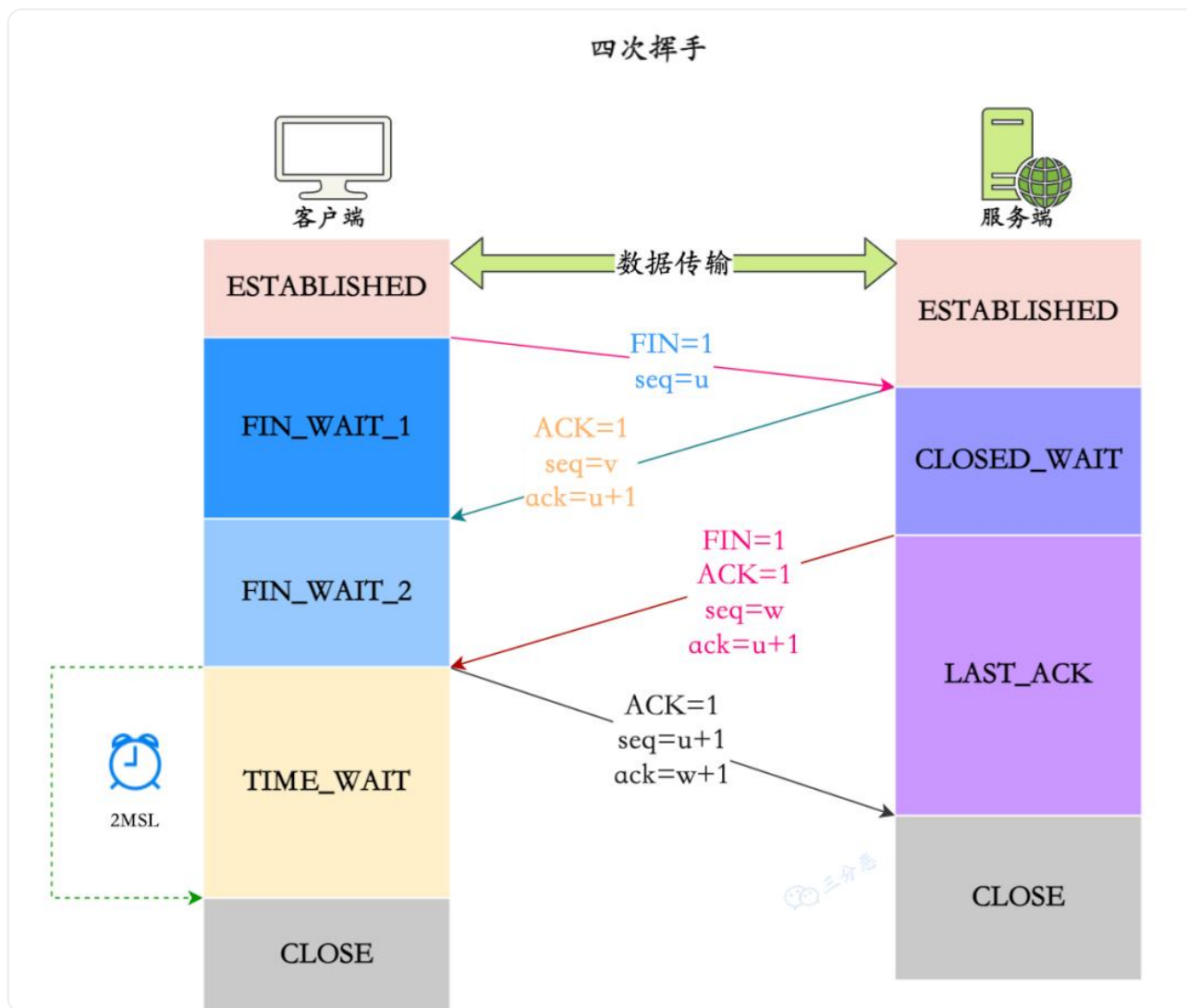
那有什么应对方案呢?

主要有 **syn cookie** 和 **SYN Proxy 防火墙**等。

- **syn cookie**: 在收到 SYN 包后, 服务器根据一定的方法, 以数据包的源地址、端口等信息为参数计算出一个 cookie 值作为自己的 SYNACK 包的序列号, 回复 SYN+ACK 后, 服务器并不立即分配资源进行处理, 等收到发送方的 ACK 包后, 重新根据数据包的源地址、端口计算该包中的确认序列号是否正确, 如果正确则建立连接, 否则丢弃该包。
- **SYN Proxy 防火墙**: 服务器防火墙会对收到的每一个 SYN 报文进行代理和回应, 并保持半连接。等发送方将 ACK 包返回后, 再重新构造 SYN 包发到服务器, 建立真正的 TCP 连接。

30.说说 TCP 四次挥手的过程?

TCP 连接的断开过程称为四次挥手 (Four-Way Handshake)。它的目的是在客户端和服务端之间优雅地关闭连接, 确保所有数据都被可靠传输。



第一次挥手 (FIN)： 客户端向服务器发送一个 FIN (结束) 报文，表示客户端没有数据要发送了，但仍然可以接收数据。客户端进入 FIN-WAIT-1 状态。

第二次挥手 (ACK)： 服务器接收到 FIN 报文后，向客户端发送一个 ACK 报文，确认已接收到客户端的 FIN 请求。服务器进入 CLOSE-WAIT 状态，客户端进入 FIN-WAIT-2 状态。

第三次挥手 (FIN)： 服务器向客户端发送一个 FIN 报文，表示服务器也没有数据要发送了。服务器进入 LAST-ACK 状态。

第四次挥手 (ACK)： 客户端接收到 FIN 报文后，向服务器发送一个 ACK 报文，确认已接收到服务器的 FIN 请求。客户端进入 TIME-WAIT 状态，等待一段时间以确保服务器接收到 ACK 报文。服务器接收到 ACK 报文后进入 CLOSED 状态。客户端在等待一段时间后也进入 CLOSED 状态。

大白话说四次挥手：

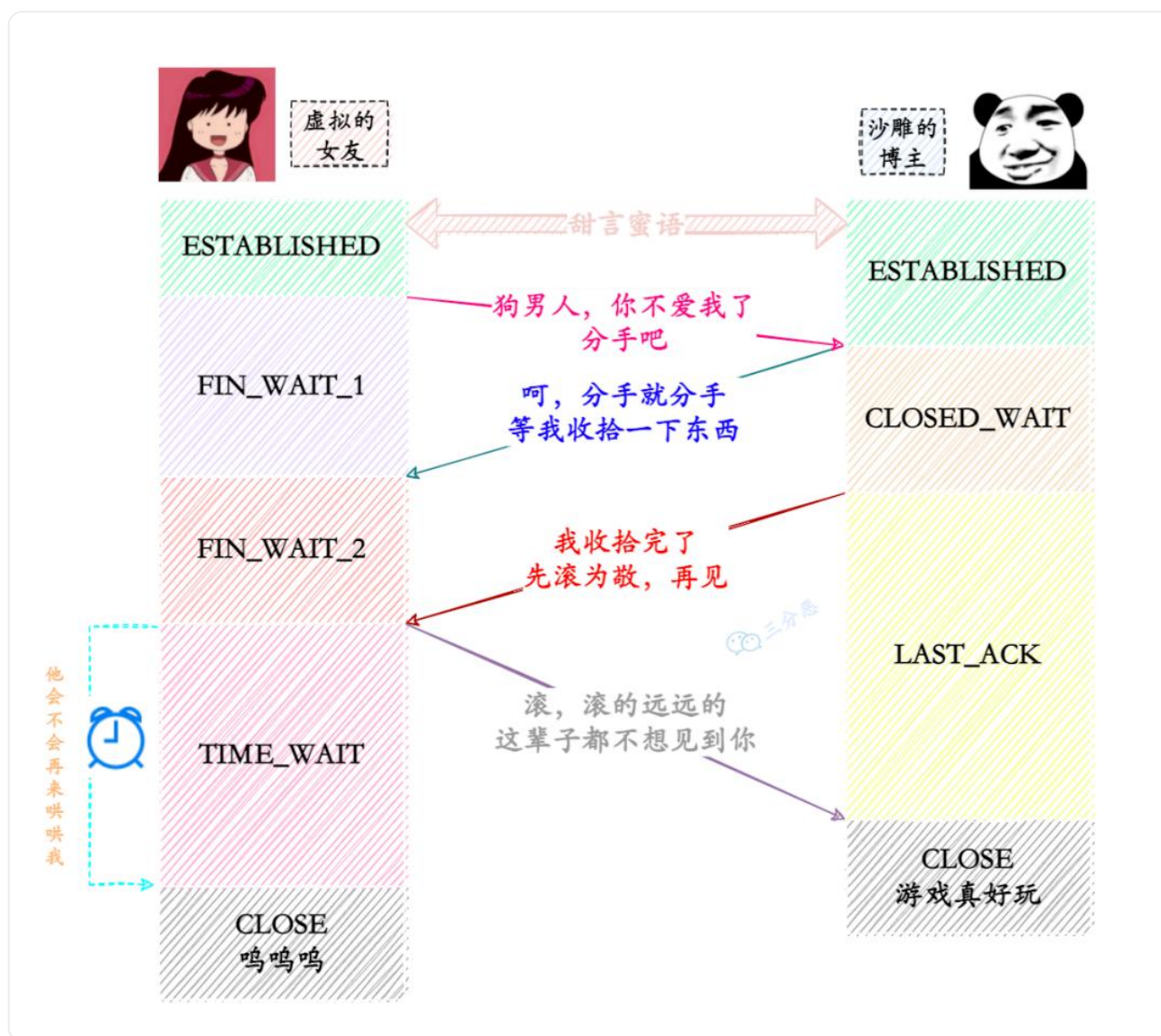
假如单身狗博主有一个女朋友—由于博主上班九九六，下班肝博客，导致没有时间陪女朋友，女朋友忍无可忍。

- 女朋友：狗男人，最近你都不理我，你是不是不爱我了？你是不是外面有别的狗子了？我要和你分手？
- 沙雕博主一愣，怒火攻心：分手就分手，不陪你闹了，等我把东西收拾收拾。

沙雕博主小心翼翼地装起了自己的青轴机械键盘。

- 哼，蠢女人，我已经收拾完了，我先滚为敬，再见！
- 女朋友：滚，滚的远远的，越远越好，我一辈子都不想再见到你。

挥手的故事总充满了悲伤和遗憾！



三分恶面渣逆袭：大白话四次挥手

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的腾讯同学 25 后端开发实习一面面试原题: TCP和UDP, TCP连接和断开过程

31.TCP 挥手为什么需要四次呢?

再来回顾下四次挥手双方发 FIN 包的过程, 就能理解为什么需要四次了。

- 关闭连接时, 客户端向服务端发送 FIN 时, 仅仅表示客户端不再发送数据了但是还能接收数据。
- 服务端收到客户端的 FIN 报文时, 先回一个 ACK 应答报文, 而服务端可能还有数据需要处理和发送, 等服务端不再发送数据时, 才发送 FIN 报文给客户端来表示同意现在关闭连接。

从上面过程可知, 服务端通常需要等待完成数据的发送和处理, 所以服务端的 ACK 和 FIN 一般都会分开发送, 从而比三次握手导致多了一次。

32.TCP 四次挥手过程中, 为什么需要等待 2MSL, 才进入 CLOSED 关闭状态?

为什么需要等待?

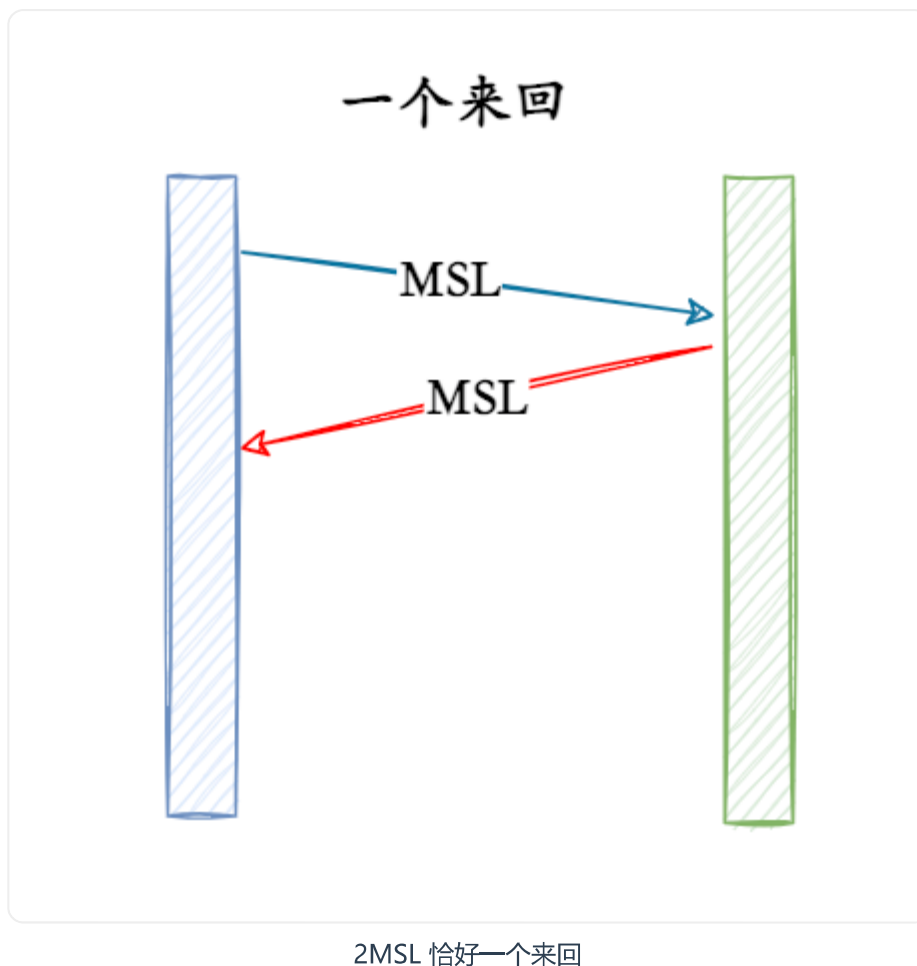
1. 为了保证客户端发送的最后一个 ACK 报文段能够到达服务端。 这个 ACK 报文段有可能丢失, 因而使处在 **LAST-ACK** 状态的服务端就收不到对已发送的 **FIN + ACK** 报文段的确认。服务端会超时重传这个 FIN+ACK 报文段, 而客户端就能在 2MSL 时间内 (**超时 + 1MSL 传输**) 收到这个重传的 FIN+ACK 报文段。接着客户端重传一次确认, 重新启动 2MSL 计时器。最后, 客户端和服务端都正常进入到 **CLOSED** 状态。

2. 防止已失效的连接请求报文段出现在本连接中。 客户端在发送完最后一个 ACK 报文段后, 再经过时间 2MSL, 就可以使本连接持续的时间内所产生的所有报文段都从网络中消失。这样就可以使下一个连接中不会出现这种旧的连接请求报文段。

为什么等待的时间是 2MSL?

MSL 是 Maximum Segment Lifetime, 报文最大生存时间, 它是任何报文在网络上存在的最长时间, 超过这个时间报文将被丢弃。

TIME_WAIT 等待 2 倍的 MSL, 比较合理的解释是: 网络中可能存在来自发送方的数据包, 当这些发送方的数据包被接收方处理后又向对方发送响应, 所以一来一回需要等待 2 倍的时间。



比如如果被动关闭方没有收到断开连接的最后的 ACK 报文，就会触发超时重发 Fin 报文，另一方接收到 FIN 后，会重发 ACK 给被动关闭方，一来一去正好 2 个 MSL。

33.保活计时器有什么用？

除时间等待计时器外，TCP 还有一个保活计时器（keepalive timer）。

设想这样的场景：客户已主动与服务器建立了 TCP 连接。但后来客户端的主机突然发生故障。显然，服务器以后就不能再收到客户端发来的数据。因此，应当有措施使服务器不要再白白等待下去。这就需要使用保活计时器了。

服务器每收到一次客户端的数据，就重新设置保活计时器，时间的设置通常是两个小时。若两个小时都没有收到客户端的数据，服务端就发送一个探测报文段，以后则每隔 75 秒钟发送一次。若连续发送 10 个探测报文段后仍然无客户端的响应，服务端就认为客户端出了故障，接着就关闭这个连接。

34.CLOSE-WAIT 和 TIME-WAIT 的状态和意义？

CLOSE-WAIT 状态有什么意义？

服务端收到客户端关闭连接的请求并确认之后，就会进入 CLOSE-WAIT 状态。此时服务端可能还有一些数据没有传输完成，因此不能立即关闭连接，而 CLOSE-WAIT 状态就是为了保证服务端在关闭连接之前将待发送的数据处理完。

TIME-WAIT 有什么意义？

TIME-WAIT 状态发生在第四次挥手，当客户端向服务端发送 ACK 确认报文后进入 TIME-WAIT 状态。

它存在的意义主要是两个：



TIME_WAIT 状态的作用

- 防止旧连接的数据包

如果客户端收到服务端的 FIN 报文之后立即关闭连接，但是此时服务端对应的端口并没有关闭，如果客户端在相同端口建立新的连接，可能会导致新连接收到旧连接残留的数据包，导致不可预料的异常发生。

- 保证连接正确关闭

假设客户端最后一次发送的 ACK 包在传输的时候丢失了，由于 TCP 协议的超时重传机制，服务端将重发 FIN 报文，如果客户端没有维持 TIME-WAIT 状态而直接关闭的话，当收到服务端重新发送的 FIN 包时，客户端就会使用 RST 包来响应服务端，导致服务端以为有错误发生，然而实际关闭连接过程是正常的。

35.TIME_WAIT 状态过多会导致什么问题? 怎么解决?

TIME_WAIT 状态过多会导致什么问题?

如果服务器有处于 TIME-WAIT 状态的 TCP, 则说明是由服务器方主动发起的断开请求。

过多的 TIME-WAIT 状态主要的危害有两种:

第一是内存资源占用;

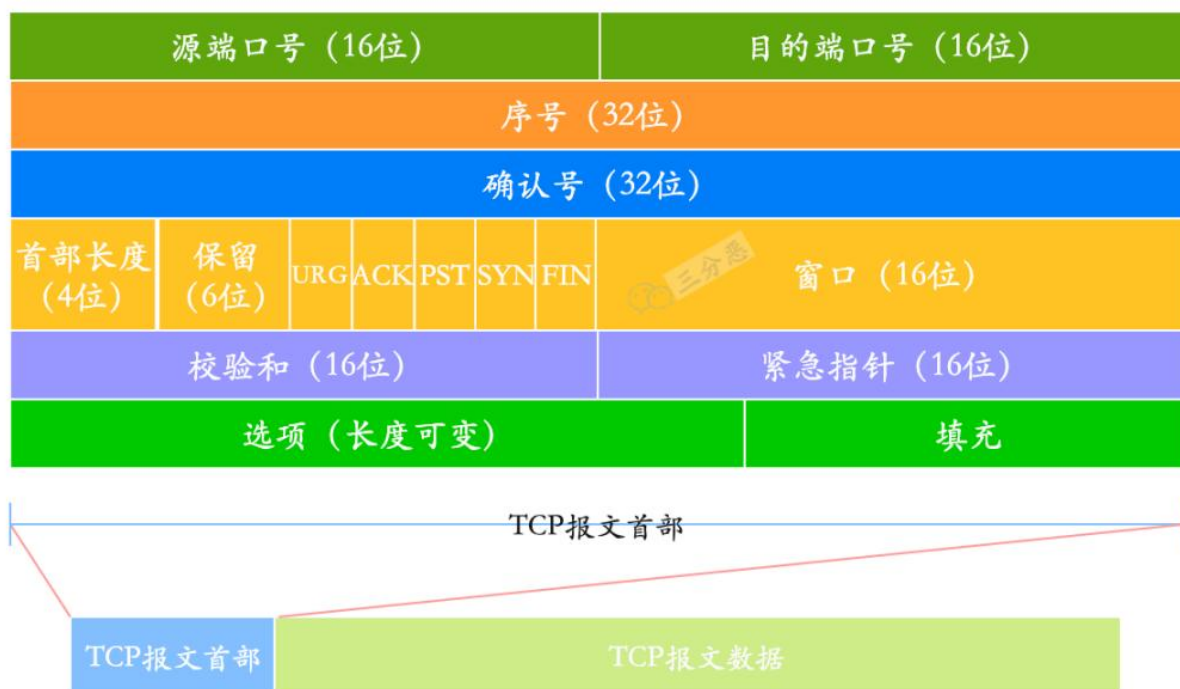
第二是对端口资源的占用, 一个 TCP 连接至少消耗一个本地端口;

怎么解决 TIME_WAIT 状态过多?

- 服务器可以设置 SO_REUSEADDR 套接字来通知内核, 如果端口被占用, 但是 TCP 连接位于 TIME_WAIT 状态时可以重用端口。
- 还可以使用长连接的方式来减少 TCP 的连接和断开, 在长连接的业务里往往不需要考虑 TIME_WAIT 状态。

36.说说 TCP 报文头部的格式?

一个 TCP 报文段主要由报文段头部 (Header) 和数据两部分组成。头部包含了确保数据可靠传输所需的各种控制信息, 比如说序列号、确认号、窗口大小等。



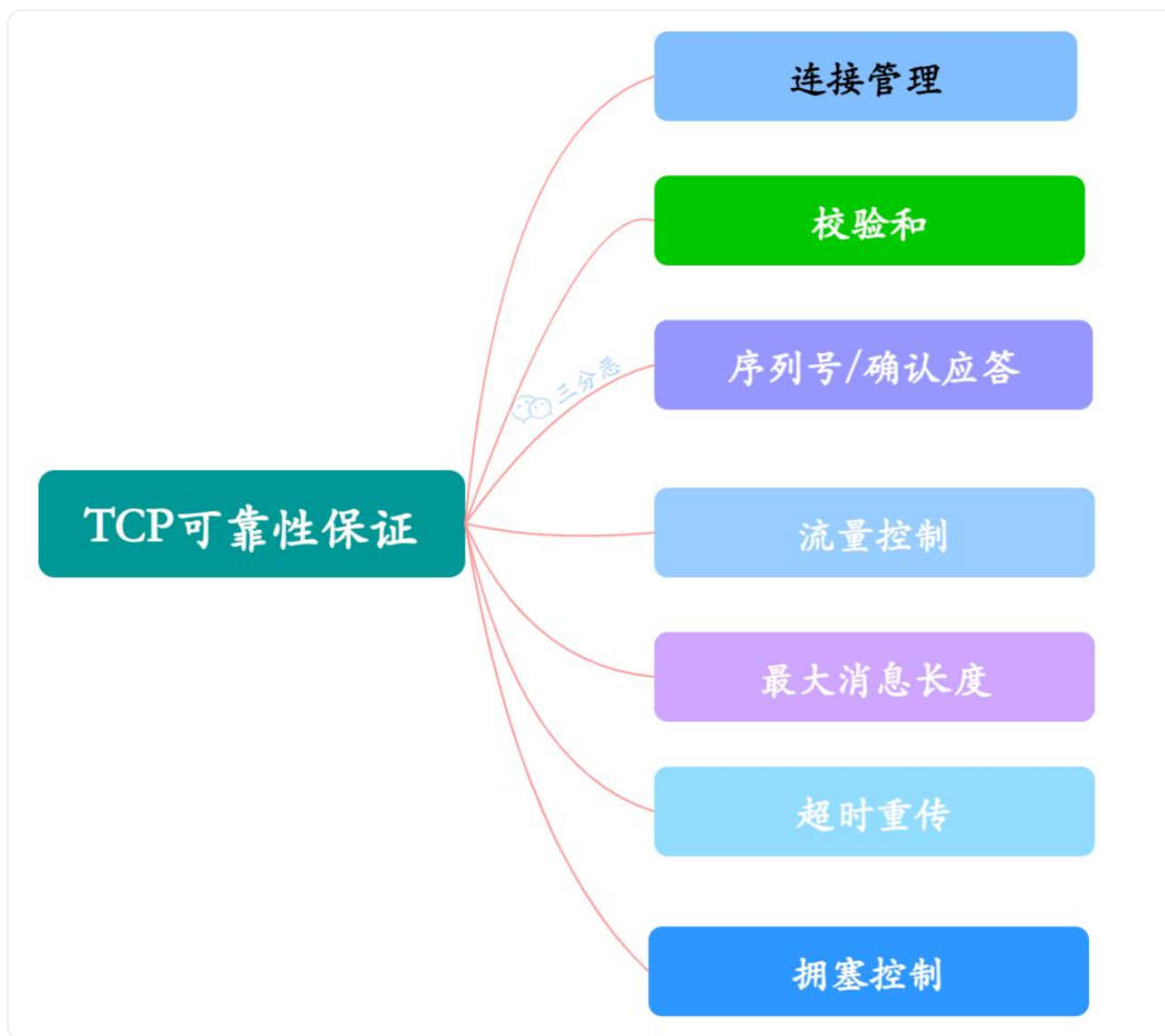
三分恶面渣逆袭：TCP 报文头部的格式

- **源端口号** (Source Port) : 16 位 (2 个字节) , 用于标识发送端的应用程序。
- **目标端口号** (Destination Port) : 也是 16 位, 用于标识接收端的应用程序。
- **序列号** (Sequence Number) : 32 位, 用于标识从 TCP 发送者发送的数据字节流中的第一个字节的顺序号。确保数据按顺序接收。
- **确认号** (Acknowledgment Number) : 32 位, 如果 ACK 标志被设置, 则该字段包含发送确认的序列号, 即接收 TCP 希望收到的下一个序列号。
- **数据偏移** (Data Offset) : 4 位, 表示 TCP 报文头部的长度, 用于指示数据开始的位置。
- **保留** (Reserved) : 6 位, 为将来使用预留, 目前必须置为 0。
- **控制位** (Flags) : 共 6 位, 包括 URG (紧急指针字段是否有效)、ACK (确认字段是否有效)、PSH (提示接收端应该尽快将这个报文段交给应用层)、RST (重置连接)、SYN (同步序号, 用于建立连接)、FIN (结束发送数据)。
- **窗口大小** (Window) : 16 位, 用于流量控制, 表示接收端还能接收的数据的字节数 (基于接收缓冲区的大小)。
- **校验和** (Checksum) : 16 位, 覆盖整个 TCP 报文段 (包括 TCP 头部、数据和一个伪头部) 的校验和, 用于检测数据在传输过程中的任何变化。
- **紧急指针** (Urgent Pointer) : 16 位, 只有当 URG 控制位被设置时才有效, 指出在报文段中有紧急数据的位置。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的字节跳动面经同学 9 飞书后端技术一面面试原题: TCP 报文的结构

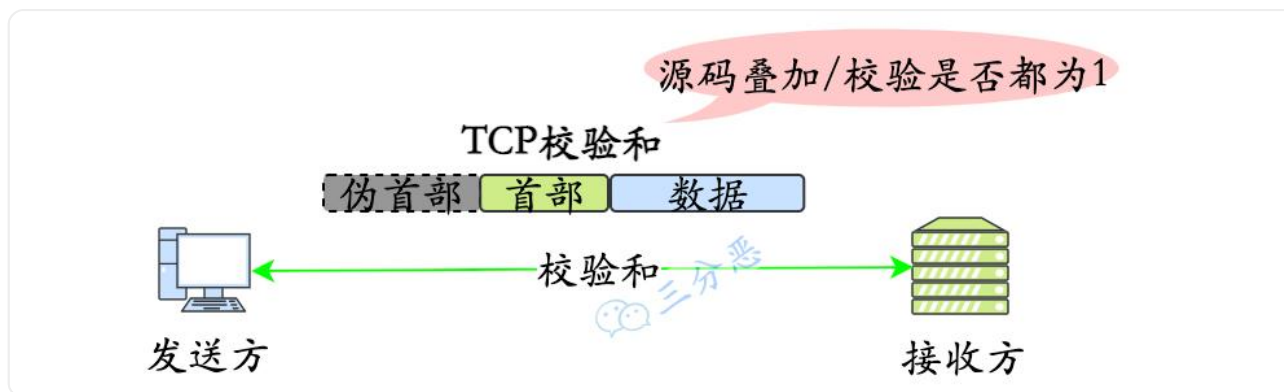
37.TCP 是如何保证可靠性的?

TCP 主要提供了检验和、序列号/确认应答、超时重传、最大消息长度、滑动窗口控制等方法实现了可靠性传输。



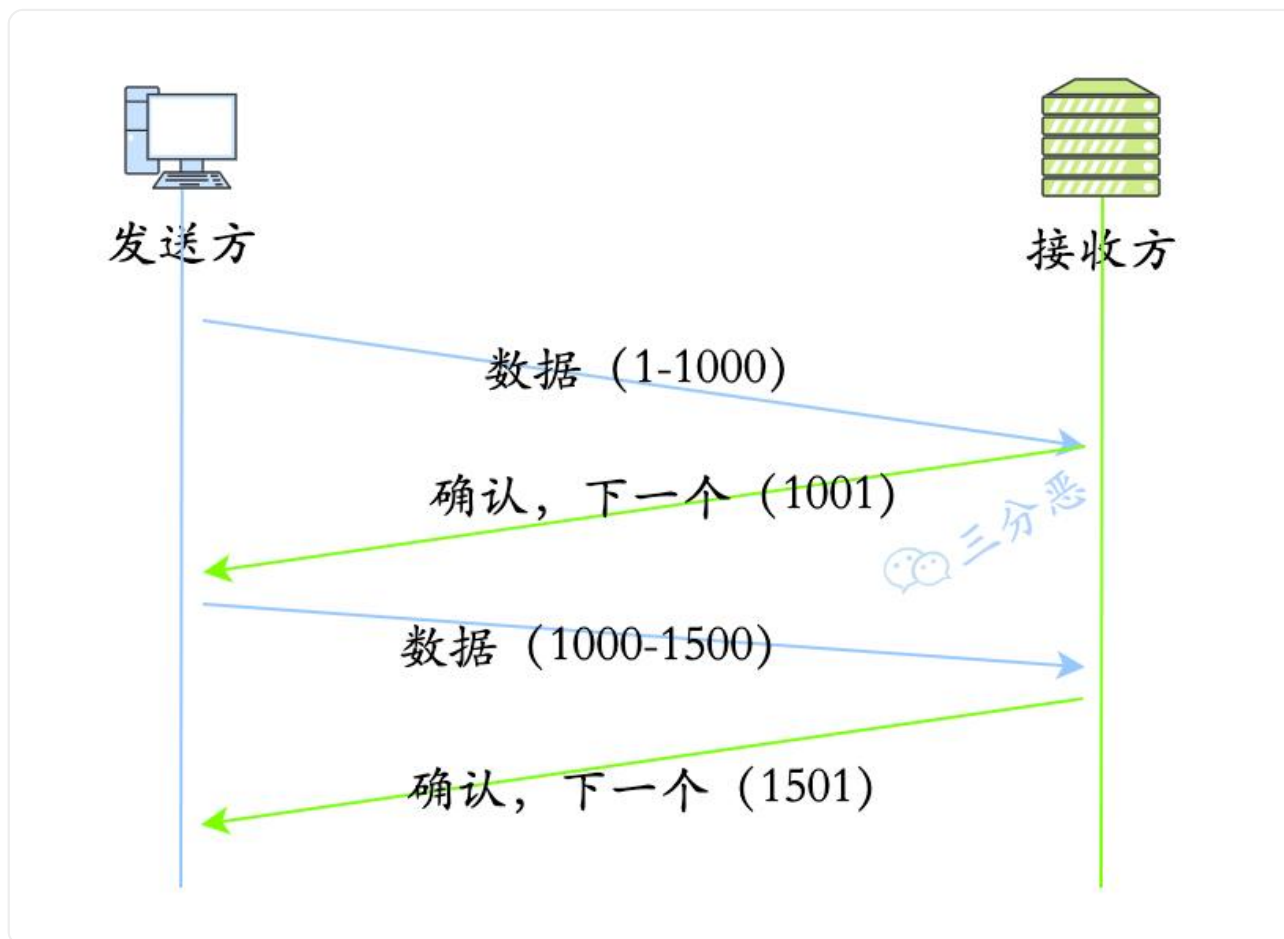
TCP 保证可靠性的方法

1. **连接管理**: TCP 使用三次握手和四次挥手保证可靠地建立连接和释放连接, 这里就不用多说了。
2. **校验和**: TCP 将保持它首部和数据的检验和。这是一个端到端的检验和, 目的是检测数据在传输过程中的任何变化。如果接收端的检验和有差错, TCP 将丢弃这个报文段和不确认收到此报文段。



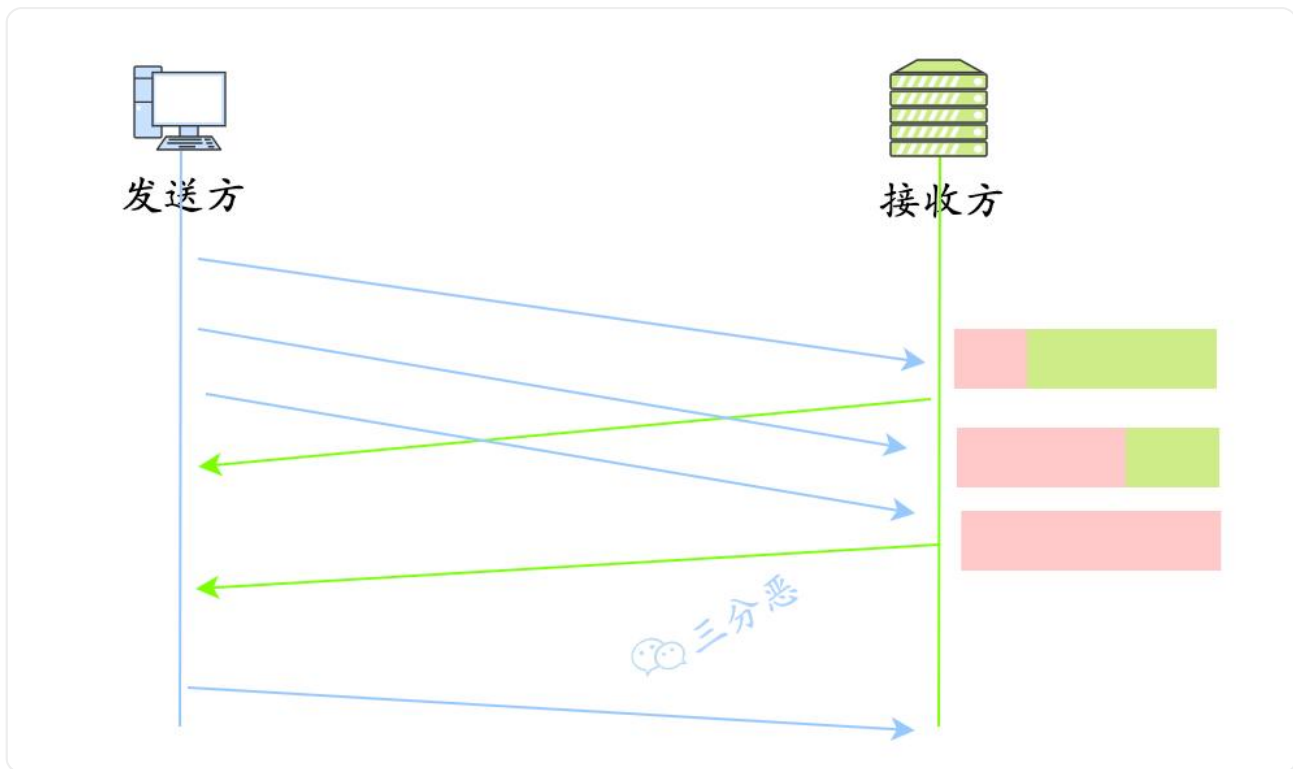
TCP 校验和

3. **序列号/确认应答**：TCP 给发送的每一个包进行编号，接收方会对收到的包进行应答，发送方就会知道接收方是否收到对应的包，如果发现没有收到，就会重发，这样就能保证数据的完整性。就像老师上课，会问一句，这一章听懂了吗？没听懂再讲一遍。



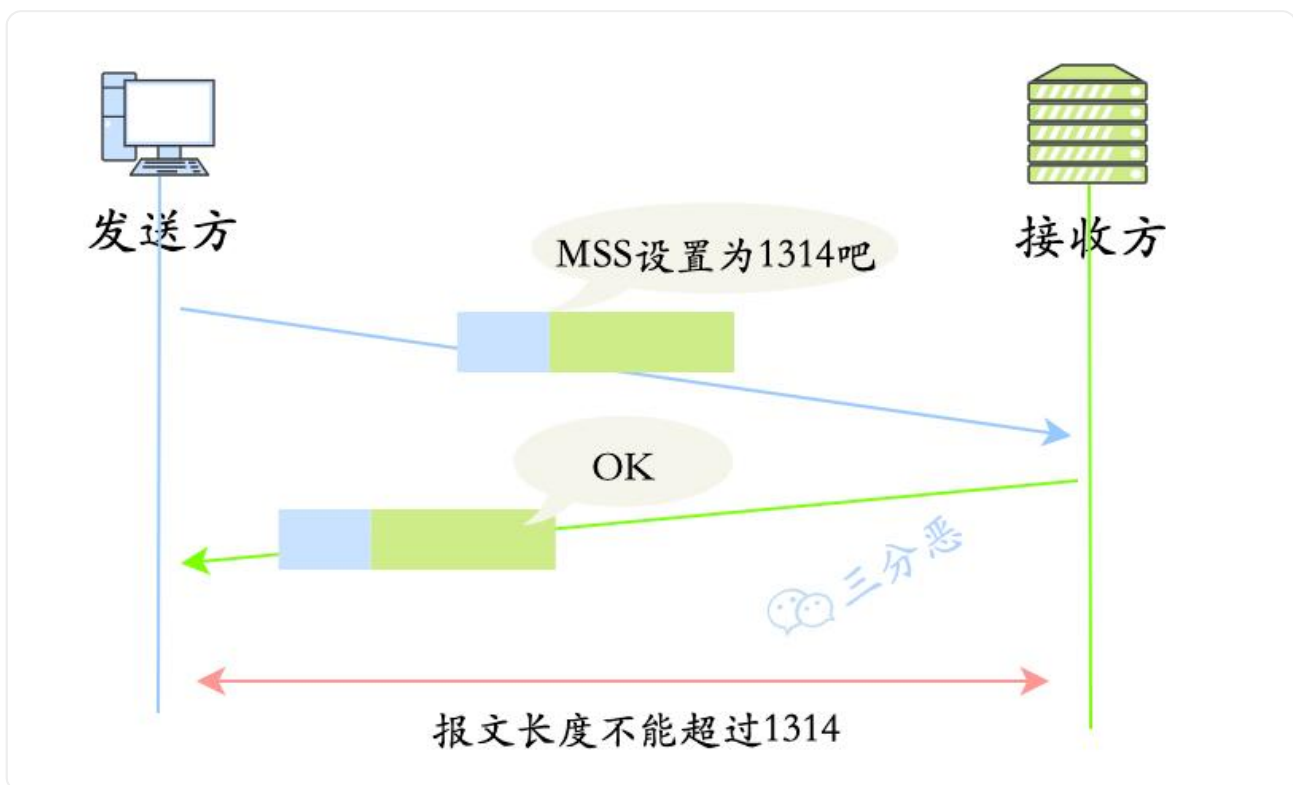
序列号/确认应答

4. ****流量控制**：**TCP 连接的每一方都有固定大小的缓冲空间，TCP 的接收端只允许发送端发送接收端缓冲区能接纳的数据。当接收方来不及处理发送方的数据，能提示发送方降低发送的速率，防止包丢失。TCP 使用的流量控制协议是可变大小的滑动窗口协议。（TCP 利用滑动窗口实现流量控制）



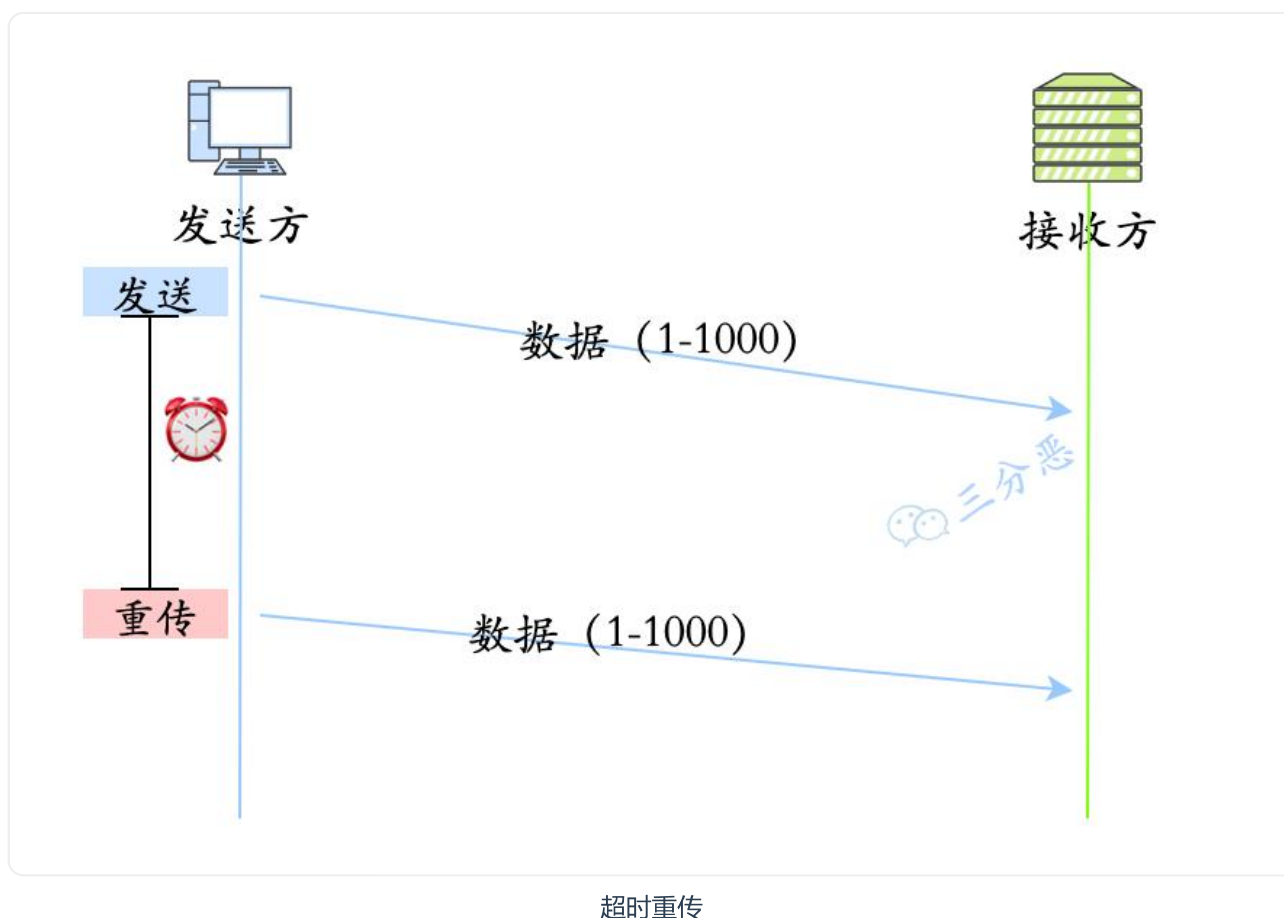
滑动窗口简图

5. **最大消息长度**：在建立 TCP 连接的时候，双方约定一个最大的长度（MSS）作为发送的单位，重传的时候也是以这个单位来进行重传。理想的情况下是该长度的数据刚好不被网络层分块。

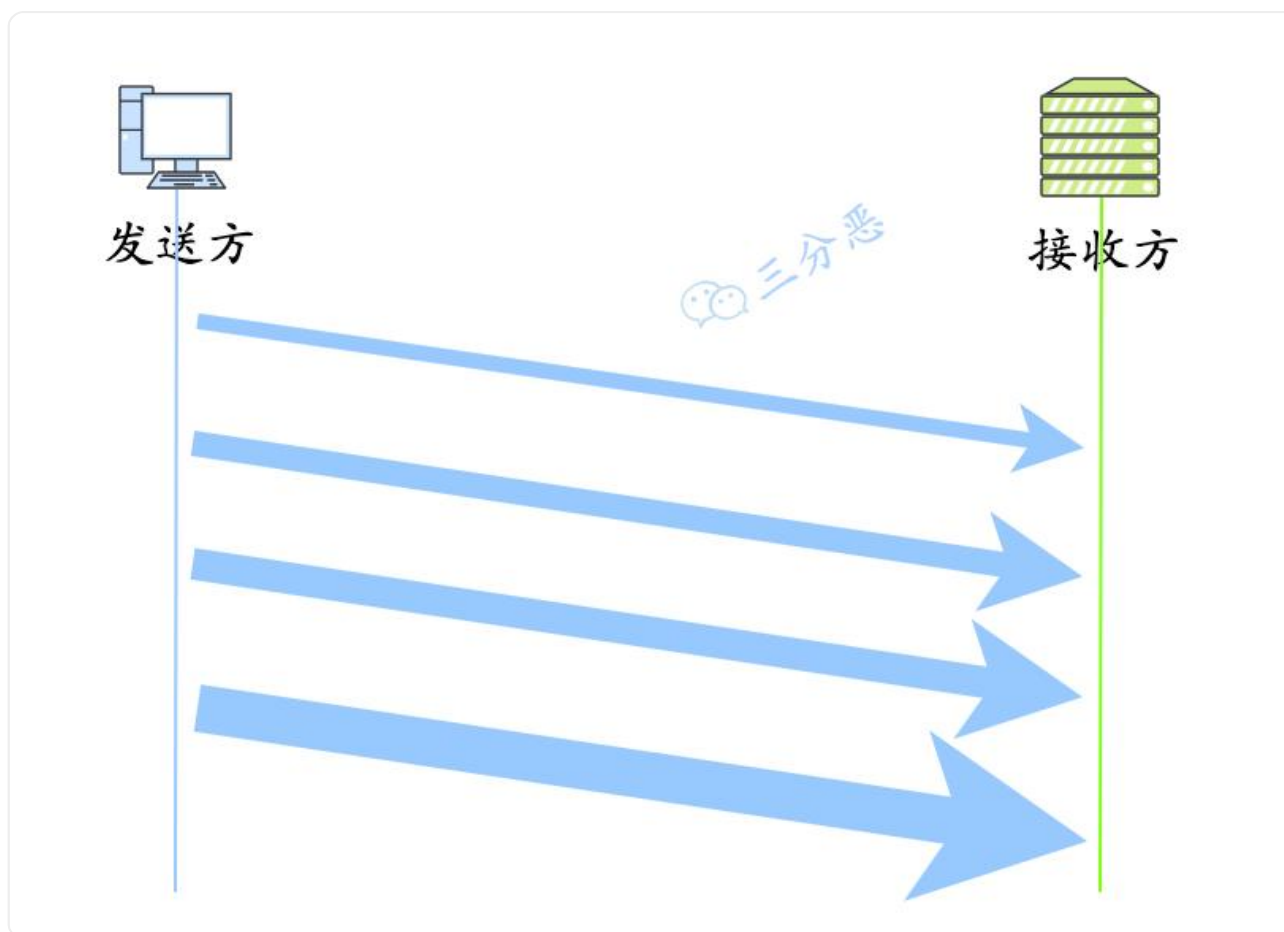


最大消息长度

6. **超时重传**：**超时重传**是指发送出去的数据包到接收到确认包之间的时间，如果超过了这个时间会被认为是丢包了，需要重传。



7. **拥塞控制**：**如果网络非常拥堵，此时再发送数据就会加重网络负担，那么发送的数据段很可能超过了最大生存时间也没有到达接收方，就会产生丢包问题。为此 TCP 引入慢启动机制，先发出少量数据，就像探路一样，先摸清当前的网络拥堵状态后，再决定按照多大的速度传送数据。**



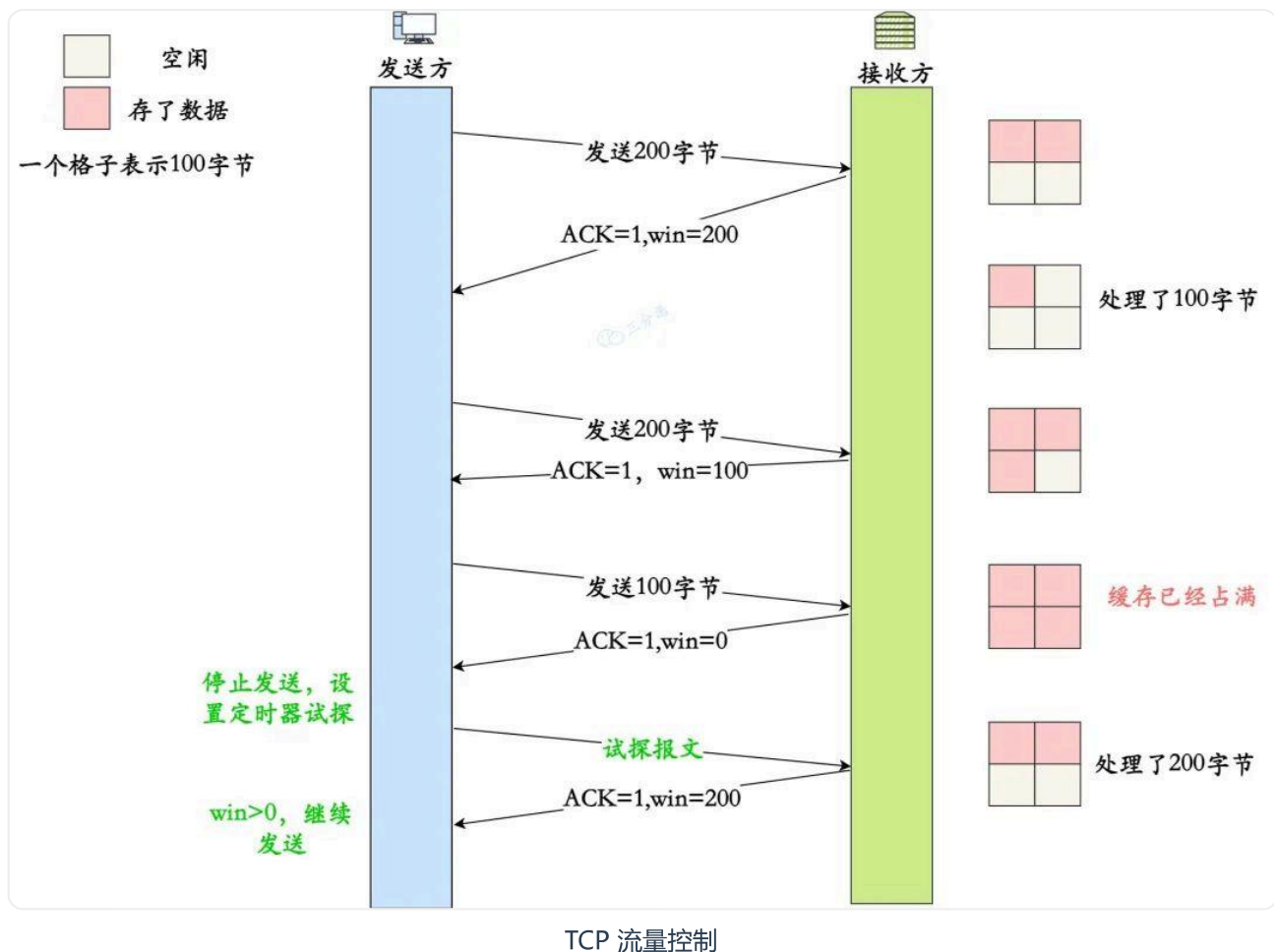
拥塞控制简略示意图

38.说说 TCP 的流量控制?

TCP 提供了一种机制, 可以让发送端根据接收端的实际接收能力控制发送的数据量, 这就是**流量控制**。

TCP 通过**滑动窗口**来控制流量, 我们看下简要流程:

- 首先双方三次握手, 初始化各自的窗口大小, 均为 400 个字节。



- 假如当前发送方给接收方发送了 200 个字节，那么，发送方的 `SND.NXT` 会右移 200 个字节，也就是说当前的可用窗口减少了 200 个字节。
- 接受方收到后，放到缓冲队列里面， $REV.WND = 400 - 200 = 200$ 字节，所以 `win=200` 字节返回给发送方。接收方会在 ACK 的报文首部带上缩小后的滑动窗口 200 字节
- 发送方又发送 200 字节过来，200 字节到达，继续放到缓冲队列。不过这时候，由于大量负载的原因，接受方处理不了这么多字节，只能处理 100 字节，剩余的 100 字节继续放到缓冲队列。这时候， $REV.WND = 400 - 200 - 100 = 100$ 字节，即 `win=100` 返回发送方。
- 发送方继续发送 100 字节过来，这时候，接收窗口 `win` 变为 0。
- 发送方停止发送，开启一个定时任务，每隔一段时间，就去询问接受方，直到 `win` 大于 0，才继续开始发送。

39.详细说说 TCP 的滑动窗口？

TCP 发送一个数据，如果需要收到确认应答，才会发送下一个数据。这样的话就会有缺点：效率会比较低。

“用一个比喻，我们在微信上聊天，你打完一句话，我回复一句之后，你才能打下一句。假如我没有及时回复呢？你是把话憋着不说吗？然后傻傻等到我回复之后再接着发下一句？”

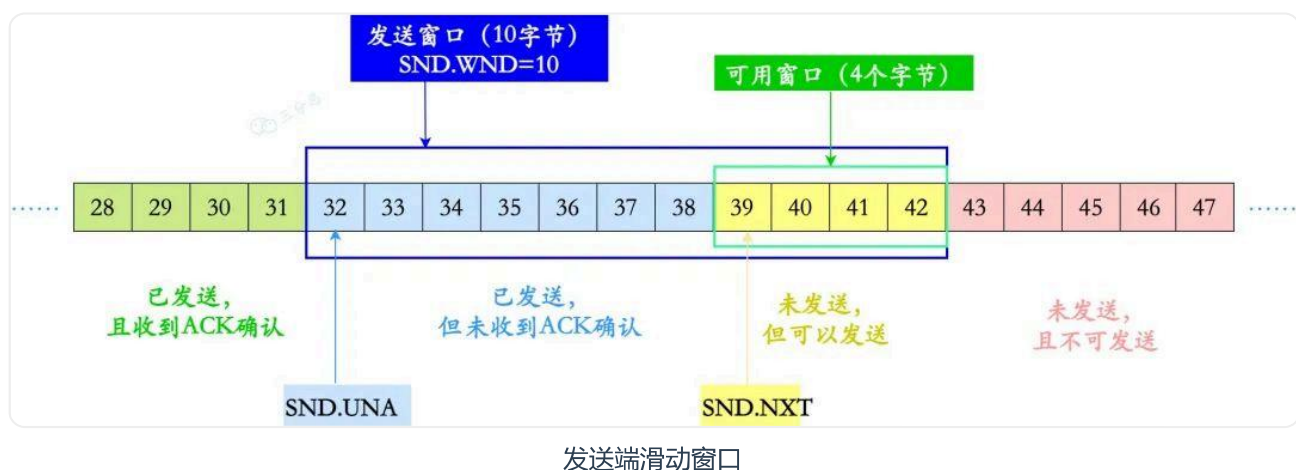
为了解决这个问题, TCP 引入了**窗口**, 它是操作系统开辟的一个缓存空间。窗口大小值表示无需等待确认应答, 而可以继续发送数据的最大值。

TCP 头部有个字段叫 win, 也即那个 **16 位的窗口大小**, 它告诉对方本端的 TCP 接收缓冲区还能容纳多少字节的数据, 这样对方就可以控制发送数据的速度, 从而达到**流量控制**的目的。

“通俗点讲, 就是接受方每次收到数据包, 在发送确认报文的时候, 同时告诉发送方, 自己的缓存区还有多少空余空间, 缓冲区的空余空间, 我们就称之为接受窗口大小。这就是 win。”

TCP 滑动窗口分为两种: 发送窗口和接收窗口。**发送端的滑动窗口**包含四大部分, 如下:

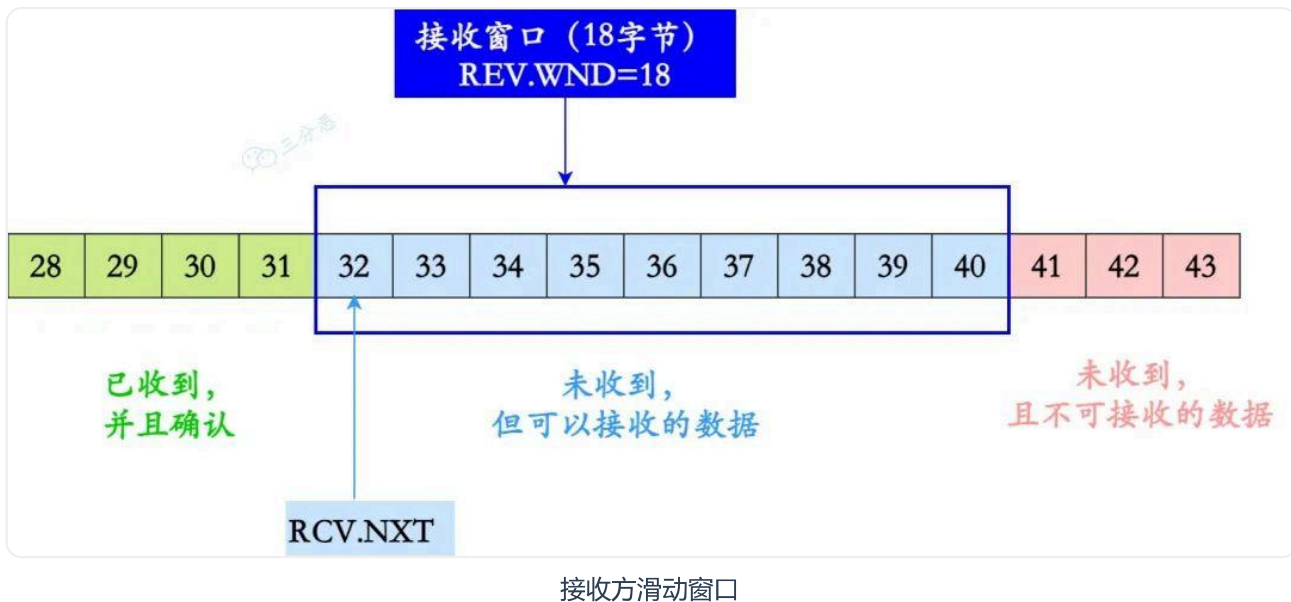
- 已发送且已收到 ACK 确认
- 已发送但未收到 ACK 确认
- 未发送但可以发送
- 未发送也不可以发送



- 深蓝色框里就是发送窗口。
- SND.WND: 表示发送窗口的大小, 上图虚线框的格子数是 10 个, 即发送窗口大小是 10。
- SND.NXT: 下一个发送的位置, 它指向未发送但可以发送的第一个字节的序列号。
- SND.UNA: 一个绝对指针, 它指向的是已发送但未确认的第一个字节的序列号。

接收方的滑动窗口包含三大部分, 如下:

- 已成功接收并确认
- 未收到数据但可以接收
- 未收到数据并不可以接收的数据

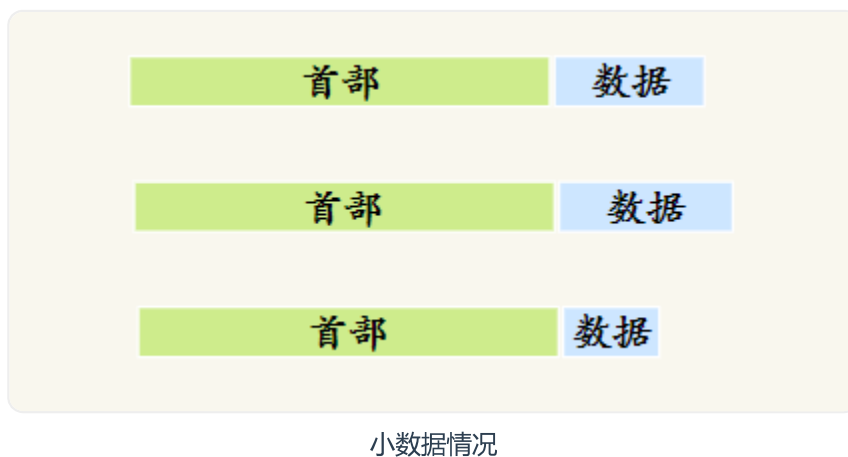


- 蓝色框内，就是接收窗口。
- REV.WND: 表示接收窗口的大小, 上图虚线框的格子就是 9 个。
- RCV.NXT: 下一个接收的位置，它指向未收到但可以接收的第一个字节的序列号。

40.了解 Nagle 算法和延迟确认吗？

Nagle 算法和延迟确认是干什么的？

当我们 TCP 报文的承载的数据非常小的时候，例如几个字节，那么整个网络的效率是很低的，因为每个 TCP 报文中都会有 20 个字节的 TCP 头部，也会有 20 个字节的 IP 头部，而数据只有几个字节，所以在整个报文中有效数据占有的比例就会非常低。



这就好像快递员开着大货车送一个小包裹一样浪费。

那么就出现了常见的两种策略，来减少小报文的传输，分别是：

- Nagle 算法

- 延迟确认

Nagle 算法

Nagle 算法：**任意时刻，最多只能有一个未被确认的小段**。所谓“小段”，指的是小于 MSS 尺寸的数据块，所谓“未被确认”，是指一个数据块发送出去后，没有收到对方发送的 ACK 确认该数据已收到。

Nagle 算法的策略：

- 没有已发送未确认报文时，立刻发送数据。
- 存在未确认报文时，直到「没有已发送未确认报文」或「数据长度达到 MSS 大小」时，再发送数据。

只要没满足上面条件中的一条，发送方一直在囤积数据，直到满足上面的发送条件。

延迟确认

事实上当没有携带数据的 ACK，它的网络效率也是很低的，因为它也有 40 个字节的 IP 头和 TCP 头，但却没有携带数据报文。

为了解决 ACK 传输效率低问题，所以就衍生出了 **TCP 延迟确认**。

TCP 延迟确认的策略：

- 当有响应数据要发送时，ACK 会随着响应数据一起立刻发送给对方
- 当没有响应数据要发送时，ACK 将会延迟一段时间，以等待是否有响应数据可以一起发送
- 如果在延迟等待发送 ACK 期间，对方的第二个数据报文又到达了，这时就会立刻发送 ACK

一般情况下，**Nagle 算法和延迟确认不能一起使用**，Nagle 算法意味着延迟发，**延迟确认**意味着延迟接收，两个凑在一起就会造成更大的延迟，会产生性能问题。

41.说说 TCP 的拥塞控制？

什么是拥塞控制？不是有了流量控制吗？

前面的流量控制是避免发送方的数据填满接收方的缓存，但是并不知道整个网络之中发生了什么。

一般来说，计算机网络都处在一个共享的环境。因此也有可能会因为其他主机之间的通信使得网络拥堵。

在网络出现拥堵时，如果继续发送大量数据包，可能会导致数据包时延、丢失等，这时 **TCP** 就会重传数据，但是一重传就会导致网络的负担更重，于是会导致更大的延迟以及更多的丢包，这个情况就会进入恶性循环被不断地放大....

所以，TCP 不能忽略整个网络中发生的事，它被设计成一个无私的协议，当网络发送拥塞时，TCP 会自我牺牲，降低发送的数据流。

于是，就有了拥塞控制，控制的目的是避免发送方的数据填满整个网络。

就像是一个水管，不能让太多的水（数据流）流入水管，如果超过水管的承受能力，水管会被撑爆（丢包）。



破解的水管-图片来源网络

发送方维护一个**拥塞窗口 cwnd (congestion window)** 的变量，调节所要发送数据的量。

什么是拥塞窗口？和发送窗口有什么关系呢？

拥塞窗口 **cwnd** 是发送方维护的一个的状态变量，它会根据网络的拥塞程度动态变化的。

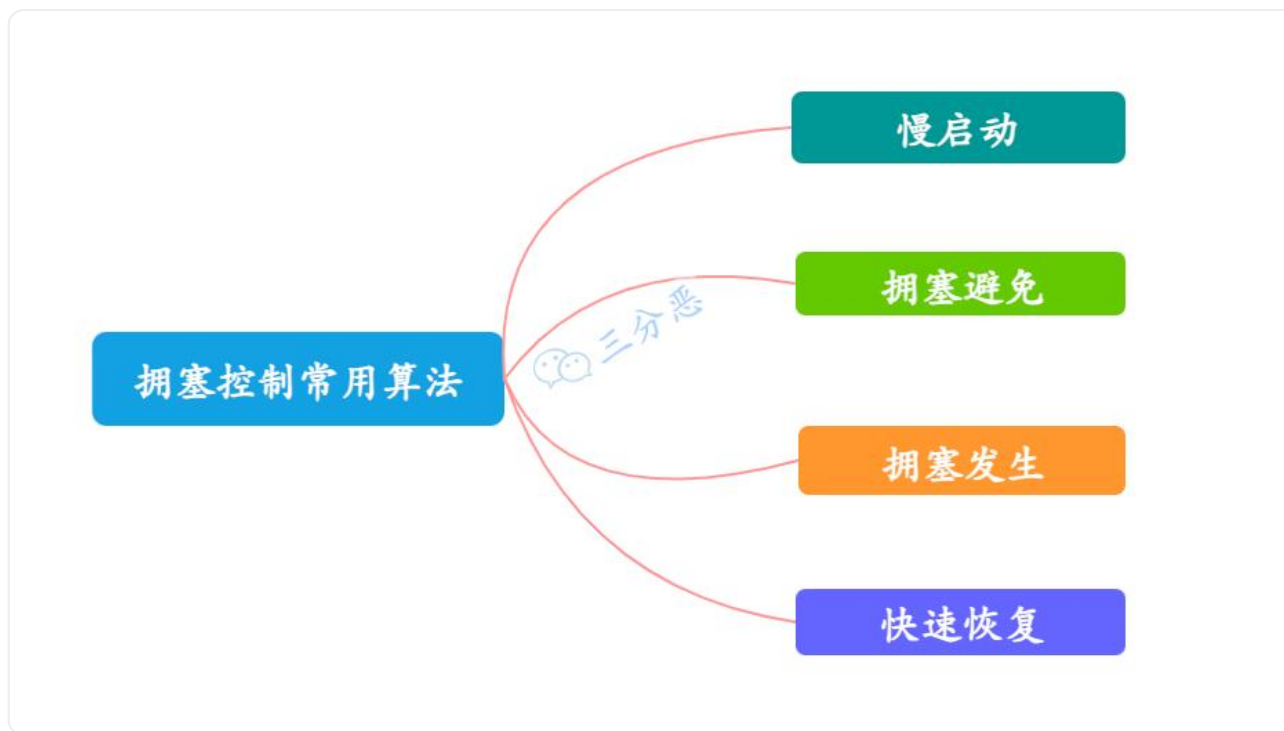
发送窗口 **swnd** 和接收窗口 **rwnd** 是约等于的关系，那么由于加入了拥塞窗口的概念后，此时发送窗口的值是 $swnd = \min(cwnd, rwnd)$ ，也就是拥塞窗口和接收窗口中的最小值。

拥塞窗口 **cwnd** 变化的规则：

- 只要网络中没有出现拥塞，**cwnd** 就会增大；
- 但网络中出现了拥塞，**cwnd** 就减少；

拥塞控制有哪些常用算法？

拥塞控制主要有这几种常用算法：



拥塞控制常用算法

- 慢启动
- 拥塞避免
- 拥塞发生
- 快速恢复

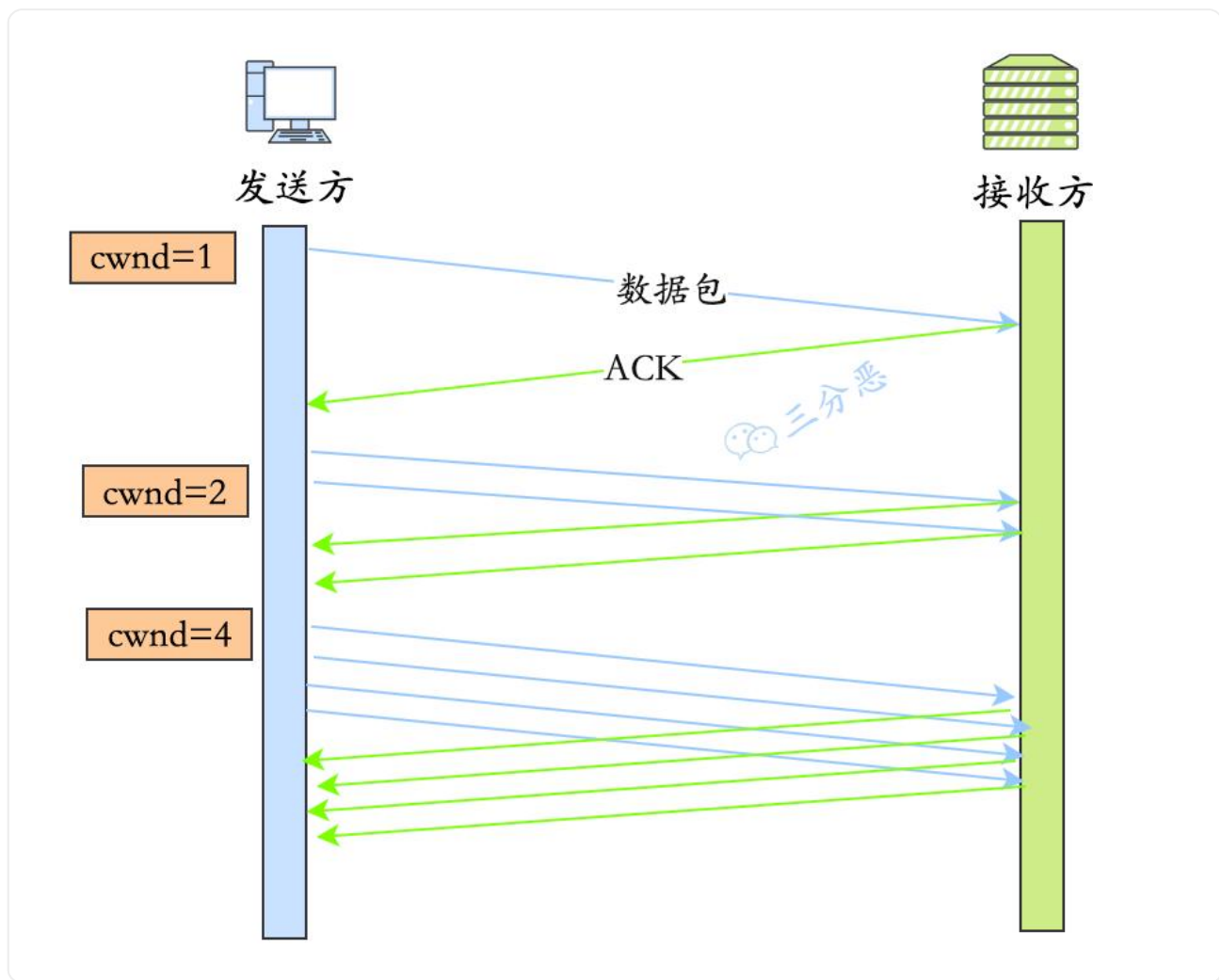
慢启动算法

慢启动算法，慢慢启动。

它表示 TCP 建立连接完成后，一开始不要发送大量的数据，而是先探测一下网络的拥塞程度。由小到大逐渐增加拥塞窗口的大小，如果没有出现丢包，**每收到一个 ACK，就将拥塞窗口 cwnd 大小就加 1（单位是 MSS）**。每轮次发送窗口增加一倍，呈指数增长，如果出现丢包，拥塞窗口就减半，进入拥塞避免阶段。

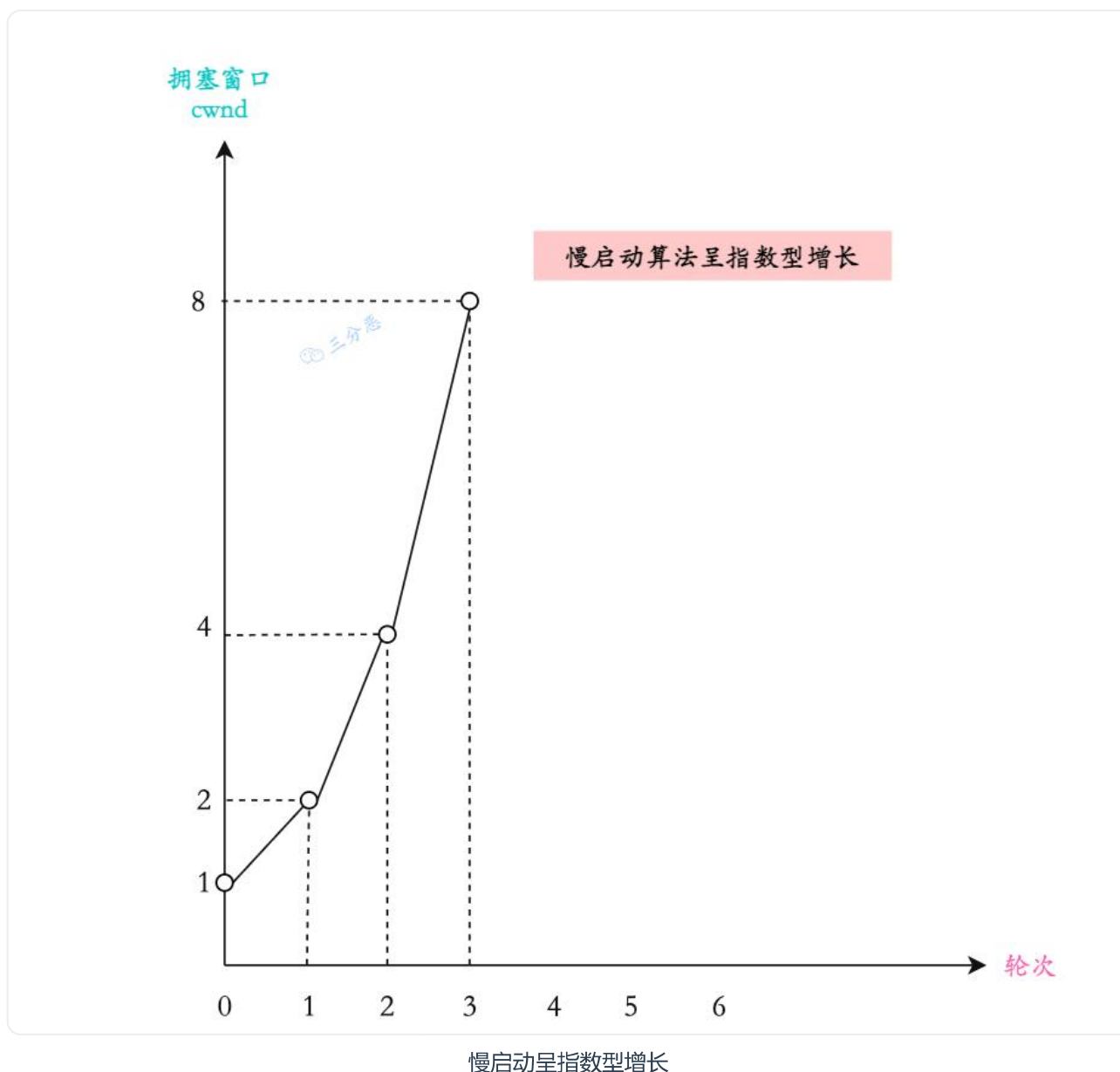
举个例子：

- 连接建立完成后，一开始初始化 $cwnd = 1$ ，表示可以传一个 MSS 大小的数据。
- 当收到一个 ACK 确认应答后， $cwnd$ 增加 1，于是一次能够发送 2 个
- 当收到 2 个的 ACK 确认应答后， $cwnd$ 增加 2，于是就可以比之前多发 2 个，所以这一次能够发送 4 个
- 当这 4 个的 ACK 确认到来的时候，每个确认 $cwnd$ 增加 1，4 个确认 $cwnd$ 增加 4，于是就可以比之前多发 4 个，所以这一次能够发送 8 个。



慢启动算法

发包的个数是指数性的增长。



为了防止 cwnd 增长过大引起网络拥塞，还需设置一个**慢启动阈值 ssthresh** (slow start threshold) 状态变量。当 cwnd 到达该阈值后，就好像水管被关小了水龙头一样，减少拥塞状态。即当 $cwnd > ssthresh$ 时，进入了**拥塞避免**算法。

拥塞避免算法

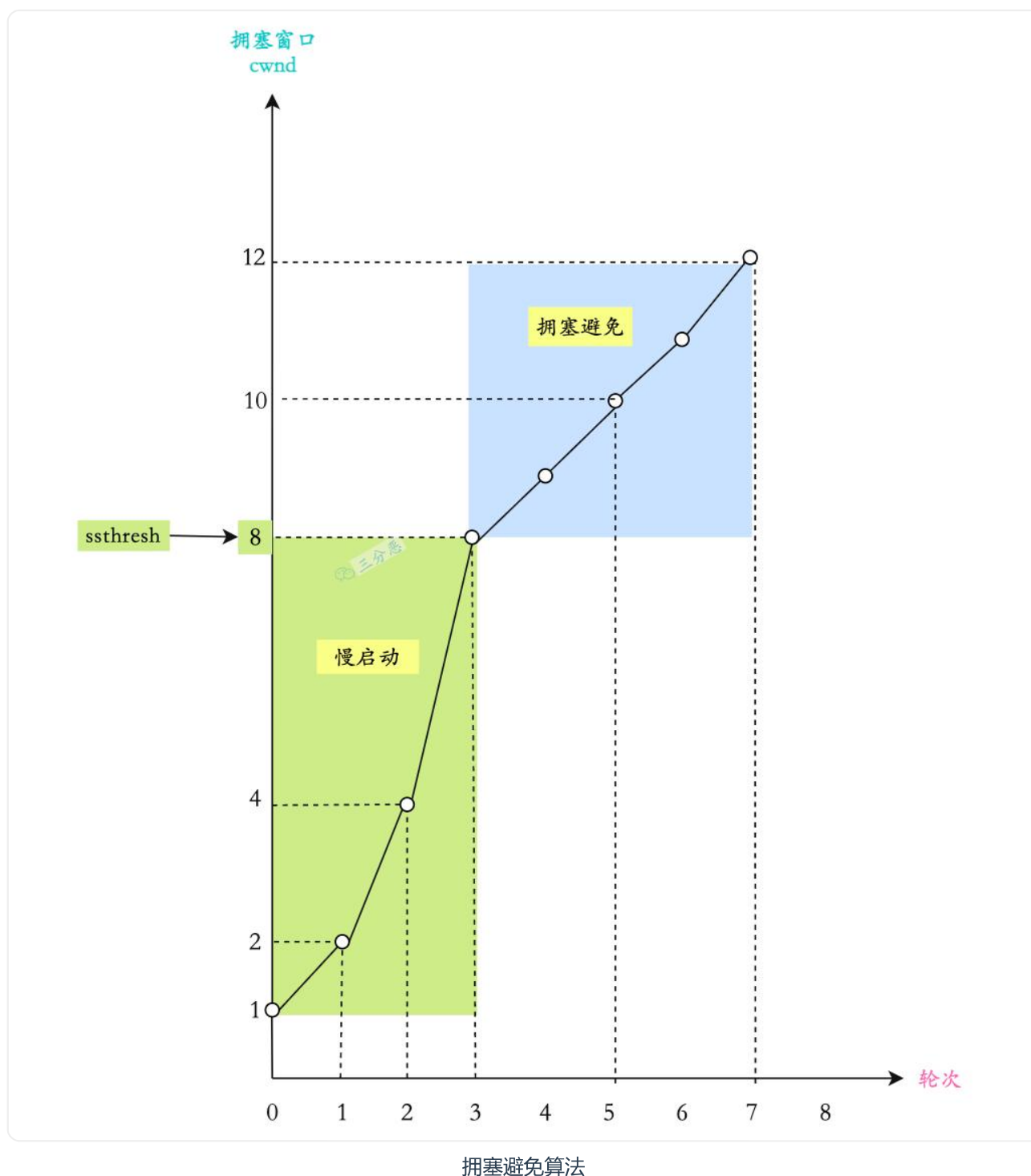
一般来说，慢启动阈值 ssthresh 是 65535 字节，cwnd 到达**慢启动阈值**后

- 每收到一个 ACK 时， $cwnd = cwnd + 1/cwnd$
- 当每过一个 RTT 时， $cwnd = cwnd + 1$

显然这是一个线性上升的算法，避免过快导致网络拥塞问题。

接着上面慢启动的例子，假定 ssthresh 为 8：

- 当 8 个 ACK 应答确认到来时, 每个确认增加 $1/8$, 8 个 ACK 确认 cwnd 一共增加 1, 于是这一次能够发送 9 个 MSS 大小的数据, 变成了线性增长。



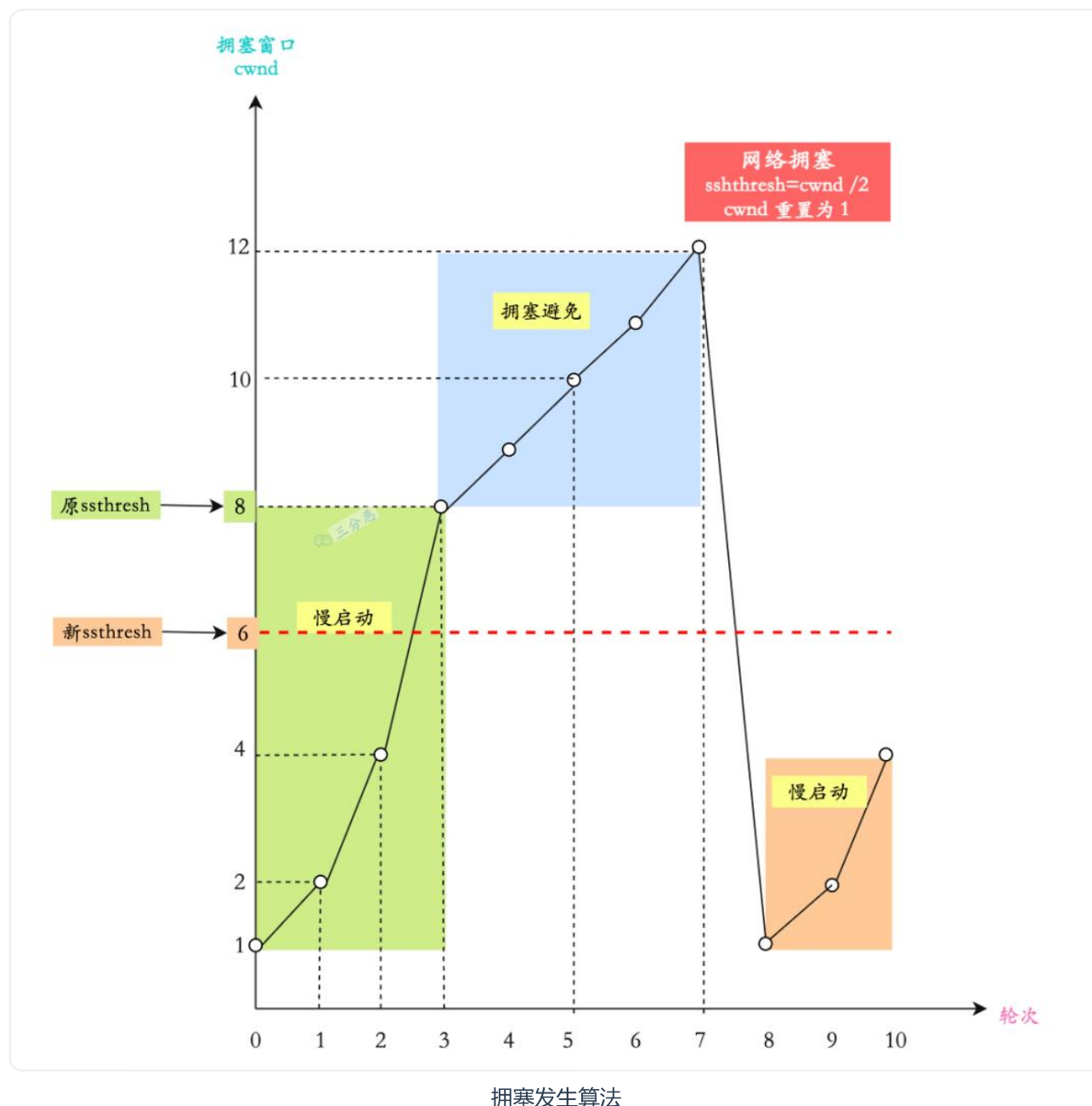
拥塞发生

当网络拥塞发生丢包时, 会有两种情况:

- RTO 超时重传
- 快速重传

如果是发生了 **RTO 超时重传**, 就会使用拥塞发生算法

- 慢启动阈值 $ssthresh = cwnd / 2$
- $cwnd$ 重置为 1
- 进入新的慢启动过程



这种方式就像是飙车的时候急刹车, 还飞速倒车, 这。。。

其实还有更好的处理方式, 就是**快速重传**。发送方收到 3 个连续重复的 ACK 时, 就会快速地重传, 不必等待 **RTO 超时**再重传。

发生快速重传的拥塞发生算法:

- 拥塞窗口大小 $cwnd = cwnd / 2$

- 慢启动阈值 $ssthresh = cwnd$
- 进入快速恢复算法

快速恢复

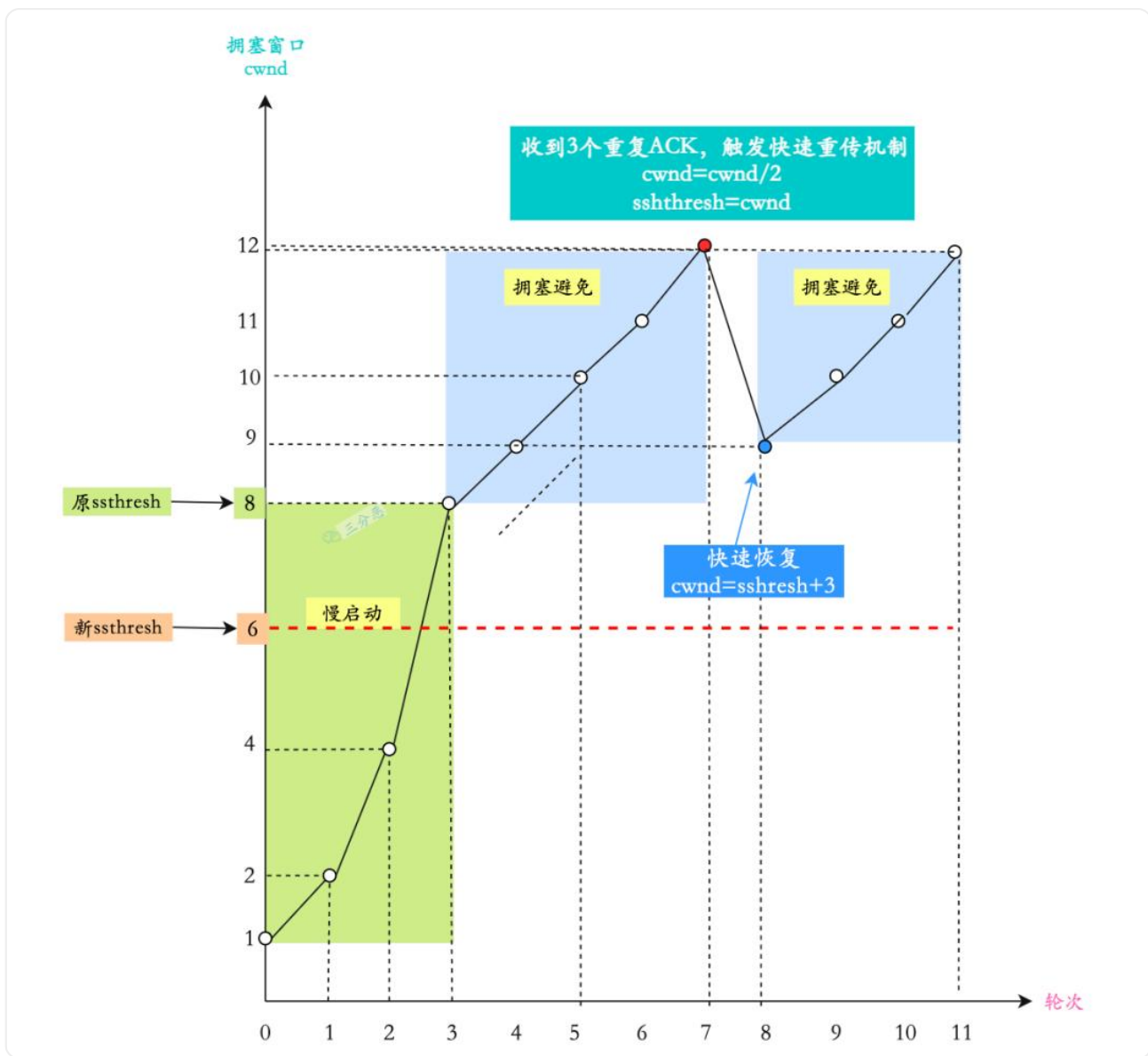
快速重传和快速恢复算法一般同时使用。快速恢复算法认为, 还有 3 个重复 ACK 收到, 说明网络也没那么糟糕, 所以没有必要像 RTO 超时那么强烈。

正如前面所说, 进入快速恢复之前, $cwnd$ 和 $ssthresh$ 已被更新:

- $cwnd = cwnd / 2$
- $ssthresh = cwnd$

然后, 进入快速恢复算法如下:

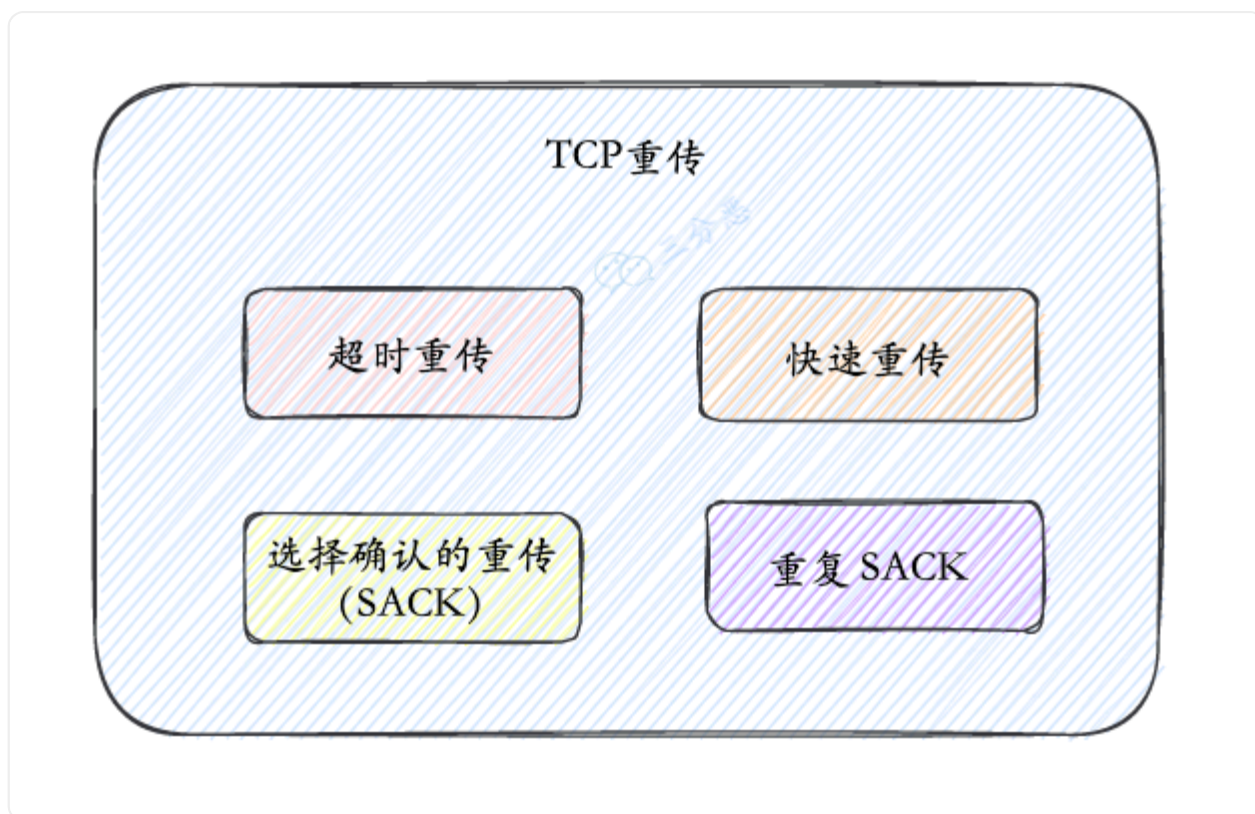
- $cwnd = ssthresh + 3$
- 重传重复的那几个 ACK (即丢失的那几个数据包)
- 如果再收到重复的 ACK, 那么 $cwnd = cwnd + 1$
- 如果收到新数据的 ACK 后, $cwnd = ssthresh$ 。因为收到新数据的 ACK, 表明恢复过程已经结束, 可以再次进入了拥塞避免的算法了。



快速恢复算法

42.说说 TCP 的重传机制？

重传包括**超时重传**、**快速重传**、**带选择确认的重传（SACK）**、**重复 SACK** 四种。



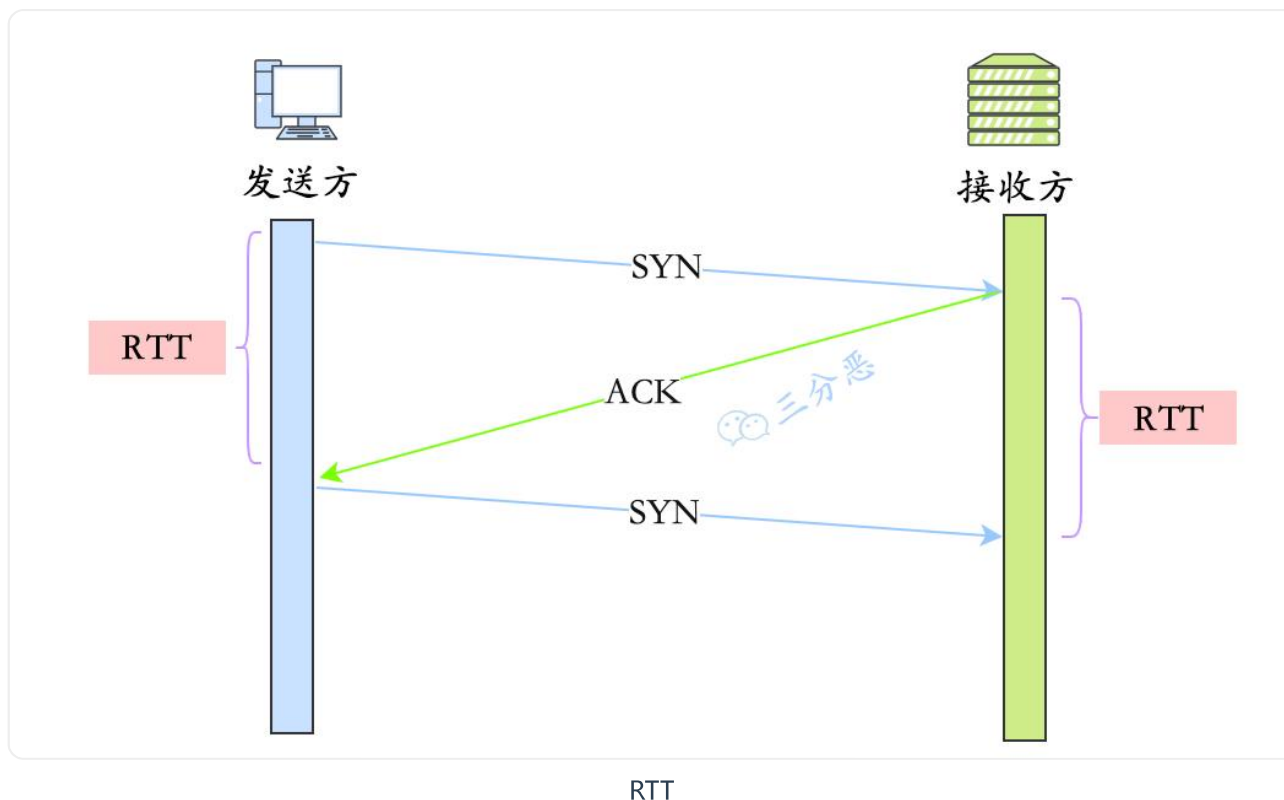
TCP 重传分类

超时重传

超时重传，是 TCP 协议保证数据可靠性的另一个重要机制，其原理是在发送某一个数据以后就开启一个计时器，在一定时间内如果没有得到发送的数据报的 ACK 报文，那么就重新发送数据，直到发送成功为止。

超时时间应该设置为多少呢？

先来看下什么叫 **RTT (Round-Trip Time, 往返时间)**。



RTT 就是数据完全发送完，到收到确认信号的时间，即数据包的一次往返时间。

超时重传时间，就是 RTO (Retransmission Timeout)。那么，**RTO 到底设置多大呢？**

- 如果 RTO 设置很大，等了很久都没重发，这样肯定就不行。
- 如果 RTO 设置很小，那很可能数据都没有丢失，就开始重发了，这会导致网络阻塞，从而恶性循环，导致更多的超时出现。

一般来说，RTO 略微大于 RTT，效果是最佳的。

其实，RTO 有个标准方法的计算公式，也叫 **Jacobson / Karels 算法**。

1. 首先计算 SRTT (即计算平滑的 RTT)

$$SRTT = (1 - \alpha) * SRTT + \alpha * RTT \quad // \text{求 SRTT 的加权平均}$$

text

2. 其次，计算 RTTVAR (round-trip time variation)

$$RTTVAR = (1 - \beta) * RTTVAR + \beta * (|RTT - SRTT|) \quad // \text{计算 SRTT 与真实值的差距}$$

text

3. 最后，得出最终的 RTO

text

$$RTO = \mu * SRTT + \vartheta * RTTVAR = SRTT + 4 * RTTVAR$$

在 Linux 下， $\alpha = 0.125$ ， $\beta = 0.25$ ， $\mu = 1$ ， $\vartheta = 4$ 。别问这些参数是怎么来的，它们是大量实践，调出的最优参数。

超时重传不是十分完美的重传方案，它有这些缺点：

- 当一个报文丢失时，会等待一定的超时周期，才重传分组，增加了端到端的时延。
- 当一个报文丢失时，在其等待超时的过程中，可能会出现这种情况：其后的报文段已经被接收端接收但却迟迟得不到确认，发送端会认为也丢失了，从而引起不必要的重传，既浪费资源也浪费时间。

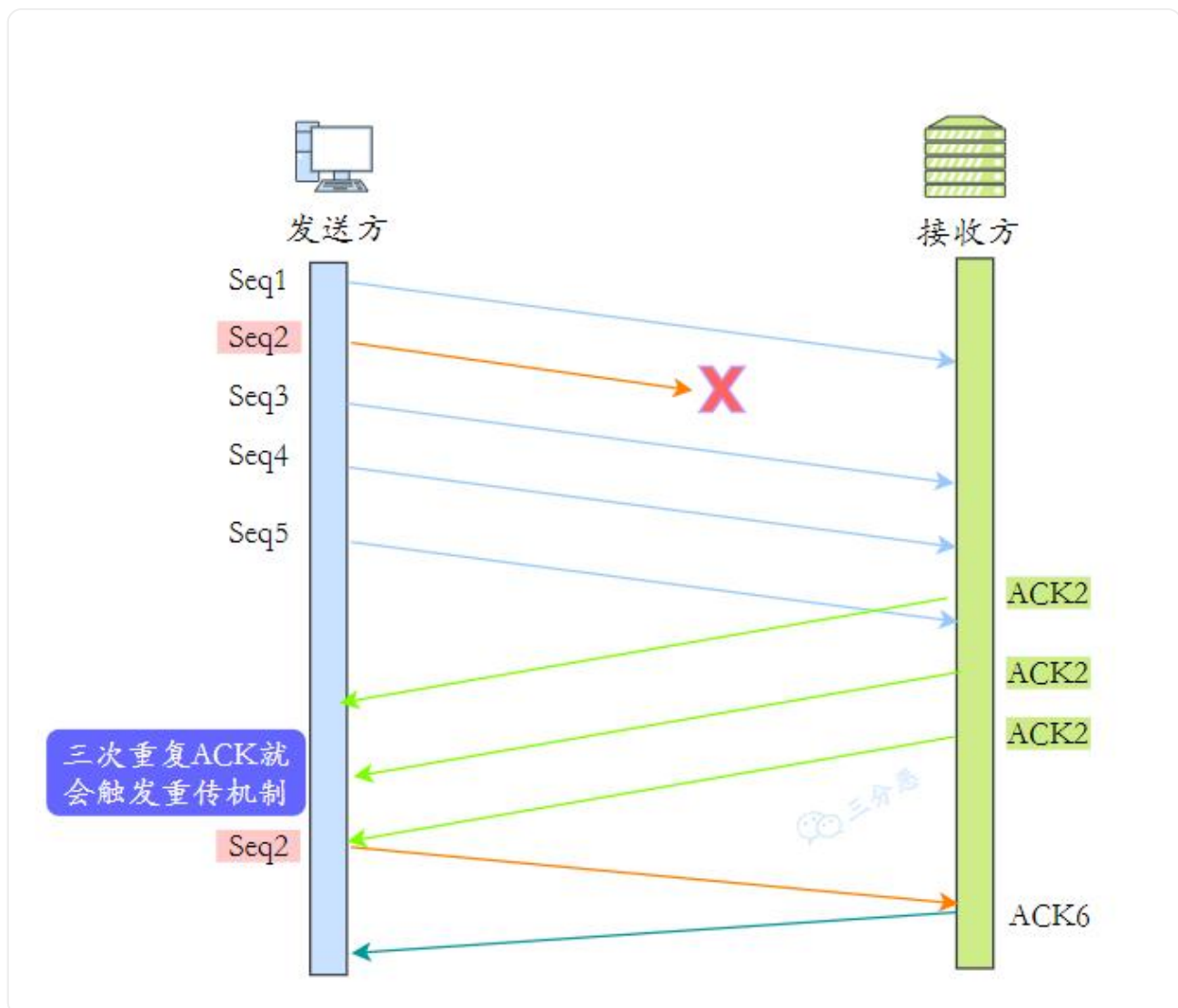
并且，对于 TCP，如果发生一次超时重传，时间间隔下次就会加倍。

快速重传

TCP 还有另外一种快速重传（**Fast Retransmit**）机制，它不以时间为驱动，而是以数据驱动重传。

它不以时间驱动，而是以数据驱动。它是基于接收端的反馈信息来引发重传的。

可以用它来解决超时重发的时间等待问题，快速重传流程如下：



在上图，发送方发出了 1, 2, 3, 4, 5 份数据：

- 第一份 Seq1 先送到了，于是就 Ack 回 2；
- 结果 Seq2 因为某些原因没收到，Seq3 到达了，于是还是 Ack 回 2；
- 后面的 Seq4 和 Seq5 都到了，但还是 Ack 回 2，因为 Seq2 还是没有收到；
- 发送端收到了三个 **Ack = 2** 的确认，知道了 **Seq2** 还没有收到，就会在定时器过期之前，重传丢失的 **Seq2**。
- 最后，收到了 Seq2，此时因为 Seq3, Seq4, Seq5 都收到了，于是 Ack 回 6。

快速重传机制只解决了一个问题，就是超时时间的问题，但是它依然面临着另外一个问题。就是重传的时候，是重传之前的一个，还是重传所有的问题。

比如对于上面的例子，是重传 Seq2 呢？还是重传 Seq2、Seq3、Seq4、Seq5 呢？因为发送端并不清楚这连续的三个 Ack 2 是谁传回来的。

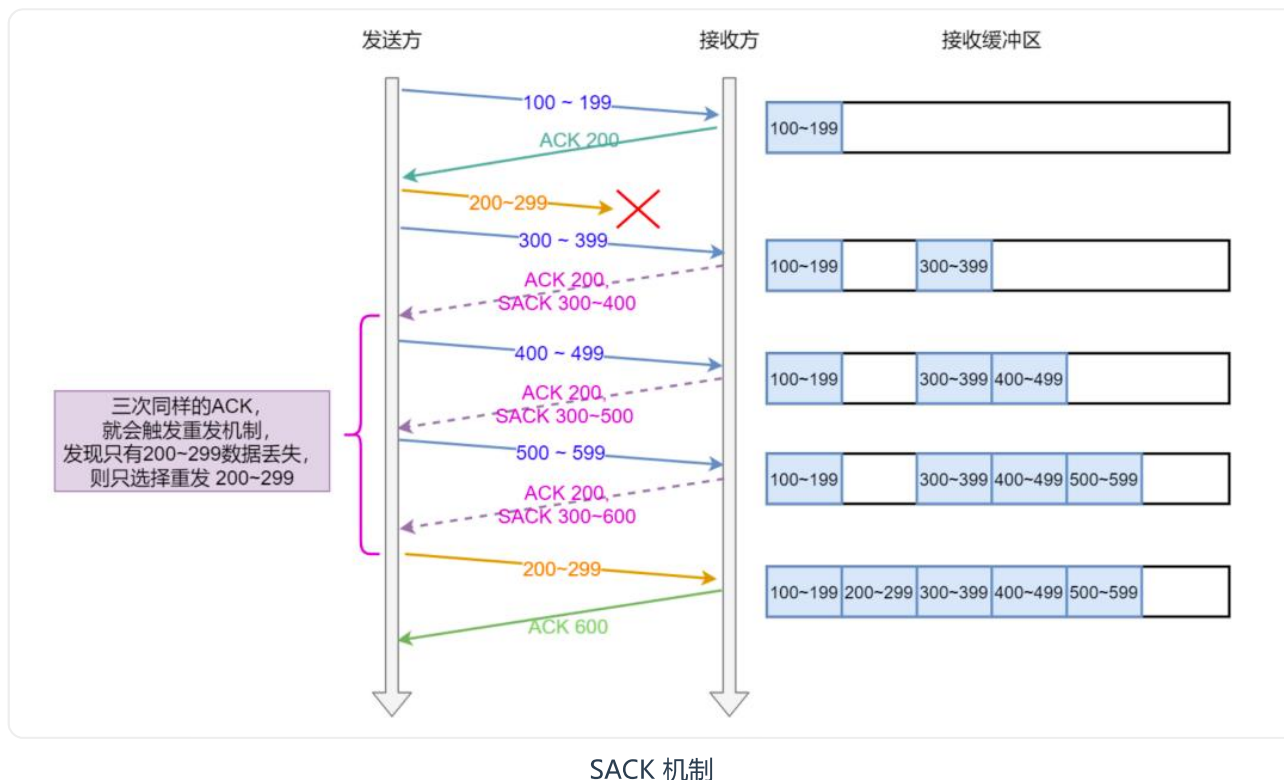
根据 TCP 不同的实现，以上两种情况都是有可能的。可见，这是一把双刃剑。

为了解决不知道该重传哪些 TCP 报文，于是就有 SACK 方法。

带选择确认的重传（SACK）

为了解决应该重传多少个包的问题？TCP 提供了**带选择确认的重传**（即 SACK, Selective Acknowledgment）。

SACK 机制就是，在快速重传的基础上，接收方返回最近收到报文段的序列号范围，这样发送方就知道接收方哪些数据包是没收到的。这样就很清楚应该重传哪些数据包。



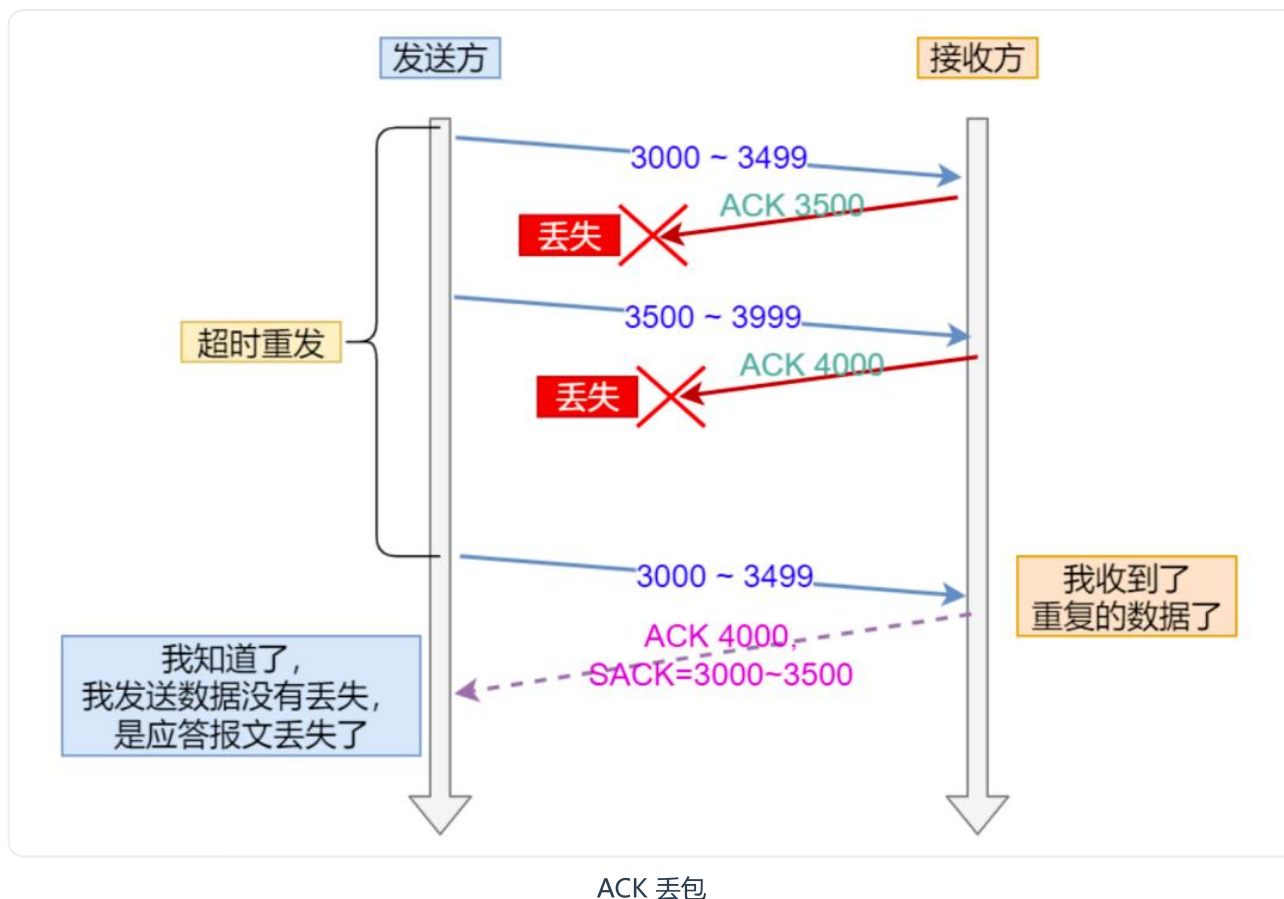
如上图，发送方收到了三次同样的 ACK 确认报文，于是就会触发快速重发机制，通过 SACK 信息发现只有 200~299 这段数据丢失，则重发时，就只选择了这个 TCP 段进行重发。

重复 SACK（D-SACK）

D-SACK，英文是 Duplicate SACK，是在 SACK 的基础上做了一些扩展，主要用来告诉发送方，有哪些数据包，自己重复接受了。

DSACK 的目的是帮助发送方判断，是否发生了包失序、ACK 丢失、包重复或伪重传。让 TCP 可以更好的做网络流控。

例如 ACK 丢包导致的数据包重复：



- 接收方发给发送方的两个 ACK 确认应答都丢失了，所以发送方超时后，重传第一个数据包（3000 ~

3499）

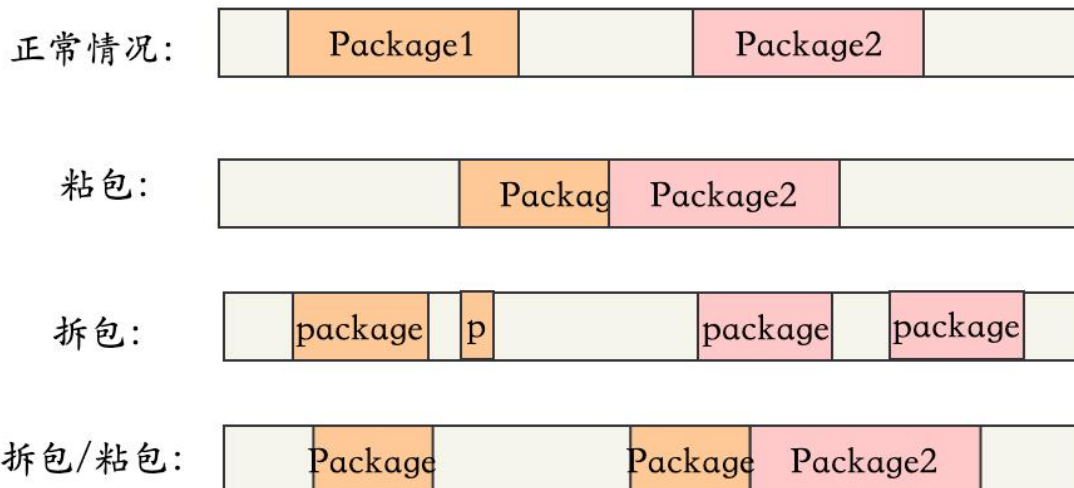
- 于是接收方发现数据是重复收到的，于是回了一个 **SACK = 3000~3500**，告诉「发送方」3000~3500 的数据早已被接收了，因为 ACK 都到了 4000 了，已经意味着 4000 之前的所有数据都已收到，所以这个 SACK 就代表着 D-SACK。这样发送方就知道了，数据没有丢，是接收方的 ACK 确认报文丢了。

43.说说 TCP 的粘包和拆包？

TCP 的粘包和拆包更多的是业务上的概念！

什么是 TCP 粘包和拆包？

TCP 是面向流，没有界限的一串数据。TCP 底层并不了解上层业务数据的具体含义，它会根据 TCP 缓冲区的实际情况进行包的划分，所以在业务上认为，**一个完整的包可能会被 TCP 拆分成多个包进行发送，也有可能把多个小的包封装成一个大的数据包发送**，这就是所谓的 TCP 粘包和拆包问题。



TCP 的粘包和拆包

为什么会产生粘包和拆包呢？

- 要发送的数据小于 TCP 发送缓冲区的大小，TCP 将多次写入缓冲区的数据一次发送出去，将会发生粘包；
- 接收数据端的应用层没有及时读取接收缓冲区中的数据，将发生粘包；
- 要发送的数据大于 TCP 发送缓冲区剩余空间大小，将会发生拆包；
- 待发送数据大于 MSS（最大报文长度），TCP 在传输前将进行拆包。即 TCP 报文长度 - TCP 头部长度 > MSS。

那怎么解决呢？

- 发送端将每个数据包封装为固定长度
- 在数据尾部增加特殊字符进行分割
- 将数据分为两部分，一部分是头部，一部分是内容体；其中头部结构大小固定，且有一个字段声明内容体的大小。

63.一个TCP连接可以发送多少次HTTP请求？（补充）

2024年05月24日新增

一个 TCP 连接可以发送多少次 HTTP 请求，取决于 HTTP 协议的版本。

在 HTTP/1.0 中, 每个 HTTP 请求-响应使用一个单独的 TCP 连接。这意味着每次发送 HTTP 请求都需要建立一个新的 TCP 连接。

HTTP/1.1 引入了持久连接 (Persistent Connection), 默认情况下允许在一个 TCP 连接上发送多个 HTTP 请求。

通过使用 `Connection: keep-alive` 头部实现, 保持连接打开状态, 直到明确关闭为止。这极大地提高了效率, 因为无需为每个请求都建立新的连接。

此外, HTTP/1.1 支持请求管道化 (Pipelining), 允许客户端在收到前一个响应之前发送多个请求。

HTTP/2 进一步优化了连接复用, 允许在单个 TCP 连接上同时发送多个请求和响应, 这些请求和响应被分割成帧并通过流传输。HTTP/2 的多路复用 (Multiplexing) 机制显著提高了并发性能和资源利用效率。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的字节跳动面经同学 1 技术二面面试原题: 一个TCP连接可以发送多少次HTTP请求?

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了! 包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等, 共计 32 万余字, 500+张手绘图, 可以说是通俗易懂、风趣幽默.....详情戳: [太赞了, GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/) (<https://javabetter.cn/overview/>)

微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二, 回复 **222** 即可免费领取。



UDP

UDP 问的不会特别多，基本上是被拿来和 TCP 作比较的。

44.说说 TCP 和 UDP 的区别？

TCP 是面向连接的，而 UDP 是无连接的。

类型	TCP	UDP
是否面向连接	是	否
传输可靠性	可靠	不可靠
传输形式	字节流	数据段报文
传输效率	慢	快
所需资源	多	少
应用场景	文件传输、邮件传输	即时通讯，域名转换
首部字节	20-60	8个字节

三分恶面渣逆袭：TCP 和 UDP 区别

可以这么形容：TCP 是打电话，UDP 是大喇叭（😄）。



三分恶面渣逆袭：TCP 和 UDP 比喻

在数据传输开始之前，TCP 需要先建立连接，数据传输完成后，再断开连接。这个过程通常被称为“三次握手”。

UDP 是无连接的，发送数据之前不需要建立连接，发送完毕也无需断开连接，数据以数据报形式发送。

在此基础上，我们可以得出：TCP 是可靠的，它通过确认机制、重发机制等来保证数据的可靠传输。而 UDP 是不可靠的，数据包可能会丢失、重复、乱序。

说说 TCP 和 UDP 的应用场景？

- **TCP**：适用于那些对数据准确性要求高于数据传输速度的场合。例如：网页浏览、电子邮件、文件传输（FTP）、远程控制、数据库链接。
- **UDP**：适用于对速度要求高、可以容忍一定数据丢失的场合。例如：QQ 聊天、在线视频、网络语音电话、广播通信。容忍一定的数据丢失。

你会如何设计 QQ 中的网络协议？

首先，我们要实现登录功能，这是使用 QQ 的第一步，为了保证账号和密码的安全性，我们可以选择 TCP + SSL/TLS 协议来进行登录。

因为 TCP 协议是一种可靠的传输协议，能够保证数据的完整性，而 SSL/TLS 能够对通信进行加密，保证数据的安全性。

接下来, 我们需要考虑消息传递的实时性, 如语音视频通话等, 这时候我们可以选择 UDP 协议。UDP 的传输速度更快, 对于实时性服务来说, 速度是最重要的。

如何保证消息的不丢失?

对于 TCP 协议来说, 如果数据包在传输过程中丢失, TCP 协议会自动进行重传。

而对于 UDP 协议来说, 我们可以通过应用层的重传机制来保证消息的不丢失。当接收方收到消息后, 返回一个确认信息给发送方, 如果发送方在一定时间内没有收到确认信息, 就重新发送消息。

同时, 每个消息都附带一个唯一的序列号, 接收方根据序列号判断是否有消息丢失, 如果发现序列号不连续, 就可以要求发送方重新发送。这样还可以防止消息重复。

当然了, 消息持久化也很重要, 可以将消息保存在服务器或者本地的数据库中, 即使在网络中断或者其他异常情况下, 也能从数据库中恢复消息。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的 华为一面原题: 说下 TCP 和 UDP 的区别?
2. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的 奇安信面经同学 1 Java 技术一面面试原题: tcp 和 udp 的区别? QQ 用的协议? 它如何保证消息的不丢失?
3. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (https://javabetter.cn/zhishixingqiu/mianshi.html) 收录的 招商银行面经同学 6 招银网络科技面试原题: UDP和TCP的区别?

45.为什么 QQ 采用 UDP 协议?

PS: 这是多年前的老题了, 拉出来怀怀旧。



QQ 使用 UDP

- 首先，QQ 并不是完全基于 UDP 实现。比如在使用 QQ 进行文件传输等活动的时候，就会使用 TCP 作为可靠传输的保证。
- 使用 UDP 进行交互通信的好处在于，延迟较短，对数据丢失的处理比较简单。同时，TCP 是一个全双工协议，需要建立连接，所以网络开销也会相对大。
- 如果使用 QQ 语音和 QQ 视频的话，UDP 的优势就更为突出了，首先延迟较小。最重要的一点是不可靠传输，这意味着如果数据丢失的话，不会有重传。因为用户一般来说可以接受图像稍微模糊一点，声音稍微不清晰一点，但是如果在几秒钟以后再出现之前丢失的画面和声音，这恐怕是很难接受的。
- 由于 QQ 的服务器设计容量是海量级的应用，一台服务器要同时容纳十几万的并发连接，因此服务器端只有采用 UDP 协议与客户端进行通讯才能保证这种超大规模的服务

简单总结一下：UDP 协议是无连接方式的协议，它的效率高，速度快，占资源少，对服务器的压力比较小。但是其传输机制为不可靠传送，必须依靠辅助的算法来完成传输控制。QQ 采用的通信协议以 UDP 为主，辅以 TCP 协议。

46.UDP 协议为什么不可靠？

UDP 在传输数据之前不需要先建立连接，远地主机的运输层在接收到 UDP 报文后，不需要确认，提供不可靠交付。总结就以下四点：

- 不保证消息交付：不确认，不重传，无超时
- 不保证交付顺序：不设置包序号，不重排，不会发生队首阻塞
- 不跟踪连接状态：不必建立连接或重启状态机
- 不进行拥塞控制：不内置客户端或网络反馈机制

47.DNS 为什么要用 UDP?

更准确地说, DNS 既使用 TCP 又使用 UDP。

当进行区域传送(主域名服务器向辅助域名服务器传送变化的那部分数据)时会使用 TCP, 因为数据同步传送的数据量比一个请求和应答的数据量要多, 而 TCP 允许的报文长度更长, 因此为了保证数据的正确性, 会使用基于可靠连接的 TCP。

当客户端想 DNS 服务器查询域名(域名解析)的时候, 一般返回的内容不会超过 UDP 报文的最大长度, 即 512 字节, 用 UDP 传输时, 不需要创建连接, 从而大大提高了响应速度, 但这要求域名解析服务器和域名服务器都必须自己处理超时和重传从而保证可靠性。

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了! 包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等, 共计 32 万余字, 500+张手绘图, 可以说是通俗易懂、风趣幽默.....详情戳: [太赞了, GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/) (<https://javabetter.cn/overview/>)

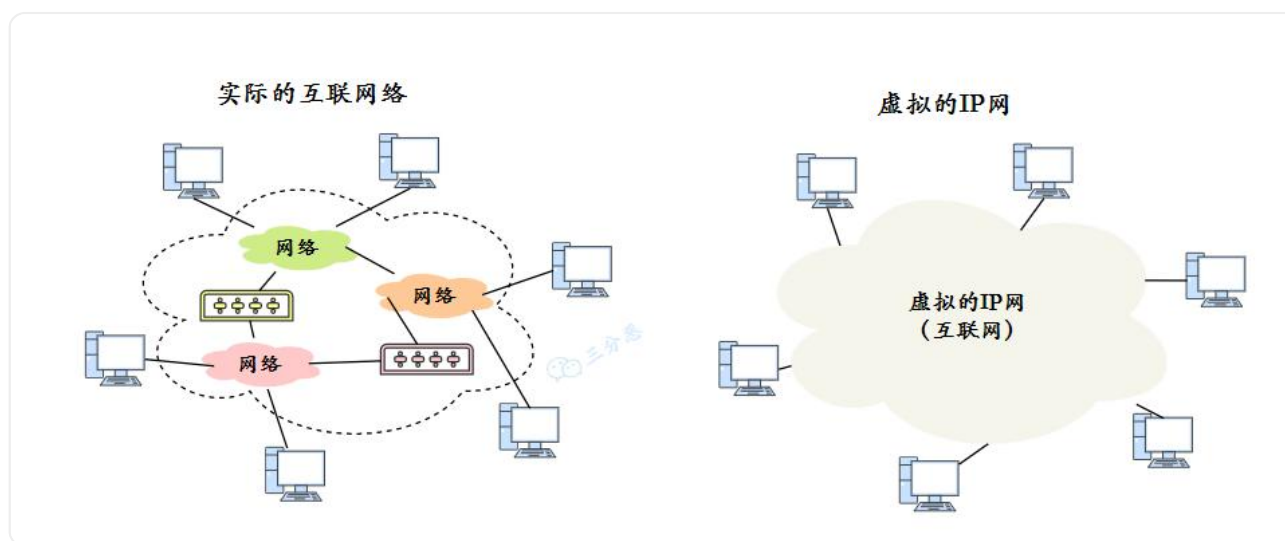
微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二, 回复 **222** 即可免费领取。



IP

48.IP 协议的定义和作用？

IP 协议（Internet Protocol）用于在计算机网络之间传输数据包，它定义了数据包的格式和处理规则，确保数据能够从一个设备传输到另一个设备，可能跨越多个中间网络设备（如路由器）。



三分恶面渣逆袭：虚拟 IP 网

IP 协议有哪些作用？

- ①、**寻址**：每个连接到网络的设备都有一个唯一的 IP 地址。IP 协议使用这些地址来标识数据包的源地址和目的地址，确保数据包能够准确地传输到目标设备。
- ②、**路由**：IP 协议负责决定数据包在网络传输中的路径。比如说路由器使用路由表和 IP 地址信息来确定数据包的最佳传输路径。
- ③、**分片和重组**：当数据包过大无法在某个网络上传输时，IP 协议会将数据包分成更小的片段进行传输。接收端会根据头部信息将这些片段重新组装成完整的数据包。

举一个实际的例子来说明？

假设有两个设备 A 和 B 通过互联网通信，A 的 IP 地址是 192.168.1.1，B 的 IP 地址是 203.0.113.5。数据包的传输过程如下：

①、设备 A 发送数据包：

- 设备 A 创建一个 IP 数据包，设置源地址为 192.168.1.1，目的地址为 203.0.113.5，将要传输的数据放入数据部分。
- 数据包封装后，通过本地网络发送到路由器。

②、路由器转发数据包：

- 路由器根据路由表查找目的地址 203.0.113.5，确定数据包的传输路径。
- 数据包可能经过多个中间路由器，每个路由器都根据路由表选择下一跳，最终到达目标设备的网络。

③、设备 B 接收数据包：

- 设备 B 接收数据包，读取 IP 头部信息，验证数据包的完整性。
- 并数据部分取出，交给上层协议处理（如 TCP 或 UDP）。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的华为面经同学 12 暑期实习一面面试原题：说说IP协议。

49.IP 地址有哪些分类？

一个 IP 地址在这个互联网范围内是惟一的，一般可以这么认为，IP 地址 = {<网络号>，<主机号>}。

1. **网络号**：它标志主机所连接的网络地址表示属于互联网的哪一个网络。
2. **主机号**：它标志主机地址表示其属于该网络中的哪一台主机。

IP 地址分为 A, B, C, D, E 五大类：

- A 类地址 (1~126)：以 0 开头，网络号占前 8 位，主机号占后面 24 位。
- B 类地址 (128~191)：以 10 开头，网络号占前 16 位，主机号占后面 16 位。
- C 类地址 (192~223)：以 110 开头，网络号占前 24 位，主机号占后面 8 位。
- D 类地址 (224~239)：以 1110 开头，保留为多播地址。
- E 类地址 (240~255)：以 1111 开头，保留位为将来使用



IP 地址分类

50.域名和 IP 的关系？一个 IP 可以对应多个域名吗？

- IP 地址在同一个网络中是惟一的，用来标识每一个网络上的设备，其相当于一个人的身份证号
- 域名在同一个网络中也是惟一的，就像是一个人的名字、绰号

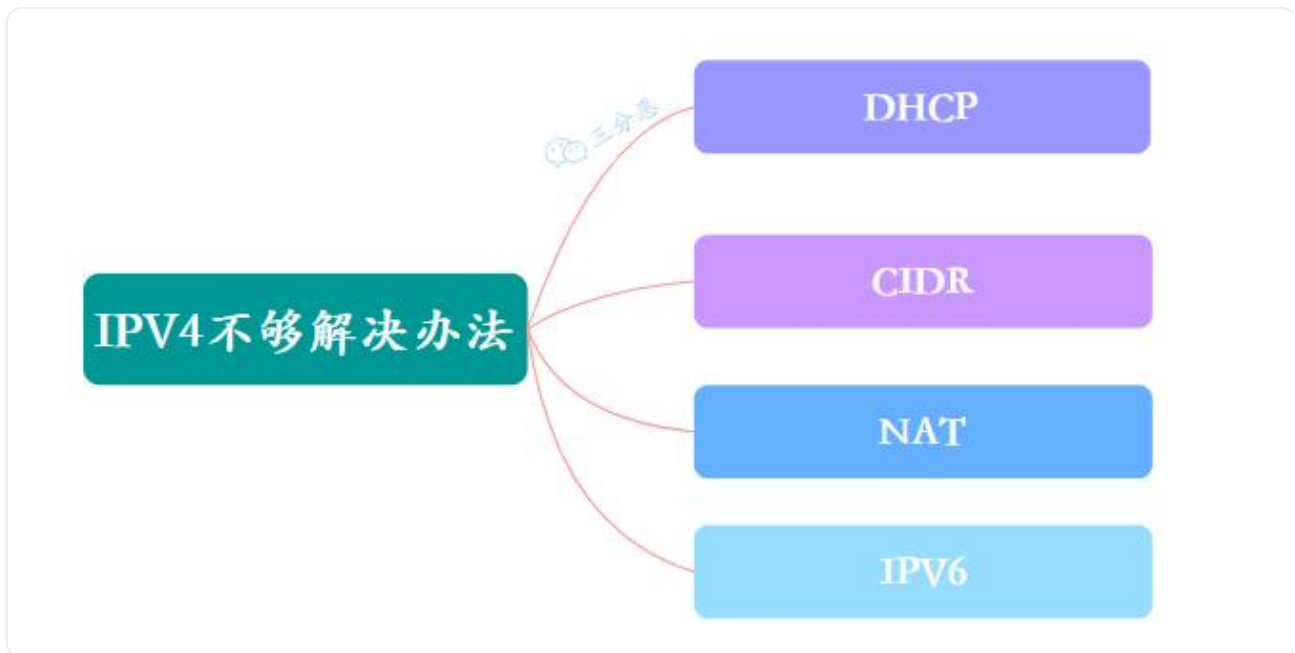
假如你有多个不用的绰号，你的朋友可以用其中任何一个绰号叫你，但你的身份证号码却是惟一的。但同时你的绰号也可能和别人重复，假如你不在，有人叫你的绰号，其它人可能就答应了。

一个域名可以对应多个 IP，但这种情况 DNS 做负载均衡的，在用户访问过程中，一个域名只能对应一个 IP。

而一个 IP 却可以对应多个域名，是一对多的关系。

51.IPV4 地址不够如何解决？

我们知道，IP 地址有 32 位，可以标记 2 的 32 次方个地址，听起来很多，但是全球的网络设备数量已经远远超过这个数字，所以 IPV4 地址已经不够用了，那怎么解决呢？



IPv4 不够解决办法

- DHCP：动态主机配置协议，动态分配 IP 地址，只给接入网络的设备分配 IP 地址，因此同一个 MAC 地址的设备，每次接入互联网时，得到的 IP 地址不一定是相同的，该协议使得空闲的 IP 地址可以得到充分利用。
- CIDR：无类别域间路由。CIDR 消除了传统的 A 类、B 类、C 类地址以及划分子网的概念，因而更加有效地分配 IPv4 的地址空间，但无法从根本上解决地址耗尽的问题。
- NAT：网络地址转换协议，我们知道属于不同局域网的主机可以使用相同的 IP 地址，从而一定程度上缓解了 IP 资源枯竭的问题，然而主机在局域网中使用的 IP 地址是不能在公网中使用的，当局域网主机想要与公网主机进行通信时，NAT 方法可以将该主机 IP 地址转换为全球 IP 地址。该协议能够有效解决 IP 地址不足的问题。
- IPv6：作为接替 IPv4 的下一代互联网协议，其可以实现 2 的 128 次方个地址，而这个数量级，即使给地球上每一粒沙子都分配一个 IP 地址也够用，该协议能够从根本上解决 IPv4 地址不够用的问题。

52.说下 ARP 协议的工作过程？

ARP（Address Resolution Protocol，地址解析协议）是网络通信中的一种协议，主要目的是将网络层的 IP 地址解析为链路层的 MAC 地址。



三分恶面渣逆袭：ARP 协议作用

①、ARP 请求

当主机 A 要发送数据给主机 B 时，首先会在自己的 ARP 缓存中查找主机 B 的 MAC 地址。

如果没有找到，主机 A 会向网络中广播一个 ARP 请求数据包，请求网络中的所有主机告诉它们的 MAC 地址；这个请求包含了请求设备和目标设备的 IP 和 MAC 地址。

②、ARP 应答

网络中的所有主机都会收到这个 ARP 请求，但只有主机 B 会回复 ARP 应答，告诉主机 A 自己的 MAC 地址。

并且主机 B 会将主机 A 的 IP 和 MAC 地址映射关系缓存到自己的 ARP 缓存中，以便下次通信时直接使用。

③、更新 ARP 缓存

主机 A 收到主机 B 的 ARP 应答后，也会将主机 B 的 IP 和 MAC 地址映射关系缓存到自己的 ARP 缓存中。

1. [Java 面试指南（付费）](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>) 收录的快手面经同学 7 Java 后端技术一面面试原题：说一下 ARP 协议的过程

53.为什么既有 IP 地址，又有 MAC 地址？

MAC 地址和 IP 地址都有什么作用？

- MAC 地址是数据链路层和物理层使用的地址，是写在网卡上的物理地址，用来定义网络设备的位置，不可变更。
- IP 地址是网络层和以上各层使用的地址，是一种逻辑地址。IP 地址用来区别网络上的计算机。

为什么有了 MAC 地址还需要 IP 地址？

如果我们只使用 MAC 地址进行寻址的话，我们需要路由器记住每个 MAC 地址属于哪个子网，不然一次路由器收到数据包都要满世界寻找目的 MAC 地址。而我们知道 MAC 地址的长度为 48 位，也就是最多共有 2^{48} 个 MAC 地址，这就意味着每个路由器需要 256T 的内存，显然是不现实的。

和 MAC 地址不同，IP 地址是和地域相关的，在一个子网中的设备，我们给其分配的 IP 地址前缀都是一样的，这样路由器就能根据 IP 地址的前缀知道这个设备属于哪个子网，剩下的寻址就交给子网内部实现，从而大大减少了路由器所需要的内存。

为什么有了 IP 地址还需要 MAC 地址？



IP 地址和 MAC 地址

- 只有当设备连入网络时，才能根据他进入了哪个子网来为其分配 IP 地址，在设备还没有 IP 地址的时候，或者在分配 IP 的过程中。我们需要 MAC 地址来区分不同的设备。
- IP 地址可以比作为地址，MAC 地址为收件人，在一次通信过程中，两者是缺一不可的。

54.ICMP 协议的功能？

ICMP (Internet Control Message Protocol) ，网际控制报文协议。

- ICMP 协议是一种面向无连接的协议, 用于传输出错报告控制信息。
- 它是一个非常重要的协议, 它对于网络安全具有极其重要的意义。它属于网络层协议, 主要用于在主机与路由器之间传递控制信息, 包括**报告错误**、**交换受限控制**和**状态信息**等。
- 当遇到 IP 数据无法访问目标、IP 路由器无法按当前的传输速率转发数据包等情况时, 会自动发送 ICMP 消息。

比如我们日常使用得比较多的 **ping**, 就是基于 ICMP 的。

55.说下 ping 的原理?

ping, **Packet Internet Groper**, 一个网络工具, 主要用来测试网络连接的可达性和延迟。

```
~ (6.134s)
ping javabetter.cn
PING javabetter.cn (39.105.208.175): 56 data bytes
64 bytes from 39.105.208.175: icmp_seq=0 ttl=49 time=89.626 ms
64 bytes from 39.105.208.175: icmp_seq=1 ttl=49 time=86.178 ms
64 bytes from 39.105.208.175: icmp_seq=2 ttl=49 time=95.585 ms
64 bytes from 39.105.208.175: icmp_seq=3 ttl=49 time=89.828 ms
64 bytes from 39.105.208.175: icmp_seq=4 ttl=49 time=91.221 ms
^C
--- javabetter.cn ping statistics ---
6 packets transmitted, 5 packets received, 16.7% packet loss
round-trip min/avg/max/stddev = 86.178/90.488/95.585/3.042 ms
```

ping 二哥的 Java 进阶之路

Ping 的过程主要基于 ICMP (Internet Control Message Protocol, 互联网控制消息协议) 实现, 其基本过程包括:

- ①、当执行 Ping 命令, 如 `ping javabetter.cn`, Ping 首先解析域名获取 IP 地址, 然后向目标 IP 发送一个 ICMP Echo Request 消息。
- ②、当目标 IP 收到 ICMP Echo Request 消息后, 它会生成一个 ICMP Echo Reply 消息并返回, 即 Ping 响应消息。
- ③、发起 Ping 命令的设备接收到 ICMP Echo Reply 消息后, 计算并显示从发送 Echo Request 到接收到 Echo Reply 的时间 (通常称为往返时间 RTT, Round-Trip Time), 以及可能的丢包情况。

Ping 通常会发送多个请求, 以便提供平均响应时间和丢包率等信息, 以便我们了解网络连接的质量。

1. [Java 面试指南 \(付费\)](https://javabetter.cn/zhishixingqiu/mianshi.html) (<https://javabetter.cn/zhishixingqiu/mianshi.html>)_收录的快手面经同学 7 Java 后端技术一面面试原题: 说一下 Ping 的过程

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer) (<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了! 包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等, 共计 32 万余字, 500+张手绘图, 可以说是通俗易懂、风趣幽默.....详情戳: [太赞了, GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/) (<https://javabetter.cn/overview/>).

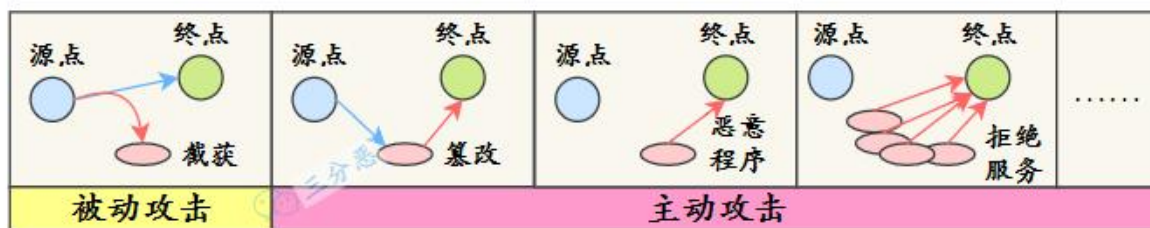
微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二, 回复 **222** 即可免费领取。



网络安全

56.说说有哪些安全攻击?

网络安全攻击主要分为两种类型, **被动攻击**和**主动攻击**:

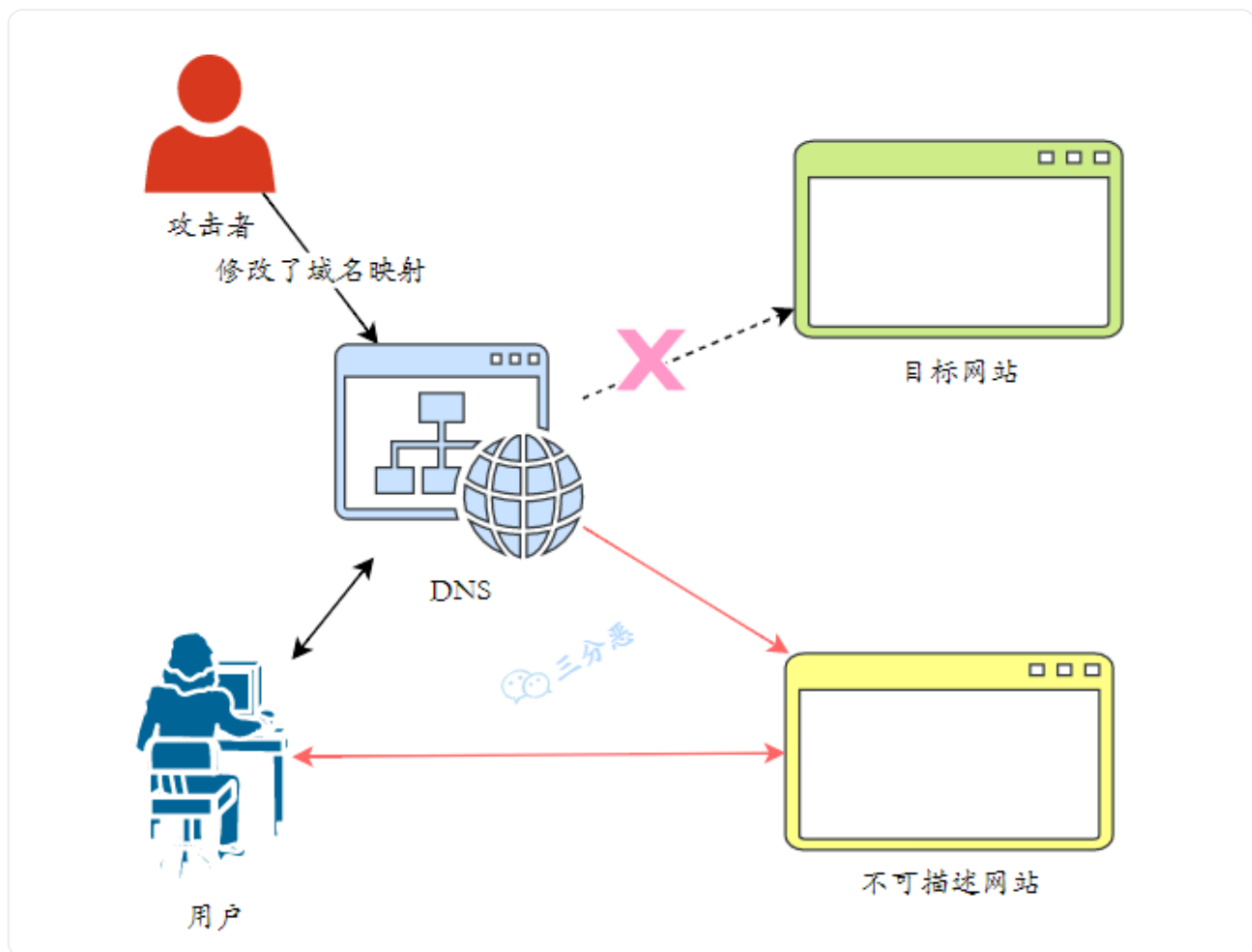


主动攻击和被动攻击

- **被动攻击**：是指攻击者从网络上窃听他人的通信内容，通常把这类攻击称为截获，被动攻击主要有两种形式：消息内容泄露攻击和流量分析攻击。由于攻击者没有修改数据，使得这种攻击很难被检测到。
- **主动攻击**：直接对现有的数据和服务造成影响，常见的主动攻击类型有：
 - **篡改**：攻击者故意篡改网络上送的报文，甚至把完全伪造的报文传送给接收方。
 - **恶意程序**：恶意程序种类繁多，包括计算机病毒、计算机蠕虫、特洛伊木马、后门入侵、流氓软件等等。
 - **拒绝服务 Dos**：攻击者向服务器不停地发送分组，使服务器无法提供正常服务。

57.DNS 劫持了解吗？

DNS 劫持即域名劫持，是通过将原域名对应的 IP 地址进行替换，从而使用户访问到错误的网站，或者使用户无法正常访问网站的一种攻击方式。



DNS 劫持示意图

域名劫持往往只能在特定的网络范围内进行，范围外的 DNS 服务器能够返回正常的 IP 地址。攻击者可以冒充原域名所属机构，通过电子邮件的方式修改组织机构的域名注册信息，或者将域名转让给其它主持，并将新的域名信息保存在所指定的 DNS 服务器中，从而使用户无法对原域名来进行解析以访问目标地址。

DNS 劫持的步骤是什么样的？

1. 获取要劫持的域名信息：攻击者会首先访问域名查询要劫持的站点的域名信息。
2. 控制域名响应的 E-Mail 账号：在获取到域名信息后，攻击者通过暴力破解或者专门的方法破解公司注册域名时使用的 E-mail 账号所对应的密码，更高级的攻击者甚至能够直接对 E-Mail 进行信息窃取。
3. 修改注册信息：当攻击者破解了 E-Mail 后，会利用相关的更改功能修改该域名的注册信息，包括域名拥有者信息，DNS 服务器信息等。
4. 使用 E-Mail 收发确认函：在修改完注册信息后，攻击者 E-Mail 在真正拥有者之前收到修改域名注册信息的相关确认信息，并回复确认修改文件，待网络公司恢复已成功修改信件后，攻击者便成功完成 DNS 劫持。

怎么应对 DNS 劫持？

- 直接通过 IP 地址访问网站，避开 DNS 劫持
- 由于域名劫持往往只能在特定的网络范围内进行，因此一些高级用户可以通过网络设置让 DNS 指向正常的域名服务器以实现对目标网址的正常访问，例如计算机首选 DNS 服务器的地址固定为 8.8.8.8。

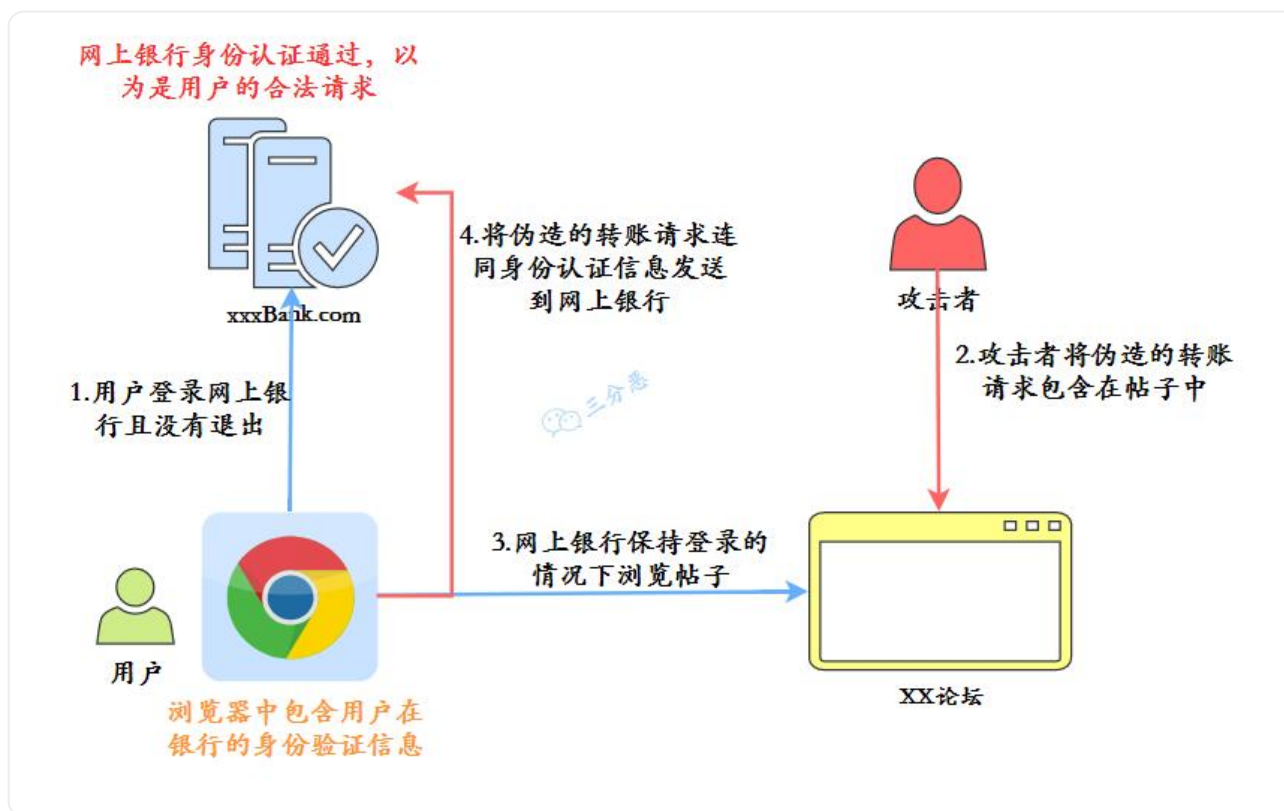
58.什么是 CSRF 攻击？如何避免？

什么是 CSRF 攻击？

CSRF，跨站请求伪造（英文全称是 Cross-site request forgery），是一种挟持用户在当前已登录的 Web 应用程序上执行非本意的操作的攻击方法。

CSRF 是如何攻击的呢？

来看一个例子：



CSRF 典型例子

1. 用户登陆银行，没有退出，浏览器包含了 用户 在银行的身份认证信息。
2. 攻击者将伪造的转账请求，包含在在帖子
3. 用户在银行网站保持登陆的情况下，浏览帖子
4. 将伪造的转账请求连同身份认证信息，发送到银行网站
5. 银行网站看到身份认证信息，以为就是 用户的合法操作，最后造成用户资金损失。

怎么应对 CSRF 攻击呢?

- 检查 Referer 字段

HTTP 头中的 Referer 字段记录了该 HTTP 请求的来源地址。在通常情况下, 访问一个安全受限页面的请求来自于同一个网站, 而如果黑客要对其实施 CSRF 攻击, 他一般只能在他自己的网站构造请求。因此, 可以通过验证 Referer 值来防御 CSRF 攻击。

- 添加校验 token

以在 HTTP 请求中以参数的形式加入一个随机产生的 token, 并在服务器端建立一个拦截器来验证这个 token, 如果请求中没有 token 或者 token 内容不正确, 则认为可能是 CSRF 攻击而拒绝该请求。

- 敏感操作多重校验

对一些敏感的操作, 除了需要校验用户的认证信息, 还可以通过邮箱确认、验证码确认这样的方式多重校验。

59.什么是 DoS、DDoS、DRDoS 攻击?



请求太多服务器着不住

- **DOS:** (Denial of Service), 翻译过来就是拒绝服务, 一切能引起拒绝 行为的攻击都被称为 DOS 攻击。最常见的 DoS 攻击就有**计算机网络宽带攻击**、**连通性攻击**。
- **DDoS:** (Distributed Denial of Service), 翻译过来是分布式拒绝服务。是指处于不同位置的多个攻击者同时向一个或几个目标发动攻击, 或者一个攻击者控制了位于不同位置的多台机器, 并利用这些机器对受害者同时实施攻击。

主要形式有流量攻击和资源耗尽攻击, 常见的 DDoS 攻击有: **SYN Flood**、**Ping of Death**、**ACK Flood**、**UDP Flood** 等。

- **DRDoS**: (Distributed Reflection Denial of Service), 中文是分布式反射拒绝服务, 该方式靠的是发送大量带有被害者 IP 地址的数据包给攻击主机, 然后攻击主机对 IP 地址源做出大量回应, 从而形成拒绝服务攻击。

如何防范 DDoS?

针对 DDoS 中的流量攻击, 最直接的方法是增加带宽, 理论上只要带宽大于攻击流量就可以了, 但是这种方法成本非常高。在有充足带宽的前提下, 我们应该尽量提升路由器、网卡、交换机等硬件设施的配置。

针对资源耗尽攻击, 我们可以升级主机服务器硬件, 在网络带宽得到保证的前提下, 使得服务器能够有效对抗海量的 SYN 攻击包。我们也可以安装专业的抗 DDoS 防火墙, 从而对抗 SYN Flood 等流量型攻击。瓷碗, 负载均衡, CDN 等技术都能有效对抗 DDos 攻击。

60.什么是 XSS 攻击, 如何避免?

XSS 攻击也是比较常见, XSS, 叫**跨站脚本攻击 (Cross-Site Scripting)**, 因为会与层叠样式表 (Cascading Style Sheets, CSS) 的缩写混淆, 因此有人将跨站脚本攻击缩写为 XSS。它指的是恶意攻击者往 Web 页面里插入恶意 html 代码, 当用户浏览网页的时候, 嵌入其中 Web 里面的 html 代码会被执行, 从而达到恶意攻击用户的特殊目的。

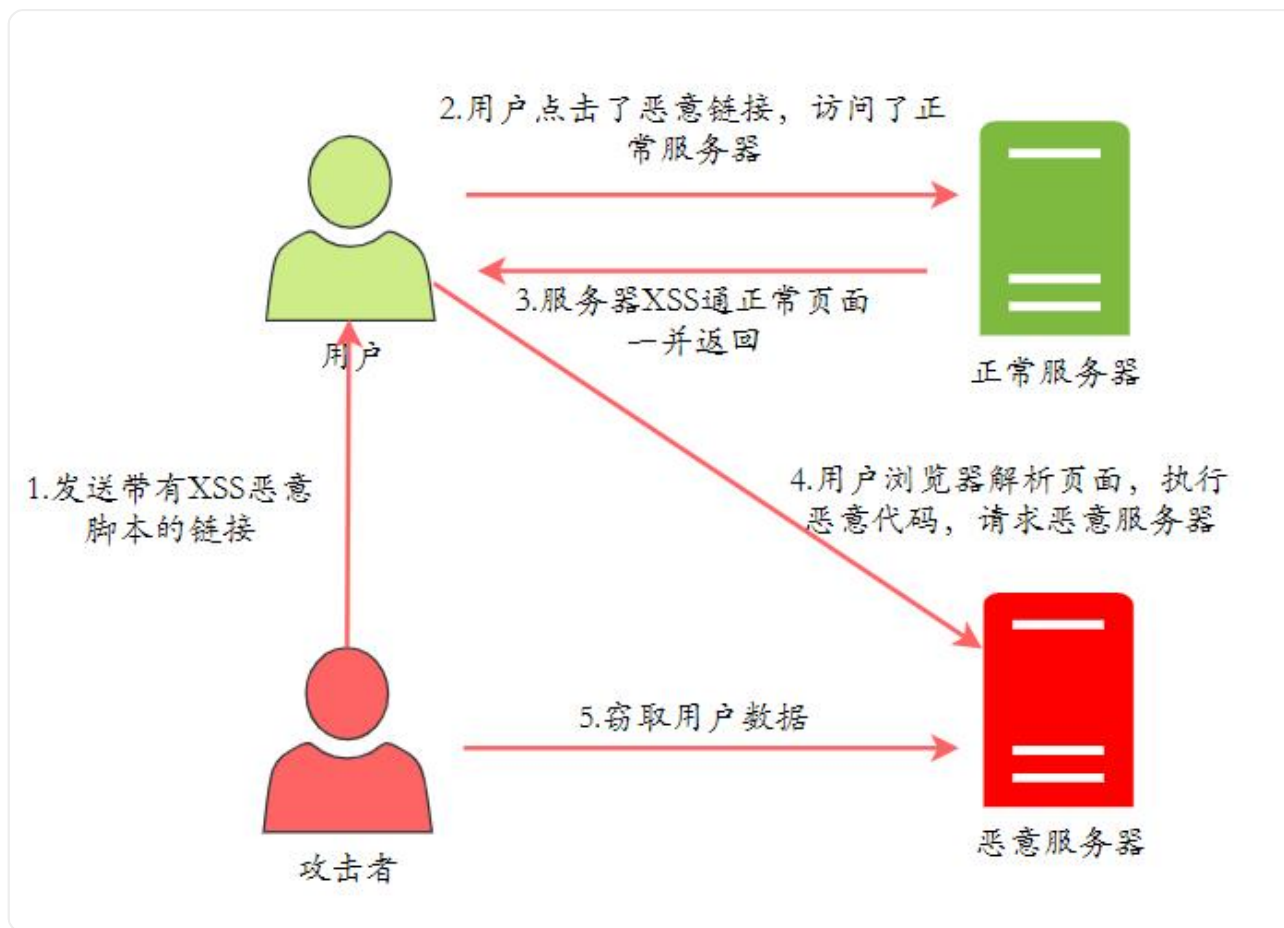
XSS 攻击一般分三种类型: **存储型**、**反射型**、**DOM 型 XSS**

XSS 是如何攻击的呢?

简单说, XSS 的攻击方式就是想办法“教唆”用户的浏览器去执行一些这个网页中原本不存在的前端代码。

拿反射型举个例子吧, 流程图如下:

1. 攻击者构造出特殊的 URL, 其中包含恶意代码。
2. 用户打开带有恶意代码的 URL 时, 访问正常网站服务器
3. 网站服务端将恶意代码从 URL 中取出, 拼接在 HTML 中返回给浏览器。
4. 用户浏览器接收到响应后解析执行, 混在其中的恶意代码也被执行, 请求恶意服务器, 发送用户数据
5. 攻击者就可以窃取用户的数据, 以此冒充用户的行为, 调用目标网站接口执行攻击者指定的操作。



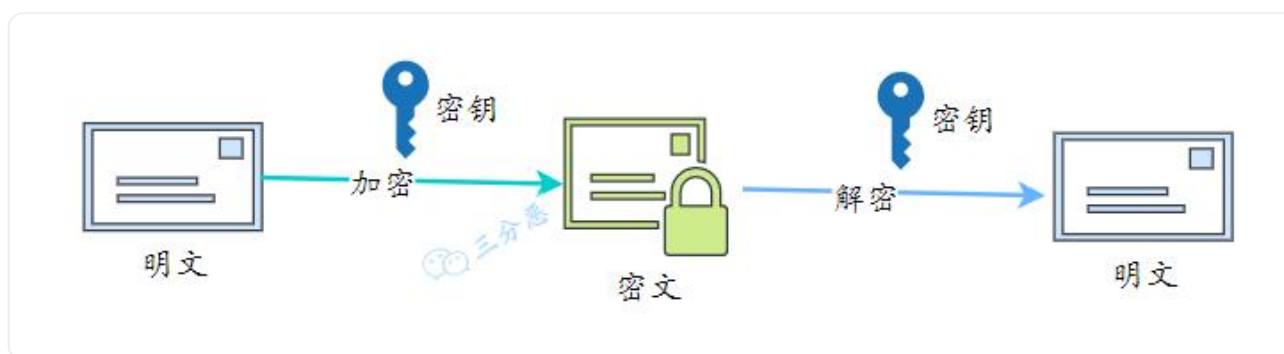
一个典型的 XSS

如何应对 XSS 攻击？

- 对输入进行过滤，过滤标签等，只允许合法值。
- HTML 转义
- 对于链接跳转，如 `<a href="xxx">` 等，要校验内容，禁止以 `script` 开头的非法链接。
- 限制输入长度

61. 对称加密与非对称加密有什么区别？

对称加密：指加密和解密使用同一密钥，优点是运算速度较快，缺点是如何安全将密钥传输给另一方。常见的对称加密算法有：DES、AES 等。



对称加密

非对称加密：指的是加密和解密使用不同的密钥（即公钥和私钥）。公钥与私钥是成对存在的，如果用公钥对数据进行加密，只有对应的私钥才能解密。常见的非对称加密算法有RSA。



非对称加密

62.RSA 和 AES 算法有什么区别？

• RSA

采用非对称加密的方式，采用公钥进行加密，私钥解密的形式。其私钥长度一般较长，由于需要大数的乘幂求模等运算，其运算速度较慢，不合适大量数据文件加密。

• AES

采用对称加密的方式，其秘钥长度最长只有 256 个比特，加密和解密速度较快，易于硬件实现。由于是对称加密，通信双方在进行数据传输前需要获知加密密钥。

图文详解 63 道计算机网络面试高频题，这次吊打面试官，我觉得稳了（手动 dog）。整理：沉默王二，戳[转载链接 \(https://mp.weixin.qq.com/s/FvxyiMyq0422yifcyoG8vg\)](https://mp.weixin.qq.com/s/FvxyiMyq0422yifcyoG8vg)，作者：三分恶，戳[原文链接 \(https://mp.weixin.qq.com/s/yAlErIC09GnjaVvwUo3Acg\)](https://mp.weixin.qq.com/s/yAlErIC09GnjaVvwUo3Acg)。

没有什么使我停留——除了目的，纵然岸旁有玫瑰、有绿荫、有宁静的港湾，我是不系之舟。

系列内容：

- [面渣逆袭 Java SE 篇👉 \(https://javabetter.cn/sidebar/sanfene/javase.html\)](https://javabetter.cn/sidebar/sanfene/javase.html)
- [面渣逆袭 Java 集合框架篇👉 \(https://javabetter.cn/sidebar/sanfene/javathread.html\)](https://javabetter.cn/sidebar/sanfene/javathread.html)

- [面渣逆袭 Java 并发编程篇 🙌](https://javabetter.cn/sidebar/sanfene/collection.html)
(<https://javabetter.cn/sidebar/sanfene/collection.html>)
- [面渣逆袭 JVM 篇 🙌](https://javabetter.cn/sidebar/sanfene/jvm.html) (<https://javabetter.cn/sidebar/sanfene/jvm.html>)
- [面渣逆袭 Spring 篇 🙌](https://javabetter.cn/sidebar/sanfene/spring.html) (<https://javabetter.cn/sidebar/sanfene/spring.html>)
- [面渣逆袭 Redis 篇 🙌](https://javabetter.cn/sidebar/sanfene/redis.html) (<https://javabetter.cn/sidebar/sanfene/redis.html>)
- [面渣逆袭 MyBatis 篇 🙌](https://javabetter.cn/sidebar/sanfene/mybatis.html) (<https://javabetter.cn/sidebar/sanfene/mybatis.html>)
- [面渣逆袭 MySQL 篇 🙌](https://javabetter.cn/sidebar/sanfene/mysql.html) (<https://javabetter.cn/sidebar/sanfene/mysql.html>)
- [面渣逆袭操作系统篇 🙌](https://javabetter.cn/sidebar/sanfene/os.html) (<https://javabetter.cn/sidebar/sanfene/os.html>)
- [面渣逆袭计算机网络篇 🙌](https://javabetter.cn/sidebar/sanfene/network.html) (<https://javabetter.cn/sidebar/sanfene/network.html>)
- [面渣逆袭 RocketMQ 篇 🙌](https://javabetter.cn/sidebar/sanfene/rocketmq.html) (<https://javabetter.cn/sidebar/sanfene/rocketmq.html>)
- [面渣逆袭分布式篇 🙌](https://javabetter.cn/sidebar/sanfene/fenbushi.html) (<https://javabetter.cn/sidebar/sanfene/fenbushi.html>)
- [面渣逆袭微服务篇 🙌](https://javabetter.cn/sidebar/sanfene/weifuwu.html) (<https://javabetter.cn/sidebar/sanfene/weifuwu.html>)
- [面渣逆袭设计模式篇 🙌](https://javabetter.cn/sidebar/sanfene/shejimoshi.html) (<https://javabetter.cn/sidebar/sanfene/shejimoshi.html>)
- [面渣逆袭 Linux 篇 🙌](https://javabetter.cn/sidebar/sanfene/linux.html) (<https://javabetter.cn/sidebar/sanfene/linux.html>)

GitHub 上标星 10000+ 的开源知识库《[二哥的 Java 进阶之路](https://github.com/itwanger/toBeBetterJavaer)

(<https://github.com/itwanger/toBeBetterJavaer>)》第一版 PDF 终于来了! 包括 Java 基础语法、数组&字符串、OOP、集合框架、Java IO、异常处理、Java 新特性、网络编程、NIO、并发编程、JVM 等等, 共计 32 万余字, 500+张手绘图, 可以说是通俗易懂、风趣幽默.....详情戳: [太赞了, GitHub 上标星 10000+ 的 Java 教程](https://javabetter.cn/overview/)
(<https://javabetter.cn/overview/>)

微信搜 **沉默王二** 或扫描下方二维码关注二哥的原创公众号沉默王二, 回复 **222** 即可免费领取。



上次编辑于: 2024/6/20 13:01:20

贡献者: itwanger,沉默王二

Copyright © 2024 三分恶